

2026 年高教社杯全国大学生数学建模竞赛

A 题 药材的烘干问题

摘要

本文针对长 25 cm、半径 2 cm 的圆柱形药材,建立轴对称一维径向热湿耦合偏微分方程模型:傅里叶导热定律描述热量传递,干基菲克扩散本构描述水分迁移,由质量守恒导出含水率演化方程;第四问的收缩几何用材料坐标变换映射为固定计算域。空间离散用守恒型节点对偶有限体积与 Kirchhoff 浓度势(精确积分强非线性含水率),时间推进用变阶变步长隐式 BDF/NDF 与解析稀疏 Jacobian,达标时刻由空间最大含水率的阈值事件定位。

四问共用该模型,仅切换物性与判据:问题一取附录 2 常热物性、热湿解耦;问题二全程用附录 3 变物性、双向耦合;问题三复用问题二的解,以"空间最大含水率严格低于 0.15 kg/kg"求烘干时长;问题四改用附录 4 物性并叠加附件 2 的半径收缩。

主要结果:(1)1800 s 时表面温度 36.7856 °C、中心 33.5753 °C,表面含水率由 2.5500 降至 1.5102 kg/kg,距中心 1 cm 以内基本不变,即 30 min 内水分变化仅局限于表层;热、水扩散率之比约 34.2。(2)3 h 时表面温度 49.9664 °C、中心与表面仅差 0.1169 °C,含水率仍相差 0.7581 kg/kg,已进入内扩散控制的恒温干燥阶段。(3)附录 3 物性、固定半径下,全域含水率首次严格低于 0.15 kg/kg 的时间为 57.4724 h(约 2.39 天)。(4)计入附件 2 半径收缩并用附录 4 物性后为 51.0906 h(约 2.13 天);同物性对照表明收缩使达标时间由约 129.85 h 缩至约 51.09 h(约 60.7%),故两问时长差不能全部归因于收缩。

模型修正与创新:题给密度经验式与“给定半径轨迹 + 固定长度 + 严格干物质守恒”不能同时精确成立,本文给出有效热物性闭合;表面汽化耗能按潜热比例作情景包络;水分通量用 Kirchhoff 浓度势处理;移动边界用材料坐标与干骨架守恒处理;并由附件 1 湿度观测反推水活度锚点,构造吸附闭合情景族。

稳健性与检验:达标时间由两族隐式积分器(BDF/NDF、Radau IIA)与两种水通量面离散格式在三档网格上一致到 10^{-6} s,阈值事件两种定位法相差 5.8×10^{-11} s,收缩使时长缩短 60.7% 由 $(R_0/R(t))^2$ 尺度折算印证到 1.37% 以内,三阶段分界由四种判据识别于同一区间,故 57.4724 h 与 51.0906 h 非特定网格、求解器或判据的产物。解析解对照最大温度偏差(节点口径) 1.80×10^{-6} K,离散质量守恒式残差约 10^{-14} kg/kg。题目未提供内部实测,结果为所列假设下的条件性预测,不对准确率、实测误差或时长置信区间作声明;4 h 后的环境历程占报告时长的 92%—93%,其平台延拓是主要不确定性来源。

关键词:热湿耦合;圆柱非稳态扩散;有限体积法;材料坐标变换;全域达标时间

1 问题重述

1.1 问题背景

中药材热风烘干含预热平衡与恒温干燥两个阶段,参数选取不当会造成效率低、能耗高、品质不稳定;用数值仿真揭示内部温度与水分的演化规律有实际意义。[1]

1.2 问题提出

药材长 25 cm、半径 2 cm,初始温度 28 °C、干基含水率 2.55 kg/kg;烘房环境由附件 1 给出,物性经验式问题一见附录 2、问题二三见附录 3、问题四见附录 4。

问题一(预热平衡阶段) 建立预热平衡阶段的温度与水分浓度模型(参数见附录 2);按表 1、表 2 格式给出 100、300、600、900、1200、1500、1800 s 共 7 个时刻、到中心距离 0、0.5、1、1.5、2 cm 共 5 个位置的温度(°C)与水分浓度(kg/kg),1800 s 内**每隔 1 s、每隔 0.1 cm**的完整结果存入`result1.xlsx`。

问题二(全过程热湿耦合) 全过程 2—3 天且两阶段参数不同,经验公式统一采用附录 3;按表 3、表 4 格式给出 3 h 内每隔 0.5 h、径向 0、0.5、1、1.5、2 cm 处的结果,完整结果按**每隔 1 s、每隔 0.1 cm**存入`result2.xlsx`。

问题三(全域达标与烘干时长) 要求各处水分浓度低于 0.15 kg/kg,据此求烘干时间(h);按表 5 格式给出每隔 6 h、径向每隔 0.5 cm 的水分浓度,完整结果按**每隔 60 s、每隔 0.1 cm**存入`result3.xlsx`。

问题四(尺寸变化下的烘干时长) 依据附件 2 的半径变化数据与附录 4 经验公式建立含尺寸变化的模型并重新确定烘干时长;按表 6 格式给出每隔 6 h、径向每隔 0.5 cm 的水分浓度,完整结果按**每隔 60 s、每隔 0.1 cm**存入`result4.xlsx`。

结果输出要求 正文按表 1—表 6 格式给出结果,完整结果存入`result1.xlsx`—`result4.xlsx`(模板见附件 3),所有结果保留四位小数;四问的输入输出要求见表 7。

表 7 四个问题的输入条件、模型增量与输出要求

问题	输入与模型增量	正文需给出的结果	结果文件
问题一	圆柱尺寸与均匀初值;附件 1 环境温度湿度;附录 2 物性	1800 s 内 7 个时刻 × 5 个半径的温度(表 9)与水分浓度(表 10)	`result1.xlsx`
问题二	同一初值;经验公式改为附录 3,物性随含水率与温度变化,覆盖全过程 2—3 天	3 h 内每隔 0.5 h × 5 个半径的温度(表 15)与水分浓度(表 16)	`result2.xlsx`
问题三	沿用问题二的模型,施加"各处水分浓度 < 0.15 kg/kg"的达标判据	每隔 6 h、每隔 0.5 cm 的水分浓度(表 20)及烘干所需时间	`result3.xlsx`
问题四	改用附录 4 物性;由附件 2 引入半径随时间的变化	每隔 6 h、每隔 0.5 cm 的水分浓度(表 22)及新的烘干时长	`result4.xlsx`

2 问题分析

2.1 总体性质与主要难点

本题为传热传质机理建模问题: $T(r,t)$ 、 $C(r,t)$ 由傅里叶导热定律与菲克扩散定律驱动,经表面对流换热、对流传质与烘房环境耦合;四问核心都是**偏微分方程定解问题**,问题三、四为**阈值事件定位**(空间最大含水率首次严格低于 0.15 kg/kg 的时刻)。难点有四:(1)**热湿时间尺度差一至两个数量级**——热扩散约 0.7—1.4 h、水分扩散约 20—106 h,刚性比 25—75,须隐式推进;(2)**两阶段不能由一条常系数模型贯穿**——全过程 2—3 天且参数不同,需变物性统一模型;(3)**问题一可解耦、问题二起必须联立**;(4)**问题四求解域随时间缩小且环境数据仅覆盖 4 h**(半径由 2 cm 收缩至约 1.2 cm)。

2.2 数据条件与数据边界

附件给出的是边界驱动与几何,不是药材内部的温度或含水率观测,故本文结果为明确假设下的条件性预测(表 8)。

表 8 数据条件与模型可用边界

数据	内容	覆盖范围	可直接支撑	必须额外引入的约定
附件 1	烘房温度与水分浓度,241 条	0—4 h,每 60 s	已记录时刻的环境边界	点间线性插值;4 h 后按平台延拓
附件 2	药材表面半径,145 条	0—72 h,每 1800 s	表面位置 $R(t)$	点间线性插值;内部"同比收缩"
附录 2—4	物性与扩散系数经验式共 9 条	全时段	给定状态下的系数	无独立系数误差、无拟合区间

附件 3	结果模板	—	输出格式(时间为行、距离为列)	—
未提供	内部温度、内部含水率、称重曲线、长度变化、吸附等温线	—	不能直接验证内部响应	内部场全部由模型预测

由此得三条结论:① 问题三、四的报告时长分别有 **93.0%** 与 **92.2%** 落在附件 1 的 4 h 观测窗之外;② 问题四的达标时刻 51.09 h 在附件 2 的 72 h 观测范围内,不需半径外推;③ 附件 1 水分浓度的参考质量基准未注明(空气干基与药材干基分母不同),将其视为等效平衡含水率是**显式约定**,非物理等价。

2.3 建模的定量依据:量级分析先行

取 $C_0=2.55$ 、 $T_0=301.15\text{ K}$ 、 $R_0=0.02\text{ m}$,定义 $\alpha=k/(\rho_{\text{eff}}c_p)$ 、 D 、 $Bi_h=hR_0/k$ 、 $Bi_m=\beta R_0/D$,结果见表 12(第 5 节)。① **$Bi_h \sim 1$** :不能近似为瞬时等温的集中参数体,须解内部温度梯度,故用 PDE 模型;② **$\alpha/D \approx 26\text{—}75$** :热量传递远快于水分迁出;③ **水分扩散特征时间 19.7—106.1 h** 与烘干时长同量级:过程由内部水分扩散控制。[12]

2.4 环境驱动的可用性

问题二至四的时间跨度远大于附件 1 的观测窗,故约定:0—4 h 按 60 s 观测点分段线性插值,4 h 后取末段平台(50 °C 与 0.05 kg/kg)作基准延拓,并比较末值保持、末段均值保持与平台扰动情景;插值是驱动条件的**重采样**而非新观测,平台延拓是**模型假设**(第 13 节)。附件 1 末段(3—4 h)已接近平台,末点 50.165 °C 与 0.04986 kg/kg,拼接跳变仅 -0.165 °C 与 +0.00014 kg/kg。

2.5 问题一:可解耦的短时基线

问题一为常热物性,温度方程可独立求解,含水率方程只通过 $D(C)$ 非线性;固定半径、常物性、恒定环境下圆柱温度场存在 **Bessel 级数解析解**,可用于检验离散求解器,故问题一定位为"可验证的短时基线"。

2.6 问题二:从解耦到双向耦合

问题二覆盖全过程(2—3 天),经验公式统一采用附录 3。此时 ρ 、 c_p 、 k 依赖 C , D 依赖 C 与 T ,两方程互为输入、必须联立求解; D 的温度项为 Arrhenius 形式,温度须换算为开尔文代入。

2.7 问题三:全域判据与阈值事件

问题三在问题二的解上施加烘干判据。因要求"药材各处水分浓度均低于 0.15 kg/kg",须取**空间最大值**判定(式 (77)),不能用中心点值或平均值代替(后期中心与表面可相差近三倍);且"严格小于"与四位小数显示存在口径问题(显示 0.1500 不能证明严格达标),须以**未舍入**判定值为准。

2.8 问题四:移动边界与收缩净效应

求解域 $0 \leq r \leq R(t)$ 随时间缩小,属**移动边界问题**。本文取材料坐标 $x=r/R(t)$ 把移动域映射为固定域 $[0,1]$, $x=1$ 始终是真实表面,几何收缩以 $1/R(t)^2$ 进入扩散算子,即半径减小会加快内部迁移(第 5 节式 (26)—(28))。两处易重复计入的项须注意:**不应加体积浓缩项**(C 为水与干物质之比),**不应再叠加网格平流项**(已用材料坐标)。问题四相对问题三同时改变物性与几何,故补充"同附录 4 物性、固定半径与收缩"的对照(第 9.5 节)。

2.9 总体建模思路与技术路线

本文建立统一的轴对称一维径向热湿耦合模型:

$$B(C) \frac{\partial T}{\partial t} = \frac{1}{R(t)^2 x} \frac{\partial}{\partial x} \left(x k(C) \frac{\partial T}{\partial x} \right), \quad (1)$$

式 (1) 中 $x=r/R(t)$ 为材料径向坐标, $R(t) \equiv R_0$ 时退化为固定半径形式,故该式为四问通用控制方程。四问只在同一框架内切换**物性来源**(附录 2→3→4)、**几何**(固定半径或 $R(t)$)与**判据**(是否施加 $M(t)<0.15$);求解用守恒型有限体积配合隐式时间推进,验证按"解析基准 → 离散守恒 → 收敛 → 跨实现对照"分层报告(表 13、图 1)。

表 13 四问的模型切换与技术路线

问题	物性	几何	求解与判据	验证要点
一	附录 2(常热物性, D 仅依赖 C)	固定 R_0	直接输出 T, C	温度可解耦;圆柱 Bessel 解析解校核
二	附录 3(依赖 C 与 T)	固定 R_0	直接输出 T, C	双向耦合联立隐式求解;离散守恒校核
三	同问题二	固定 R_0	复用问题二的解,施加全域最大含水率阈值事件	逐时检查最大值位置;独立二分求根互验
四	附录 4	移动域 $R(t)$ (附件 2)	同问题三	材料坐标变换;同物性固定/收缩对照



图 1 总体建模思路与技术路线:四个问题共享“圆柱一维径向传热传质框架 + 中心零通量 + 表面对流与传质边界 + 质量守恒校验 + 收敛性检验”基座,依次放开物性、引入达标判据、引入移动边界;框内列出每问的难点、处理与对应结果文件。

3 数据说明与数据审计

3.1 数据来源与字段说明

原始数据为附件 1、2 与附录 2—4 的物性经验式;附件 3 工作簿仅用于确定输出格式。[1]

附件 1(烘房环境):241 条,0—14400 s、间隔 60 s,字段为时间(s)、温度(°C)与水分浓度(kg/kg)。

附件 2(药材半径):145 条,0—259200 s(0—72 h)、间隔 1800 s,半径由 2.000 cm 单调减至约 1.198 cm。

附录 2—4:物性经验式共 9 条(问题一 1 条,问题二三 4 条,问题四 4 条); $h=25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 与 $\beta=8 \times 10^{-7} \text{ m/s}$ 仅附录 2 给出,按"同一药材、同一烘房"沿用(A07)。

3.2 数据审计过程与结果

六方面核查结果见表 14,其中三项须在正文说明:① **单位与量纲**——秒、摄氏度(入 Arrhenius 项前换算为开尔文)、厘米(入方程前换算为米)、干基 kg/kg,未对原始数据作任何剔除或平滑;② **数据边界**

——问题三报告时长 57.4724 h 中有 $57.4724 - 4 = 53.4724$ h 在附件 1 观测窗之外(93.04%),问题四 51.0906 h 中有 $51.0906 - 4 = 47.0906$ h 在窗外(92.17%);问题四报告时刻半径 1.2000 cm、时间 $51.0906\text{ h} < 72\text{ h}$,未作半径外推;③ 数据性质——两附件均为驱动条件,题目未提供内部温度、含水率、称重曲线、长度变化与吸附等温线,故无计算实测误差的参照真值。

表 14 数据审计要点汇总

项目	附件 1	附件 2	结论
记录条数	241	145	合计 386 条
非时间观测值	482	145	合计 627 个
时间范围	0—14400 s	0—259200 s	分别为 0—4 h 与 0—72 h
采样间隔	60 s	1800 s	恒定,无跳变
取值范围	28—50.246 °C;0.01963—0.05025 kg/kg	2.000—1.198 cm	均落在合理工况内
缺失/重复/错误	0	0	无需插补或删除
单调性	非单调(升温后趋平台)	严格非增	与物理过程一致

3.3 数据的可视化说明

附件 1、附件 2 的原始观测见图 2、图 3;按 60 s 观测点作分段线性插值只是驱动条件的重采样,不增加独立实测样本。

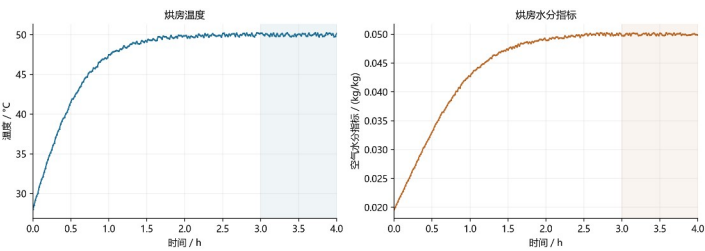


图 2 附件 1 烘房环境观测:横轴为时间(h),纵轴为烘房温度(°C)与水分浓度(kg/kg);观测覆盖 0—4 h(241 条、间隔 60 s),其后为模型采用的平台延拓。

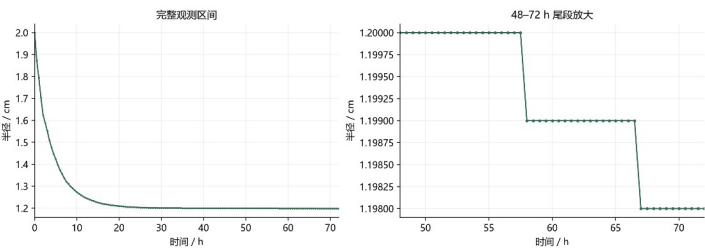


图 3 附件 2 药材半径观测:横轴为时间(h),纵轴为药材半径(cm);观测覆盖 0—72 h(145 条、间隔 1800 s),半径由 2.000 cm 单调降至约 1.198 cm。

4 模型假设与符号说明

4.1 模型假设

七条假设均按"假设内容 → 依据 → 影响与边界 → 检验"四步说明。

A01(几何与维度) 轴对称一维径向场; $L=0.25\text{ m}$ 固定,问题一至三取 $R_0=0.02\text{ m}$ 。

- *依据*:题目按"到药材中心的距离"给出结果;长径比 $L/(2R_0)=6.25$,径向特征长度远小于轴向。

- *影响与边界*: "有限长"不等于端面绝热;面积比 $R_0/L=0.08$ 是几何比值而非 8% 误差上界;端面效应被忽略。
- *检验*: 二维轴对称对照仪支持"本组参数下端面影响较小"(第 13.6 节)。

A02(能量闭合) 热方程取有效显热容量闭合 $B(C)=\rho_{\text{eff}}c_p$;基线不含相变潜热,也不计水分迁移携带的焓。

- *依据*: 题目未给相变焓、含水率—焓关系与气固相平衡关系。
- *影响与边界*: 这是最主要的物理选择:取 $L_v \approx 2.4 \text{ MJ/kg}$,若失水全在表面汽化,潜热与升温 22 K 的显热之比约 21.59(问题二/三)、21.73(问题四),二者同量级甚至更大,不能以"潜热很小"论证省略;故第 11.2 节按情景包络处理。
- *检验*: 给出 $L \in 0, 0.25, 0.5, 1$ 四点包络; $L=1$ 等价于全部排水在表面汽化且无吸附约束,仅自由水面可维持该表面温度。

A03(水分状态变量与骨架守恒) C 为干基含水率(kg 水/kg 干物质);干骨架守恒,水分通量取干基有效菲克本构 $j_w = -\rho_d D \partial C / \partial r$;问题一至三取均匀固定骨架,问题四取均匀同比径向收缩骨架。

- *依据*: 题目给出"水分浓度(即干基含水率)"的定义;干物质守恒与给定半径轨迹可相容确定骨架运动。
- *影响与边界*: ρ_{d0} 只决定总水质量的单位尺度,骨架均匀时不影响 C 分布;把题给 $\rho(C)$ 当作守恒的湿物料总密度将与均匀骨架冲突(第 5.1 节)。
- *检验*: 以干物质加权的离散质量恒等式诊断,残差约 10^{-14} kg/kg (第 10.9 节)。

A04(表面水分边界) 取烘房水分浓度作为表面等效平衡含水率,传质取线性 Robin 条件 $-D \partial C / \partial r|_s = \beta(C_s - C_{eq})$ 。

- *依据*: 题目给环境水分浓度(kg/kg)与 $\beta(\text{m/s})$,未给该药材的吸附/解吸等温线。
- *影响与边界*: 空气干基的分母是"每 kg 干空气"、材料干基是"每 kg 干物质",分母不同,单位文字相同不构成物理等价;本假设是使边界可算的显式简化约定。
- *检验*: 对 C_{eq} 映射作比例与偏移敏感性;第 11.4 节由附件 1 湿度观测反推相对湿度,构造数据锚定的吸附闭合情景族;缺吸附数据时真实映射无法唯一恢复。

A05(环境驱动与延拓) 0—14400 s 按附件 1 分段线性插值; $t > 14400 \text{ s}$ 基准取 $T_\infty = 323.15 \text{ K}$ (50 °C)、 $C_{eq} = 0.05 \text{ kg/kg}$ 平台。

- *依据*: 附件 1 末段(3—4 h)已接近平台,末点 50.165 °C 与 0.04986 kg/kg,拼接跳变仅 -0.165 °C 与 +0.00014 kg/kg。
- *影响与边界*: 50 °C/0.05 是候选工况而非其后数十小时的实测值,且本模型不给统计置信区间;本假设是主要不确定性来源。
- *检验*: 比较"末值保持""末 1 h 均值保持""平台温湿扰动"情景;尾段取 49 °C 与 51 °C 时问题二/三临界时间分别变化 +3.25% 与 -3.11%(第 13.2 节)。

A06(收缩材料运动) 问题四由附件 2 的 $R(t)$ 定表面位置,内部取同比径向收缩 $u=rR'/R$ 、长度固定;以 $x=r/R(t)$ 为材料坐标,网格随材料运动。

- *依据*:这是与表面速度、空间均匀干骨架守恒相容的**最简闭合**;材料导数中两项恰好抵消。
- *影响与边界*:附件**只给表面半径**,无内部位移、长度或速度场观测,无法证明内部收缩均匀;坐标变换不构成物理证据。
- *检验*:检查 R 插值保持正性与非增;给出同物性"固定半径 vs 收缩"对照,以独立 $(R_0/R)^2$ 折算互验(残差 1.37%,第 12.4 节)。

A07(边界系数的沿用) 问题二至四沿用 $h=25\text{ W}/(\text{m}^2\cdot\text{K})$ 、 $\beta=8\times 10^{-7}\text{ m/s}$ 。

- *依据*:附录 3、4 未重给这两个系数;研究对象与烘房装置未变。
- *影响与边界*:风速或温湿工况变化时系数可能改变,当前无实测记录。
- *检验*:对 h 、 β 各作 $\pm 10\%$ 、 $\pm 20\%$ 情景,报告为**设置差异而非实测误差**(第 13.3 节)。

4.2 符号说明

表 25 符号说明

符号	含义	单位	说明
t	时间	s	正文显示用 h 时除以 3600
r	到药材中心的径向距离	m	输入为 cm,须乘 10^{-2}
$x=r/R(t)$	材料归一化径向坐标	1	$x=1$ 恒为真实表面;固定 x 与固定 cm 不同
z	轴向坐标(仅二维对照用)	m	一维模型中不出现
T	药材温度	K	初值 301.15 K;摄氏显示为 $T-273.15$
C	干基含水率 $=m_w/m_d$	kg/kg	初值 2.55;允许大于 1
$w=C/(1+C)$	湿基质量分数	1	仅用于代入题给经验式
$R(t)$	药材当前半径	m	$R_0=0.02\text{ m}$;问题四由附件 2 插值
R_0, L	初始半径、药材长度	m	0.02 m、0.25 m
R'	半径变化率 dR/dt	m/s	由附件 2 的插值折线给出
$u(r, t)$	骨架径向速度	m/s	问题四取 $u=rR'/R$
ρ_{eff}	有效密度 ,仅用于热容量	kg/m ³	问题一 820;问题二三 $650+128C$;问题四 $760+90C$
ρ_d	守恒干物质密度	kg/m ³	另一个量,全文与 ρ_{eff} 严格分开
c_p	比热容	J/(kg·K)	题给经验式
k	导热系数	W/(m·K)	题给经验式
D	有效水分扩散系数	m ² /s	题给经验式,依赖 C (问题二三、四还依赖 T)

$B = \rho_{\text{eff}} c_p$	有效体积显热容量	J/(m ³ ·K)	随 C 变化
$\alpha = k / (\rho_{\text{eff}} c_p)$	热扩散率	m ² /s	由 k 、 ρ_{eff} 、 c_p 导出,非题给
h	对流换热系数	W/(m ² ·K)	25
β	对流传质系数	m/s	8×10^{-7}
q_r	径向导热热流密度	W/m ²	取"沿 r 增大方向(向外)为正"
j_w	径向水分质量通量	kg/(m ² ·s)	取"向外为正"
$\ell(t)$	每单位初始干物质的累计失水	kg/kg	初始为 0;用于守恒诊断
T_{∞}, C_{eq}	烘房温度、等效平衡含水率	K、kg/kg	来自附件 1(等效闭合)
t_*	全域达标时刻	s	正文按 h 报告
$M(t)$	空间最大干基含水率	kg/kg	达标判据的被控量
N	空间区间数	1	节点数为 $N+1$
L_v	水的汽化潜热	J/kg	取 2.4×10^6 ,仅用于量级与情景
a_w	水活度	1	仅在第 11.4 节的闭合情景中出现

提醒: ρ_{eff} (有效密度,仅用于 $B = \rho_{\text{eff}} c_p$)与 ρ_d (干物质密度)是两个不同的量,全文不得混用,也不得由 $\rho_{\text{eff}}/(1+C)$ 反推并同时用于两处。

5 统一物理模型

5.1 几何与定义

5.1.1 薄壳几何

用途:轴对称降维;自变量: r ;因变量: dV ;单位:m³、m²;物理含义:面积体积线性增长。

$$dV = \pi [(r + dr)^2 - r^2] L = 2\pi r L dr + O(dr^2) \quad (2)$$

5.1.2 干湿基含水率

用途:定义状态变量;自变量: m_w 、 m_d ;因变量: C 、 w ;单位:kg/kg;物理含义: w 仅用于经验式。[3]

$$C = \frac{m_w}{m_d}, \quad w = \frac{m_w}{m_d + m_w} = \frac{C}{1+C}, \quad C = \frac{w}{1-w} \quad (3)$$

判据 $C = 0.15$ 即湿基 13.04%,不是 15%。

5.1.3 初始干密度

用途: C 与水质量密度换算;自变量: C_0 ;因变量: ρ_{d0} ;单位:kg/m³;物理含义: ρ_d 由守恒定。

$$\rho_{d0} = \frac{\rho_{\text{eff}}(C_0)}{1+C_0} \Rightarrow \begin{array}{l} \text{问题一: } 820/3.55 = \\ \text{问题二、三: } (650 + \\ \text{问题四: } (760 + 90 \times \end{array} \quad (4)$$

5.2 守恒方程

5.2.1 干物质守恒

用途:约束骨架运动;自变量: t 、 r ;因变量: ρ_d 、 u ;单位: $\text{kg}/(\text{m}^3 \cdot \text{s})$;物理含义:随骨架输运变化。

$$\frac{\partial \rho_d}{\partial t} + \frac{1}{r} \frac{\partial}{\partial r} (r \rho_d u) = 0 \quad (5)$$

5.2.2 水分守恒

用途:水的守恒;自变量: t 、 r ;因变量: j_w ;单位: $\text{kg}/(\text{m}^3 \cdot \text{s})$;物理含义:输运与迁移之和。

$$\frac{\partial}{\partial t} (\rho_d C) + \frac{1}{r} \frac{\partial}{\partial r} [r (\rho_d C u + j_w)] = 0 \quad (6)$$

5.2.3 含水率方程

用途:消去骨架速度;单位: $1/\text{s}$;物理含义:随体变化率=扩散净流入。式 (6) 减 C 倍式 (5):

$$\rho_d \left(\frac{\partial C}{\partial t} + u \frac{\partial C}{\partial r} \right) = - \frac{1}{r} \frac{\partial}{\partial r} (r j_w) \quad (7)$$

5.3 本构关系

5.3.1 傅里叶定律

用途:温度梯度与热流;自变量: $\partial T / \partial r$;因变量: q_r ;单位: W/m^2 ;物理含义:取向外为正。[2]

$$q_r = -k \frac{\partial T}{\partial r} \quad (8)$$

5.3.2 菲克本构

用途:含水率梯度与通量;因变量: j_w ;单位: $\text{kg}/(\text{m}^2 \cdot \text{s})$;物理含义:水由高向低迁移。[4]

$$j_w = -\rho_d D \frac{\partial C}{\partial r} \quad (9)$$

5.4 主控方程

5.4.1 热方程

用途:内部温度演化;自变量: t 、 r ;因变量: T ;单位: K/s ;物理含义:热容量×升温率=净流入热流。[2]

$$B(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k(C) \frac{\partial T}{\partial r} \right), \quad (10)$$

5.4.2 水分方程

用途:含水率演化;因变量: C ;单位: $1/\text{s}$ 。[13]

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, T) \frac{\partial C}{\partial r} \right) \quad (11)$$

5.4.3 不能移出散度

用途:须保留散度形式;单位: K/s ;物理含义: k 变化引入梯度驱动项。

$$\frac{1}{r} \frac{\partial}{\partial r} \left(rk \frac{\partial T}{\partial r} \right) = k \frac{\partial^2 T}{\partial r^2} + \frac{k}{r} \frac{\partial T}{\partial r} + \frac{dk}{dC} \quad (12)$$

两处必须写明的边界:

① 式 (12) 末项 $\frac{dk}{dC} C_r T_r$ 不能漏, 它体现物性随状态变化;

② 不能把式 (10) 左端的 $B(C) \frac{\partial T}{\partial t}$ 改写成 $\frac{\partial}{\partial t} [B(C) T]$:

$$\frac{\partial}{\partial t} (B T) = B \frac{\partial T}{\partial t} + T \frac{dB}{dC} \frac{\partial C}{\partial t},$$

多出的 $T \frac{dB}{dC} \cdot \partial_t C$ 项无对应的物理过程。

5.5 材料坐标与移动域

5.5.1 坐标变换

用途: 收缩域映射为材料域; 自变量: $x = r / R(t)$; 单位: 无量纲; 物理含义: x 恒对应同一材料。[6]

$$x = \frac{r}{R(t)}, \quad F(x, t) = f(R(t)x, t) \quad (13)$$

$$\left. \frac{\partial f}{\partial t} \right|_r = \left. \frac{\partial F}{\partial t} \right|_x - \frac{x R'}{R} \frac{\partial F}{\partial x}, \quad \frac{\partial f}{\partial r} = \frac{1}{R} \frac{\partial F}{\partial x} \quad (14)$$

5.5.2 干骨架守恒

用途: 推断内部运动与 ρ_d ; 因变量: u 、 ρ_d ; 单位: m/s、kg/m³; 物理含义: 与骨架守恒相容。

$$u(r, t) = \frac{r R'(t)}{R(t)}, \quad \rho_d(t) = \rho_{d0} \frac{R_0^2}{R(t)^2} \quad (15)$$

5.5.3 统一方程

用途: 实际求解形式(式 (1)); 单位: K/s、1/s; 物理含义: R^{-2} 仅来自几何。

$$B(C) \frac{\partial T}{\partial t} = \frac{1}{R(t)^2 x} \frac{\partial}{\partial x} \left(x k(C) \frac{\partial T}{\partial x} \right), \quad (16)$$

必须在论文中写明的两个“不能”:

① 因为 C 是“水与干物质的质量比”而不是“每立方米的水质量”, 收缩不改变其基准, 不能额外添加 $-2 R' C / R$ 的体积浓缩项;

② 因为已采用材料坐标且网格随材料运动, 不能再重复添加 $x R' C_x / R$ 的网格平流项。

5.5.4 扩散尺度

用途: D_4 / D_{23} 比值; 单位: 无量纲; 物理含义: 附录 4 的 D 更小。

$$\left. \frac{D_4}{D_{23}} \right|_{T, C} = \frac{4.2 \times 10^{-4}}{2.4 \times 10^{-3}} \exp \left(\frac{0.15}{C} \right) = 0.175 e^{\frac{0.15}{C}} \quad (17)$$

5.6 初值与边界条件

5.6.1 初始条件

用途:唯一解;单位:K、kg/kg;物理含义:初值均匀。

$$T(x,0)=301.15 \text{ K } (28^\circ\text{C}), \quad C(x,0): \quad (18)$$

5.6.2 中心对称

用途:处理 $r \rightarrow 0$ 奇异性;单位:K/m、1/m;物理含义:零通量,非固定温度。

$$\left. \frac{\partial T}{\partial x} \right|_{x=0}=0, \quad \left. \frac{\partial C}{\partial x} \right|_{x=0}=0, \quad \text{中心对称} \quad (19)$$

5.6.3 热边界

用途:表面能量交换;因变量: T_s 、 C_s ;单位:W/m²;物理含义:热流分为升温与汽化。

$$k \left. \frac{\partial T}{\partial n} \right|_s = h(T_\infty - T_s) + \varepsilon \sigma (T_w^4 - T_s^4) - L_v j_n \quad (20)$$

基线取无辐射与潜热边界:

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h[T_s - T_\infty(t)], \quad h=25 \text{ W} \quad (21)$$

材料坐标:

$$-\frac{k}{R} \left. \frac{\partial T}{\partial x} \right|_{x=1} = h(T_s - T_\infty(t)) \quad (22)$$

5.6.4 水分边界

用途:表面水分交换;单位:kg/(m²·s);物理含义:由 $C_s - C_{eq}$ 与 β 驱动。[5][7]

$$-D \left. \frac{\partial C}{\partial r} \right|_{r=R} = \beta [C_s - C_{eq}(t)], \quad \beta = 8 \times \quad (23)$$

材料坐标:

$$-\frac{D}{R} \left. \frac{\partial C}{\partial x} \right|_{x=1} = \beta [C_s - C_{eq}(t)] \quad (24)$$

C_{eq} 取烘房给出的水分浓度(假设 A04);严格气固平衡需

$$p_{v,\infty} = \frac{p Y}{0.621945 + Y}, \quad p_{v,s} = a_w(C_s, T_s)$$

缺少吸附等温线时 C_{eq} 无法唯一恢复。

5.6.5 等效通量

用途:式 (23) 为等效通量;单位:kg/(m²·s);物理含义:乘 ρ_d 后为真实失水率。

$$j_w|_s = \rho_d \beta (C_s - C_{eq}) \quad (25)$$

5.7 题给经验关系

定性说明(必须写进论文):以下 9 条经验式均为题目给定,本文按原始形式采用,未作重新拟合;题目未提供样本,故不报告、也不能报告拟合优度或系数置信区间。

5.7.1 问题一

用途:常物性与 $D(C)$;自变量: C ;因变量: ρ 、 c_p 、 k 、 D ;单位: kg/m^3 、 $\text{J}/(\text{kg}\cdot\text{K})$ 、 $\text{W}/(\text{m}\cdot\text{K})$ 、 m^2/s 。

[1]

$$\rho = 820 \text{ kg/m}^3, \quad c_p = 2600 \text{ J}/(\text{kg}\cdot\text{K}) \quad (26)$$

$$D_1(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right) \text{ m}^2/\text{s} \quad (27)$$

5.7.2 问题二、三

用途:随 C 变化的物性与 $D(C, T)$;自变量: C 、 C, T ;单位:同式 (26);物理含义: ρ 、 k 近似线性, D 为 Arrhenius 型。

$$\rho_{23}(C) = 650 + 128C \quad (28)$$

$$c_{p,23}(C) = 1450 + 2736 \frac{C}{C+1} \quad (29)$$

$$k_{23}(C) = 0.21 + 0.38 \frac{C}{C+1} \quad (30)$$

$$D_{23}(C, T) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T}\right) \quad (31)$$

5.7.3 问题四

$$\rho_4(C) = 760 + 90C \quad (32)$$

$$c_{p,4}(C) = 1850 + 2150 \frac{C}{C+1} \quad (33)$$

$$k_4(C) = 0.12 + 0.20 \frac{C}{C+1} \quad (34)$$

$$D_4(C, T) = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}\right) \exp\left(-\frac{3850}{T}\right) \quad (35)$$

5.7.4 结构解释

同温按质量加权:[3]

$$c_p = \frac{m_d c_d + m_w c_w}{m_d + m_w} = c_d + (c_w - c_d) \frac{C}{1+C} \quad (36)$$

对式 (31) 求导:

$$\ln \frac{D_{23}}{D_0} = -\frac{0.45}{C} - \frac{3850}{T}, \quad \frac{\partial}{\partial T} \ln \frac{D_{23}}{D_0} = -\frac{3850}{T^2} \quad (37)$$

5.8 无量纲数

用途:判断梯度与传热传质快慢;单位:无量纲。[12]

$$Bi_h = \frac{hR_0}{k}, \quad Bi_m = \frac{\beta R_0}{D}, \quad \alpha = \frac{k}{\rho_{\text{eff}} c_p}, \quad (38)$$

表 12 的完整内容见附录 C。

表 11 题给经验关系汇总

编号	用途	自变量 → 因变量	适用问题	表达式	单位
E01	水分扩散系数	$C \rightarrow D$	一	$D = 7 \times 10^{-9} e^{-C}$	m ² /s
E02	密度	$C \rightarrow \rho$	二、三	$\rho = 650 + 128C$	kg/m ³
E03	比热容	$C \rightarrow c_p$	二、三	$c_p = 1450 + 273C$	J/(kg·K)
E04	导热系数	$C \rightarrow k$	二、三	$k = 0.21 + 0.38C$	W/(m·K)
E05	水分扩散系数	$(C, T) \rightarrow D$	二、三	$D = 2.4 \times 10^{-3} e^{-C}$	m ² /s
E06	密度	$C \rightarrow \rho$	四	$\rho = 760 + 90C$	kg/m ³
E07	比热容	$C \rightarrow c_p$	四	$c_p = 1850 + 215C$	J/(kg·K)
E08	导热系数	$C \rightarrow k$	四	$k = 0.12 + 0.20C$	W/(m·K)
E09	水分扩散系数	$(C, T) \rightarrow D$	四	$D = 4.2 \times 10^{-4} e^{-C}$	m ² /s

6 问题一：预热平衡阶段

6.1 模型建立

附录 2 常物性($\rho=820$ 、 $c_p=2600$ 、 $k=0.36$), D 仅依赖 C (式 (27)):

$$B_1 \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k_1 \frac{\partial T}{\partial r} \right), \quad B_1 = \rho_1 \quad (39)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D_1(C) \frac{\partial C}{\partial r} \right] \quad (40)$$

初边值条件:式 (18)、(19)、(21)、(23)。

6.2 求解方法

参数: $N=3200$ (节点 3201,网格 6.25 μm), $rtol=10^{-10}$, $atol_T=10^{-10}$ K, $atol_C=10^{-12}$ kg/kg,前 4 h 步长 2 s、之后 120 s;式 (39) 独立可解、式 (40) 与温度不耦合。

6.3 结果

表 9(题面表 1) 30 分钟内药材的温度(单位:°C)

时间/s	0	0.5	1	1.5	2
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 10(题面表 2) 30 分钟内药材的水分浓度(单位:kg/kg,干基)

时间/s	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2470

300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

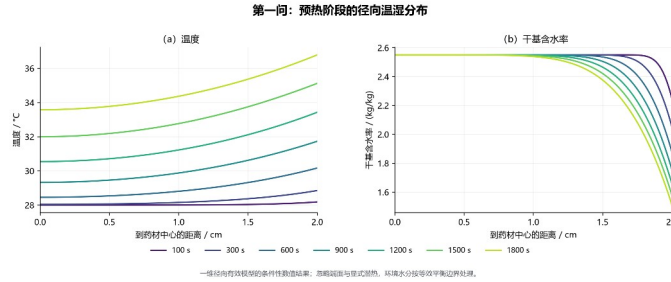


图 4 问题一的径向温湿剖面:(a) 温度(°C)与 (b) 干基含水率(kg/kg)随到中心距离(cm)的分布;曲线对应 100、300、600、900、1200、1500、1800 s 七个时刻。

6.4 结果分析

(1)1800 s 未达热平衡:表面 28.0000→36.7856 °C、中心 33.5753 °C,温差 3.2103 °C, $Bi_h=1.3889$ 须用分布参数模型。

(2)水分变化仅限表层:中心与 1 cm 内基本不变(2.5500、2.5383),表面 2.5500→1.5102(-40.8%)。

(3)热量渗透远快于水分: $Fo_T=0.75985$ 、 $Fo_C=0.022219$,相差约 34 倍($\alpha/D=34.20$)。

(4)热场可验证:240 项 Bessel 解最大偏差 $1.803776172 \times 10^{-6}$ K,480 项 $1.790786825 \times 10^{-6}$ K。

7 问题二:热湿耦合全过程

7.1 模型建立

经验式采用附录 3, $t=0$ 起全程用式 (28)—(31):

$$B(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k_{23}(C) \frac{\partial T}{\partial r} \right), \quad (41)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D_{23}(C, T) \frac{\partial C}{\partial r} \right] \quad (42)$$

初边值条件同 6.1;0—4 h 环境由附件 1 插值, $t > 14400$ s 取 $T_\infty = 323.15$ K、 $C_{eq} = 0.05$ 。

三处必须写明的口径:

① 本问从 $t=0$ 重新开始,全程采用附录 3,不是在 1800 s 之后才切换物性,更不能以问题一末状态为本问初值;

② 式 (41) 中的 $B(C)$ 位于时间导数外侧, $k_{23}(C)$ 与 $D_{23}(C, T)$ 必须留在空间导数内部,二者不能按常数提出;

③ 温度 T 进入式 (31) 的 Arrhenius 项前必须换算为开尔文。

7.2 双向耦合

式 (41)、式 (42) 双向耦合,须联立求解。

7.3 求解设置

参数同 6.2; $t=14400$ s 处断开分段。

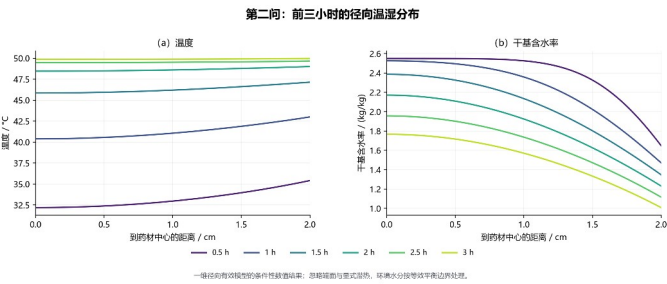
7.4 结果

表 15(题面表 3) 3 小时内药材的温度(单位:°C)

时间/h	0	0.5	1	1.5	2
0.5	32.1892	32.3821	32.9659	33.9608	35.4130
1.0	40.3816	40.5536	41.0605	41.8782	42.9976
1.5	45.8468	45.9348	46.1930	46.6051	47.1401
2.0	48.4502	48.4881	48.5984	48.7736	49.0033
2.5	49.4670	49.4792	49.5136	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

表 16(题面表 4) 3 小时内药材的水分浓度(单位:kg/kg,干基)

时间/h	0	0.5	1	1.5	2
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2.0	2.1709	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081



7.5 结果分析

- (1)温度约 2 h 内均衡:3 h 表面 **49.9664 °C**(环境约 50 °C)、中心 49.8495 °C,差 **0.1169 °C**,0.5 h 表面 35.4130 °C。
- (2)水分梯度远大于温度梯度:3 h 表面 **1.0081**、中心 **1.7662**,差 **0.7581 kg/kg**(中心值的 42.9%)。
- (3)已进入恒温干燥阶段: $\alpha / D \approx 25.7$ (快约 26 倍),题述两阶段无需分段。
- (4)解覆盖 $t=0$ —**206902 s**(约 57.47 h),逐秒、每 0.1 cm 输出;3 h 仅为展示窗口。

8 问题三:全域达标时间的确定

8.1 达标判据

问题三只在问题二的解上施加判据。题面要求"药材各处低于 0.15 kg/kg",故被控量取**空间最大值**:

$$M(t)=\max_{0\leqslant x\leqslant 1}C(x,t),\quad t_*= \inf\ t\geqslant \tag{43}$$

用途:定义烘干时间;自变量: t ;因变量: $M、t_*$;单位:kg/kg、s;物理含义:每一处达标,而非平均值达标。

三点须写明:① 判据为**全域最大值**,用平均值会显著低估时长(第 7.5 节:3 h 时中心为表面的 1.75 倍);② 最大值位置**不能事先假定**,程序对全部 $N + 1$ 个节点逐一检查;③ 显示值 `0.1500` **不能**证明严格小于 0.15,判定须用**未舍入值**。

8.2 严格性处理

数值求根给出**连续穿越时刻**,与四位小数报告口径的关系为

$$\hat{n} = \lceil \frac{10^4 \wedge t_*}{3600} \rceil, \quad t_{\text{rep},s} = 0.36 \wedge n,$$

(44)

用途:连续临界时刻→严格可行的报告时刻;**单位:** $\hat{n}$ 无量纲, $t_{\text{rep},s}$ 、 $t_{\text{rep},h}$ 为 s、h;**物理含义:**网格向上**取整**后用未舍入场值复核 $M(t_{\text{rep}}) < 0.15$ 。

结果:连续临界时刻 $t_* = 57.47230195056044$ h,报告时间 $t_{\text{rep}} = \mathbf{57.4724}$ h,报告时刻未舍入全域最大含水率 $0.14999989718225315 < 0.15$ ✓,折合 2.3947 天(约 **2.39 天**),最慢位置为圆心(材料坐标 $x = 0$)。

报告口径:57.4724 h 是 0.0001 h 网格向上**取整**得到的严格可行时刻,**不等于**"精确到 0.36 秒";连续穿越时刻为 57.4723 h(四位),差异仅来自取整方向。

8.3 结果

表 20(题面表 5) 药材烘干过程的水分浓度(单位:kg/kg,干基)

时间/h	0	0.5	1	1.5	2
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2915	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
57.4724(烘干结束)	0.1500	0.1477	0.1402	0.1249	0.0527

末行中心列显示 `0.1500` 而未舍入值小于 0.15,不得手改为 0.1499。

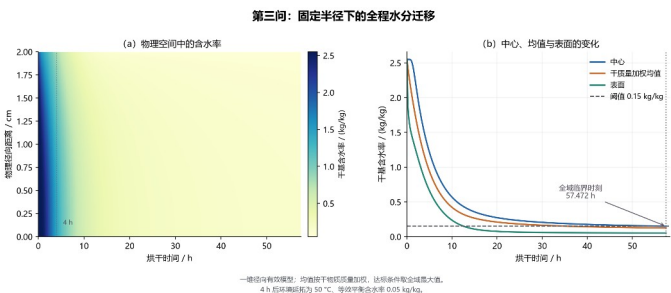


图 6 问题三的全域水分迁移:(a) 物理空间含水率(kg/kg)随烘干时间(h)与径向距离(cm)的分布;(b) 中心、干质量加权均值与表面含水率随时间的变化,水平虚线为 0.15 kg/kg 判据,标注点为全域临界时刻 57.472 h。

8.4 最慢位置与严格性证据

(1)最慢位置为圆心。 57.4724 h 时中心 0.1500(未舍入 0.1499999)、0.5 cm 处 0.1477、表面 0.0527;只看表面会在约 30 h 误判达标。

(2)独立复核。 另用独立二分求根求 $M(t)-0.15$:事件根 57.47232976164861 h 与二分根 57.472329761648595 h 相差 5.82×10^{-11} s;严格复核时刻未舍入最大含水率 $0.14999953019871928 < 0.15 \Rightarrow$ 报告时刻不依赖事件求解器实现。

(3)网格敏感性。 $N=800$ 事件 57.472329762 h 对正式网格 $N=3200$ 的 57.47230195056044 h, 差约 1.0×10^{-4} h(约 0.36 s),粗网格不能作为正式时长。

(4)最慢位置的严谨表述。 在采样时刻直接取 $\arg \max$ 时,最大值可能落在多个不同节点上(问题四出现 21 个不同 x)。经核查,其原因是此时剖面已平坦到浮点舍入量级:圆心相对全域最大值的超出量,问题三最大为 8.882×10^{-16} kg/kg、中位为 0。故本文的表述是“最大值位于圆心,圆心与全域最大值的差不超过约 10^{-15} kg/kg”,而不写成“逐时 $\arg \max$ 恒为圆心”。

8.5 结论

以空间最大含水率严格低于 0.15 kg/kg 为判据,固定半径下烘干时间为 57.4724 h(约 2.39 天),落在题目 2—3 天周期内,由药材中心控制。干燥速率迅速衰减:0→6 h 表面含水率由 2.5500 降到 0.5328,48→57.4724 h 中心仅由 0.1618 降到 0.1500(传质驱动力减小,且式 (37) $D \propto e^{-0.45/C}$ 随含水率降低迅速减小)。

9 问题四:收缩条件下的烘干时长

9.1 模型建立

问题四整组用附录 4 物性(式 (32)—(35))并叠加附件 2 的半径收缩,控制方程取式 (16) 的材料坐标形式:

$$B_4(C) \frac{\partial T}{\partial t} = \frac{1}{R(t)^2 x} \frac{\partial}{\partial x} \left(x k_4(C) \frac{\partial T}{\partial x} \right) \quad (45)$$

$$B_4(C) = \rho_4(C) c_{p,4}(C), \quad \rho_d(t) = \rho_{d0} \left(\quad \right) \quad (46)$$

边界条件取式 (22)、式 (24) 的材料坐标形式,判据同式 (43)。半径 $R(t)$ 由附件 2 的 145 个观测点分段线性插值得到,程序只用 SR\$ 值、不对折线求数值导数;半径由 2.000 cm 单调减至约 1.198 cm。

9.2 数据依据与净效应识别

问题四相对问题三同时改变了物性(附录 3→附录 4)与几何(固定半径→收缩),故两问时长差(57.4724 - 51.0906 = 6.3818 h)不能全部归因于收缩;收缩净效应只能由 9.5 节的同物性对照单独给出。

9.3 结果

表 22(题面表 6) 药材烘干过程的水分浓度(单位:kg/kg,干基)

时间/h	0	0.5	1	1.5	2	药材表面
6	1.7196	1.5377	1.0226	—(域外)	—(域外)	0.4207
12	0.7377	0.6546	0.4083	—(域外)	—(域外)	0.1670
18	0.4087	0.3686	0.2397	—(域外)	—(域外)	0.0892
24	0.2851	0.2611	0.1790	—(域外)	—(域外)	0.0673
30	0.2263	0.2094	0.1493	—(域外)	—(域外)	0.0595
36	0.1928	0.1797	0.1317	—(域外)	—(域外)	0.0560
42	0.1712	0.1604	0.1200	—(域外)	—(域外)	0.0541

48	0.1561	0.1468	0.1116	—(域外)	—(域外)	0.0530
51.0906(烘干结束)	0.1500	0.1413	0.1081	—(域外)	—(域外)	0.0526

半径 6 h 已缩至 1.3740 cm 并继续收缩,故 **1.5 cm 与 2.0 cm** 两列自始至终位于药材之外:实际输出应填"—(域外)",**绝不能填 0——0** 表示"完全干燥",与"不在药材内"含义相反。末行中心列显示 `0.1500`,未舍入值严格小于 0.15。

表 23 问题四各报告时刻的药材表面半径(单位:cm)

时间/h	6	12	18	24	30	36	42	48	51.0906
表面半径/cm	1.3740	1.2480	1.2140	1.2040	1.2010	1.2000	1.2000	1.2000	1.2000

结果:连续临界时刻 $t_*=51.09057478683054$ h,报告时间 $t_{\text{rep}}=$ **51.0906 h**,报告时刻未舍入全域最大含水率 $0.14999995339076624<0.15$ ✓,折合 2.1288 天(约 **2.13 天**),结束时半径 1.2000 cm,最慢位置为圆心,且**未使用半径外推**(51.09 h < 72 h)。

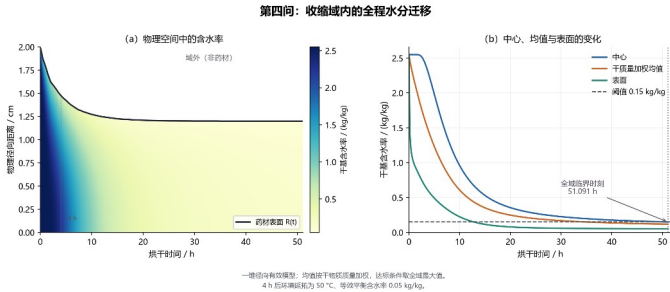


图 7 问题四的全域水分迁移:物理空间含水率(kg/kg)随烘干时间(h)与径向距离(cm)的分布,以及材料点含水率随时间的演化;标注全域临界时刻 51.0906 h。

9.4 结果分析

- (1)收缩对网格更敏感。 $N=800$ 事件 51.090599156 h 对正式网格 $N=6400$ 的 51.09057478683054 h,差约 2.44×10^{-5} h(约 0.088 s),故 $N=800$ 绝不能当作正式时长。
- (2)中心控制达标不变: 51.0906 h 时中心 0.1500、1 cm 处 0.1081、表面 0.0526。
- (3)归因限制。 前中期问题四中心含水率反而更高(24 h:0.2851 对 0.2378),约 40 h 后反转;因两问同时改变了物性与几何,该交叉只能作为观测事实报告,**不能单独归因于任一因素**,单独归因须依据 9.5 节同物性对照。

9.5 收缩的净效应

用同附录 4 物性、固定半径与收缩半径的对照(同参数、同源码版本)分离收缩作用:
表 26 同附录 4 物性下的收缩净效应对照

对照条件(附录 4 物性)	全域达标时间	折合
固定半径 $R_0=2$ cm	129.8452 h	约 5.41 天
实际收缩 $R(t)$ (附件 2)	51.0910 h	约 2.13 天
收缩带来的缩短	缩短 60.65%	—
按 $(R_0/R)^2$ 折算的等效定半径时间	131.6198 h	与直接模拟相差 1.37%

同物性下收缩使烘干时间由约 129.85 h 缩短到约 51.09 h,缩短约 **60.7%**;机理是扩散时间尺度 $\tau \sim R^2/D$ 随半径减小而缩短(R 由 2 cm 降至约 1.2 cm, R^2 降至约 36%)。

两条限制:1. 固定半径对照的 129.85 h 超出题目"2—3 天"范围,也超出附件 2 的 72 h 观测范围,只能作机理对照、**不能作为答案**;2. 附件只给半径轨迹、**无内部位移观测**,同比径向收缩是"与表面速度和均匀干骨架守恒相容的最简闭合",属假设 A06。

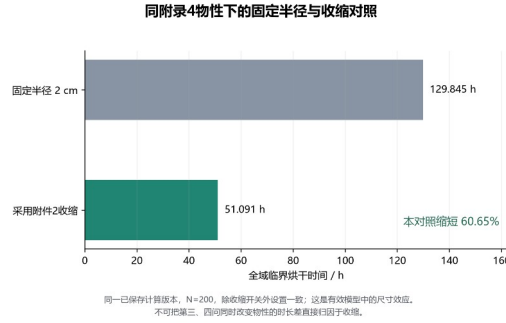


图 8 收缩的净效应:同附录 4 物性、固定半径与收缩半径两种情形下,空间最大含水率(kg/kg)随烘干时间(h)的变化;水平虚线为 0.15 kg/kg 判据。

9.6 结论

计入附件 2 半径收缩后烘干时间为 **51.0906 h(约 2.13 天)**,落在题目 2—3 天区间内;报告时刻半径收缩至 1.2000 cm,且未使用 72 h 之后的半径外推。与问题三的 6.38 h 差值因两问物性与几何同时不同,不能全部解释为收缩;同物性对照显示收缩本身贡献约 60.7%。

10 数值求解方法与模型检验

10.1 求解策略概述

式 (10)、式 (11)、式 (16) 为带非线性系数与 Robin 边界的刚性抛物型方程组。求解链:有限体积离散 → Kirchhoff 浓度势湿通量 → 刚性 ODE → 隐式 BDF/NDF → 事件复核 → 稠密输出。

10.1.1 方程组的四个数学特征与方法选择的理由

① 热扩散 0.6580—1.4154 h 与水分扩散 19.6947—106.1119 h 差 25—75 倍,显式步长限在 $10^3—10^4$ s 而过程 2×10^5 s,故用隐式。② $D \propto e^{-a/C}$ 跨约 3.6 个数量级($3.12 \times 10^{-12}—1.25 \times 10^{-8}$ m²/s),故湿通量按浓度势精确积分。③ 驱动由附件 1 线性插值给出,故网格含表面节点并在 $t=14400$ s 断开。④ 问题四半径 2→约 1.2 cm,故以式 (13)—(16) 化为固定域。

10.2 空间离散:守恒型节点对偶有限体积

10.2.1 网格与对偶控制体

式 (47) 网格与控制体加权体积($\sum_i w_i = 1/2$):

$$x_i = \frac{i}{N} \quad (i=0,1,\dots,N), \quad x_{i+1/2} = \frac{x_i + x_{i+1}}{2} \quad (47)$$

其中 $a_i = x_{i-1/2}$ 、 $b_i = x_{i+1/2}$, $V_i = 2\pi L R^2 w_i$;网格含真实表面节点,Robin 条件直接给出最外侧通量。

10.2.2 守恒性的证明(为什么"内部通量严格相消")

把式 (52) 第二式乘 $2w_i$ 并对全部节点求和:

$$\sum_{i=0}^N 2w_i \odot C_i = \sum_{i=0}^N \frac{2}{R^2} (G_{i+1/2}^C - G_{i-1/2}^C) \quad (47a)$$

右端为望远镜求和:内部面通量 $G_{i+1/2}^C$ 在第 i 式取正号、第 $i+1$ 式取负号,故内部面项两两相消,只剩表面上界项与中心下界项:

$$\sum_{i=0}^N 2w_i \odot C_i = \frac{2}{R^2} (G_{N+1/2}^C - G_{1/2}^C) \quad (47b)$$

由式 (51) $G_{1/2}^C = 0$ 、 $G_{N+1/2}^C = -\beta R(C_N - C_{eq})$,代入并用式 (53) 的 $\odot \ell$ 定义即得

$$\frac{d}{dt} [\odot C(t) + \ell(t)] = 0 \quad \Rightarrow \quad \odot C(t) + \ell(t) = \text{const} \quad (47c)$$

式 (47c) 即式 (54):用"同一套面通量同时贡献给左右两侧"的写法守恒自动成立,是发现实现错误的强诊断量(检验二)。

10.3 内部面热通量:调和平均及其适用性判据

式 (48) 调和平均面热导(W/m),把两段半间距视为串联热阻:

$$G_{i+1/2}^T = \frac{x_{i+1/2}}{\Delta x} k_H (T_{i+1} - T_i), \quad k_H = \frac{2k_i}{k_i + k_{i+1}} \quad (48)$$

它不等于对 $k(C)$ 的精确积分(检验三)。

10.4 内部面湿通量:Kirchhoff 浓度势(本文的主要数值方法修正)

式 (31)、式 (35) 给出 $D = D_0 e^{-b/T} e^{-a/C}$, (D_0, a, b) 取 $(2.4 \times 10^{-3}, 0.45, 3850)$ 或 $(4.2 \times 10^{-4}, 0.30, 3850)$;式 (49) 的浓度势 $\Phi(C)$ (Ei 为指数积分函数):[10]

$$\Phi(C) = C e^{-a/C} + a \text{Ei}(-\frac{a}{C}), \quad \Phi'(C) =: \quad (49)$$

式 (50) 面湿通量($\text{m}^2 \cdot (\text{kg}/\text{kg})/\text{s}$):只对 $\Phi(C)$ 作差,温度因子取面温度定值:

$$G_{i+1/2}^C = \frac{x_{i+1/2}}{\Delta x} D_0 e^{-b/\odot T} [\Phi(C_{i+1}) - \Phi(C_i)] \quad (50)$$

对 $e^{-b/T} \Phi(C)$ 整体作差会引入题目未定义的温度梯度驱动交叉扩散项(Soret 型),属伪物理。

直接对式 (49) 求导可得

$$\Phi'(C) = e^{-a/C} + \frac{a}{C} e^{-a/C} - \frac{a}{C} e^{-a/C} = e^{-a/C} \quad (50a)$$

10.5 表面与中心面通量

式 (51) 表面 Robin 通量与中心对称条件(符号同式 (21)、式 (23),向内为正):

$$G_{N+1/2}^C = -\beta R(C_N - C_{eq}), \quad G_{N+1/2}^T = -h \quad (51)$$

10.6 半离散常微分方程组

式 (52) 控制体半离散方程(K/s、1/s):热容量 × 升温速率 = 净导入导热通量

$$\odot \cdot T_i = \frac{G^T_{i+1/2} - G^T_{i-1/2}}{R^2 w_i B_i}, \quad \odot \cdot C_i = \frac{G^C_{i+1/2}}{R} \quad (52)$$

内部面通量以相反符号贡献给左右两侧(见式 (47a)——(47c))。

10.7 累计失水作为附加状态变量

式 (53) 累计失水 $\ell(t)$ (kg/kg)与干物质加权场平均 $\odot C(t)$:

$$\odot \cdot \ell = \frac{2\beta}{R} (C_N - C_{eq}), \quad \odot C(t) = 2 \sum_{i=0} \quad (53)$$

由式 (52)、式 (53) 即得离散质量恒等式:

$$\odot C(t) + \ell(t) = \odot C(0) = 2.55 \quad (54)$$

10.8 时间推进与解析 Jacobian

10.8.1 时间推进方法与解析 Jacobian

BDF/NDF(SciPy `solve\_ivp`, 隐式、变阶 1—5、变步长)配手写**解析稀疏 Jacobian**(式 (52) 每点导数只依赖相邻两点,故为三对角块状稀疏矩阵)。[8][9]

10.8.2 解析 Jacobian 的组成

式 (55) 容量系数对含水率的导数(问题二、三):

$$\rho_C = 128, \quad c_{p,C} = \frac{2736}{(1+C)^2}, \quad k_C = \frac{0.36}{(1+C)} \quad (55)$$

式 (56) 扩散系数对状态的导数:

$$D_C = D \frac{a}{C^2}, \quad D_T = D \frac{b}{T^2} \quad (56)$$

式 (57) 调和平均面对的导数:

$$\frac{\partial k_H}{\partial k_i} = \frac{2k_{i+1}^2}{(k_i + k_{i+1})^2}, \quad \frac{\partial k_H}{\partial k_{i+1}} = \frac{2k_i^2}{(k_i + k_{i+1})^2} \quad (57)$$

式 (58) 面通量对相邻含水率的导数:

$$\frac{\partial G^C_{i+1/2}}{\partial C_{i+1}} = \frac{x_{i+1/2}}{\Delta x} D_0 e^{-b/\odot T} \Phi'(C_{i+1}), \quad (58)$$

最易遗漏项在温度行 × 含水率列(来自分母 B_i):

$$\left. \frac{\partial \odot T_i}{\partial C_i} \right|_{\text{热容量项}} = -\odot T_i \frac{B_{C,i}}{B_i} \quad (59)$$

10.8.3 生产参数设置

表 21 生产数值设置

参数	问题一	问题二、三	问题四
空间区间数 N	3200	3200	6400
节点数	3201	3201	6401

ODE 状态长度(含 ℓ)	6403	6403	12803
初始物理网格间距	$6.25 \mu\text{m}$	$6.25 \mu\text{m}$	$3.125 \mu\text{m}$
相对容差 $rtol$	10^{-10}	10^{-10}	10^{-10}
温度绝对容差	10^{-10}K	10^{-10}K	10^{-10}K
含水率绝对容差	10^{-12}kg/kg	10^{-12}kg/kg	10^{-12}kg/kg
最大步长(前 4 h / 之后)	2 s / 120 s	2 s / 120 s	2 s / 120 s
时间方法	BDF/NDF	BDF/NDF	BDF/NDF
湿面格式	Kirchhoff 势	Kirchhoff 势	Kirchhoff 势
Jacobian	解析稀疏	解析稀疏	解析稀疏
稠密输出存储	磁盘	磁盘	磁盘
事件后复核尾段	—	1 s	1 s

问题二、三接受 14283 个时间点,问题四 15996 个,均**无警告**;1 s、60 s、0.1 cm 是**输出抽样间隔**而非步长。

10.9 模型检验

检验一:Bessel 解析对照(验证离散求解器)

物性冻结为问题一的常系数、无潜热、给定 Robin 边界时存在解析解(本文**独立重写** Bessel 根求解与正交展开):[10][14]

$$T(r, t) = T_{\infty} + \sum_{n=1}^{\infty} A_n J_0\left(\frac{\mu_n r}{R}\right) \exp\left(-\frac{\alpha t}{l}\right) \quad (60)$$

$$A_n = \frac{2 J_1(\mu_n)}{\mu_n [J_0^2(\mu_n) + J_1^2(\mu_n)]} (T_0 - T_{\infty}) \quad (61)$$

μ_n 为 $\mu J_1(\mu) = Bi_h J_0(\mu)$ 的第 n 个正根。**实测**: $N=3200$ 与 240 项级数解的节点口径最大偏差 $\$1.803776172 \times 10^{-6}\$ \text{K}$ (480 项 $1.790786825 \times 10^{-6} \text{K}$),恒出现在**表面节点**;改用**精确环形平均口径**, $N=800/1600/3200/6400$ 的最大差为 2.0512×10^{-3} 、 1.0258×10^{-3} 、 5.1292×10^{-4} 、 $2.5647 \times 10^{-4} \text{K}$,恰好一阶收敛(阶 0.99986),源于**表面控制体是单侧的**。故节点口径内部偏差约 $1.8 \times 10^{-6} \text{K}$ 、控制体平均口径表面偏差约 $2.1 \times 10^{-3} \text{K}$ 且一阶收敛——**两者不是同一误差,不能互相替代**。

检验二:离散质量恒等式(验证守恒与实现自治)

按式 (53) 每个接受时刻算残差 $\oint C(t) + \ell(t) - 2.55$,两途径共用**同一套面通量**;实测最大残差:问题一 5.77×10^{-15} 、问题二、三 1.15×10^{-14} 、问题四 $7.99 \times 10^{-15} \text{kg/kg}$ 。

应写"残差约 $\$10^{-14}\$ \text{kg/kg}$,表明格式守恒且实现自治",不得写成"误差为 $\$10^{-14}\$"$ 。本文不

构造 $\frac{d}{dt}[B(C)T]$ 形式的焓式能量恒等式:式 (10) 用**有效显热闭合**、 $B(C)$ 在时间导数**外侧**,

$$\frac{d}{dt}[B(C)T] = B(C) \frac{\partial T}{\partial t} + T \frac{dB}{dC} \frac{\partial C}{\partial t} \neq$$

仅当物性为常数(问题一)时二者相等,故问题一能过"焓式"能量检验(相对残差 3.48×10^{-9})而问题二/三、四不能——该恒等式在变物性下本就不成立,不是求解器的缺陷。

检验三:空间网格收敛(验证空间离散误差)

相邻网格比较公共整数秒 $\times 21$ 个固定半径($\Delta T / \Delta C$ / 临界时刻差):问题一 $N1600 \rightarrow 3200$ 为 1.9162×10^{-6} K、 4.6903×10^{-5} kg/kg、—;问题二、三为 2.8929×10^{-6} K、 3.8303×10^{-5} kg/kg、 -0.0201464 s;问题四 $N3200 \rightarrow 6400$ 为 9.6339×10^{-7} K、 1.9799×10^{-5} kg/kg、 -0.00417055 s;问题四另比材料点 $S_x=1S$:温差 $2.328249423 \times 10^{-6}$ K、含水率差 8.2256×10^{-6} 。

经验收敛阶:含水率 **2.0173 / 2.0151 / 2.0151 / 1.8171**,临界时刻 **1.9890 / 2.0034**。

检验四:时间推进设置敏感性(验证时间离散误差)

基准与 tight(更紧容差 + 更小步长)对比($\Delta T / \Delta C$ / 临界时刻差):问题一 $N3200$ 为 $1.627586983 \times 10^{-6}$ K、 $3.619134326 \times 10^{-9}$ kg/kg、—;问题二、三为 $2.660259952 \times 10^{-6}$ K、 $9.941051204 \times 10^{-9}$ kg/kg、 $+3.975728760 \times 10^{-6}$ s;问题四 $N6400$ 为 $9.443490399 \times 10^{-7}$ K、 $9.818015290 \times 10^{-10}$ kg/kg、 $-3.415538231 \times 10^{-6}$ s;问题四真实表面 ($x=1$)3066 个共同 60 s 时刻最大温差 $3.239170269 \times 10^{-6}$ K、含水率差 $8.699361120 \times 10^{-9}$ 。tight 同改容差与步长,故只称"时间推进设置敏感性"。

检验五:跨实现交叉核验(验证实现正确性)

MATLAB 独立有限体积 + 'ode15s' 重写同一套方程($N=40$),与 Python 生产实现比较 100 个标量场值与 2 个事件;最大差:温度 2.375×10^{-7} K、含水率 5.851×10^{-10} kg/kg、事件 2.297×10^{-4} s。[11]

10.10 检验结果汇总

表 24 的完整内容见附录 C。

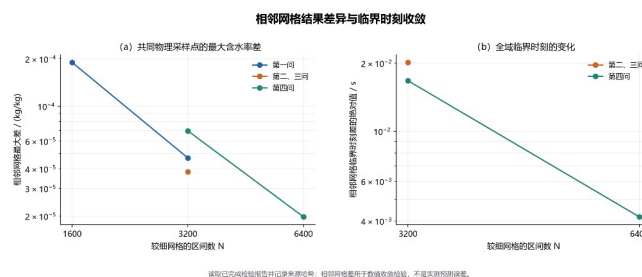


图 9 相邻网格结果差异与临界时刻收敛:(a) 共同物理采样点的最大含水率差(kg/kg)与 (b) 相邻网格临界时刻差的绝对值(s)随较细网格区间数 N 的变化(双对数坐标)。

10.11 本节结论(统一声明)

须披露的不完美:四位小数并非稳定末位——按 `'round(float(x),4)` 重核,温度显示不同的单元数在问题一/二、三/四为 6/63/45(含水率 0/2/2):问题二 $t=5400$ s、 $r=2$ cm 处 47.140050004863326 °C 与 47.140049992523245 °C 仅差 1.23×10^{-8} K,四位小数却为 47.1401 与 47.1400,属存储与展示口径。

两个报告时间(57.4724 h、51.0906 h)在两套时间设置下稳定。

上述检验支持:所列假设与参数下数值结果可复现且离散误差受控;因缺少实测数据,本文不对物理预测准确率与连续时空严格误差界作声明。

11 模型的修正与创新

11.1 修正一:密度与几何的相容性修正

问题。题面 $\rho_4 = 760 + 90C$ (式 (32)) 若按真实湿质量密度理解,则干物质密度为

$$\rho_d = \frac{\rho_4(C)}{1+C} = \frac{760+90C}{1+C} \leq 760 \text{ kg/m}^3 \quad (62)$$

固定长度、干质量守恒下(式 (15)),于是

$$R(t) \geq R_0 \sqrt{\frac{\rho_{d0}}{760}} = 0.02 \times \sqrt{\frac{278.7324}{760}} = 1.2 \quad (63)$$

附件 2 半径 19.5 h 后即低于该值,72 h 完全无水也仅容纳初始干质量的

$$\frac{R(72 \text{ h})^2}{R_0^2} \cdot \frac{760}{\rho_{d0}} = \frac{1.198^2}{2.000^2} \times \frac{760}{278.7324} = 0.97 \quad (64)$$

即最多 97.83%、至少短缺 2.1683% $\Rightarrow$ 题给密度式、给定半径轨迹、固定长度与严格干物质守恒四者不能同时精确成立。

处理。 ρ 限定为有效热物性、仅入 $B = \rho_{\text{eff}} c_p$ (式 (10)), ρ_d 由式 (15) 独立确定、按式 (4) 在初态标定。

效果。越界点 19.5 h 距界仅约 2.03 μm 、小于半个显示单位,不能称为矛盾精确起点。

11.2 修正二:表面蒸发的能量闭合(潜热包络)

问题。基线只用式 (23)、按 A02 不计相变能耗,但潜热/显热 ≈ 21.59 与 21.73 ,不能用"潜热很小"论证省略。(问题二、三潜热 497.705 kJ 对显热 23.048 kJ,问题四 504.382 kJ 对 23.214 kJ)[5]

表 27 的完整内容见附录 C。

处理。补全表面能量平衡(式 (20) 去辐射项、留潜热项):

$$h(T_\infty - T_s) = k \frac{\partial T}{\partial r} \Big|_s + L_v j_w \Big|_s, \quad j_w \Big|_s = \quad (65)$$

驱动力仍为题给的等效含水率差(式 (23)、式 (25)),仅补"水离开表面须带走汽化热";对未给吸附等温线引入表面潜热比例 $L \in [0,1]$ 作情景包络。

效果。 $L=1$ 时 +5.26% / +10.56%、最低表面温度 10.90 / 14.55 $^\circ\text{C}$, $N=1600$ 下为 60.497191 / 56.484445 h(差 2.7×10^{-5} 、 1.6×10^{-5} h);但 $L=1$ 需自由水面,故本表是包络而非标定模型,基线仍取 $L=0$ 。

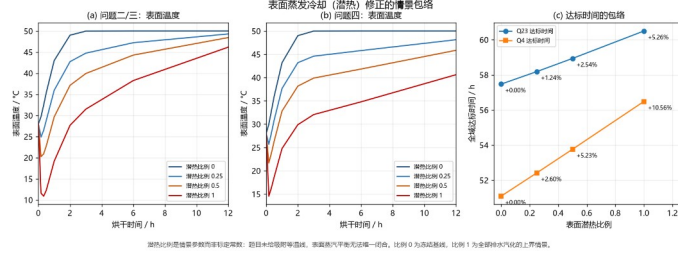


图 10 表面蒸发冷却(潜热)修正的情景包络:(a) 问题二、三与 (b) 问题四的表面温度(°C)随烘干时间(h)的曲线族;
(c) 全域达标时间(h)随表面潜热比例的变化及相对基线偏差。

11.3 创新三:Kirchhoff 浓度势及其必要性证明

问题。 式 (31)、(35) 的 $D(C, T) = D_0 e^{-b/T} e^{-a/C}$ 在干燥表层可跨约 3.6 个数量级,常规界面调和平均式 (66) 因 $e^{-a/C}$ 极陡而高估表层阻力,给出错误时长与错误的参数敏感性方向。

$$G_{i+1/2}^C \approx D_{\text{face}} \frac{C_{i+1} - C_i}{\Delta x} \quad (66)$$

处理。 改用式 (49)、(50) 的 Kirchhoff 浓度势:由 $\Phi^1(C) = e^{-a/C}$,面内 $D \, dC$ 在固定面温度下解析积分,消除该误差。

表 32 的完整内容见附录 C。

效果。 调和平均在 $N=1600$ 仍偏离收敛值约 $0.009 / 0.0012 \, \text{h}$,Kirchhoff 势从 $N=400$ 到 1600 仅变 $1.09 \times 10^{-4} \, \text{h}$;只换面通量离散的独立验证中,常数扩散极限下两格式逐位差 0.0,变扩散下最大差随加密为 $3.73 \times 10^{-6} \rightarrow 9.33 \times 10^{-7} \rightarrow 2.33 \times 10^{-7} \rightarrow 5.83 \times 10^{-8} \, \text{kg/kg}$ 、观测阶 **1.99907**(预先声明 1.0);但温度冻结、圆柱离散与节点近似仍带来误差。

11.4 创新四:材料坐标移动域 + 干骨架守恒

问题。 附件 2 只给表面半径轨迹;物理坐标求解须处理网格运动与守恒的相容性,易重复计入平流或稀释项。

处理。 取式 (13)—(16) 材料坐标 $x = r/R(t)$ 、同比径向收缩 $u = rR'/R$ 与干骨架守恒 $\rho_d = \rho_{d0} (R_0/R)^2$:材料导数中两项恰好抵消,方程只留 $1/R(t)^2$,不另加平流项、不重构网格。

效果。 按 R^2 折算收缩时间轴应得定半径问题的时间

$$\tau_{\text{eq}} = \int_0^{t_{\text{shrink}}} \left(\frac{R_0}{R(t)} \right)^2 dt \quad (67)$$

实测同物性定半径 129.8452 h 对收缩 51.0910 h(缩短 60.65%),式 (67) 折算为 **131.6198 h**、偏差 +1.37%,残差来自剖面形状而非收缩运动学。

11.5 创新五:数据锚定的吸附/水活度闭合

问题。 A04 把烘房水分浓度直接当作表面等效平衡含水率;本文基线更深一层隐含 $a_w \equiv 1$ (视为自由水面),高估干燥后段表面驱动力。[7]

处理。 由附件 1 湿度比换算水蒸气分压

$$p_v = p_{\text{sat}}(T) \cdot \text{RH}, \quad p_{v,\infty} = \frac{p Y}{0.621945 + Y} \quad (68)$$

烘房平台段($t \geq 3600$ s)相对湿度 0.5942—0.6199、均值 0.6073,温度均值 49.66 °C; $C_{eq}=0.05$ kg/kg 处既不吸水也不失水,故该点水活度被题面数据强制为

$$a_w(C_{eq}=0.05, T_{\infty} \approx 49.7 \text{ °C}) = \text{RH}_{\infty} = 0.60 \quad (69)$$

单一锚点 (0.05, 0.607) 不能唯一确定 $a_w(C)$,故取 $p \geq 1$ 单参数族($p < 1$ 使低含水率处 a_w 变负,不可用):

$$a_w(C) = 1 - (1 - a_{w,\text{ref}}) \left(\frac{C_{\text{ref}}}{C} \right)^{1/p} \quad (70)$$

化为表面驱动缩放因子

$$g_{\text{表面}} = -\beta R K_{\text{eff}}(C_s)(C_s - C_{eq}), \quad K_{\text{eff}} \quad (71)$$

两项恒等式检验:① 锚点检验 $a_w(C_{\text{ref}}) = 0.600$ 与平台相对湿度之差为 0.0;② 基线检验 $p=1$ 时 $K_{\text{eff}} \equiv 1$,且问题一、二三、四中冻结与修正模型同初态的右端函数最大差恰为 0.0(逐位相同) $\Rightarrow$ 本文基线正是本族的 $p=1$ 成员。

该族达标时间($N=800$)随 p 单调延长: $p=1$ 为 57.4723 h / 51.0906 h(基线), $p=4$ 为 65.4447 h / 55.3162 h。

一个被质量守恒卫兵抓住的真实错误(如实披露):构造本闭合族时,曾按“向外为正”的约定写出表面边界,得到与式 (51) 反号的表达式。其后果是 $p=1$ 时表面通量反号、含水率单调上升至 15.96 kg/kg,被式 (54) 的离散质量恒等式直接拦截(残差远超 10^{-14})。修正为对基线驱动量的直接缩放后, $p=1$ 与基线模型的右端函数逐位一致(最大差恰为 0.0)。这说明符号约定必须在全文统一,且守恒诊断能独立发现符号错误,不依赖肉眼检查。

效果。 承认“材料在 $C=0.05$ 处处于平衡”的闭合都使末段表面驱动小于本文基线的隐含值,即高估驱动力、低估时长;故 57.4724 h 与 51.0906 h 应表述为该闭合族的下界(限该族内、无反向温度耦合), p 的上界无题面依据,本文报告的是包络而非标定结果。

11.6 创新小结

表 29 的完整内容见附录 C。

12 多方法交叉验证

无内部实测真值时,“检验”只能由相互独立的方法对同一结论的交叉印证构成;本章按 L1—L10 给出结果,容差运行前预先声明。

12.1 交叉验证架构

各层的原始输出与判定记录随源程序一并提供于支撑材料。

表 31 的完整内容见附录 C。

12.2 L1:时间积分器互换

同网格、同容差下仅换积分方法(BDF/NDF $\rightarrow$ 三阶 Radau IIA,另一积分族):

表 33 的完整内容见附录 C。

结论:两积分族在三档网格上事件微秒级一致 $\Rightarrow$ 时长不是积分器的产物。

12.3 L2:湿分面离散格式的必要性

调和平均在 $N=1600$ 仍与 Kirchhoff 相差 8.759×10^{-3} h(问题二三)、 1.211×10^{-3} h(问题四),而 Kirchhoff 由 $N=400$ 到 1600 只变 1.09×10^{-4} h;故生产格式已收敛、替代格式不可用(表 32)。

12.4 L5:收缩效应的独立尺度互验

等效定半径时间 131.6198 h 与直接模拟 129.8452 h 相差 1.3667%(预声明容差 5%;式 (67))。

12.5 L6:三阶段分界

四种独立判据的分界(h):

表 34 的完整内容见附录 C。

诚实结论:

环境、半径与计算温度序列都是渐变的,不存在统计意义上的突变点(Mann-Kendall 检验 UF/UB 曲线在置信带内不相交),故分界只能以区间报告(图 11 为判据位置与 UF/UB 曲线)。

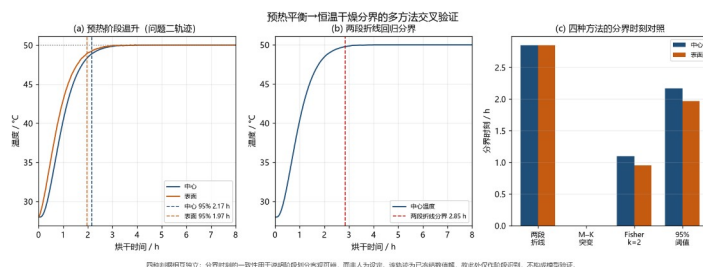


图 11 预热平衡与恒温干燥分界的多方法交叉验证:(a) 问题二中心与表面温度(°C)随烘干时间(h)的轨迹及两段折线回归的 95% 分界点;(b) 两段折线回归的分界位置;(c) 四种判据(两段折线、Mann-Kendall 突变、Fisher 最优分割、95% 阈值)给出的分界时刻(h)对照。

12.6 方法学提醒:注入类实验必须自带哨兵

敏感性实验中修正了两处"看似跑两套方法、实际只跑一套"的真实缺陷:

- 替代通量从未被调用:对照脚本内部自建模型实例,使"先建实例再替换其方法"不生效,"两法完全一致(差 0.0)"实为未调用;修正:加 `fluxPathUsed` 守卫,失效即抛错。
- 参数注入失效:Kirchhoff 分支内部面通量用硬编码的扩散系数前因子、不读取物性函数返回的 $\$D\$$ 数组,"只缩放 D "不改变模型,筛选结果呈现" D 影响恰为 0"的假象;修正:对返回通量直接乘尺度因子(Kirchhoff 面通量对 D_0 严格线性)并加 $scale=1.0$ 基准自检。

方法学结论:注入类实验必须先验证"注入确实生效",否则"参数无影响"会被误读为稳健性(与 12.3 节同属一类风险)。

12.7 交叉验证小结

达标时间由两个积分族与两种湿面离散格式互验且三档网格一致,阈值事件根差 5.8×10^{-11} s,收缩短效应经 R^2 尺度折算互验到 1.37%,三阶段分界由四判据交叉识别落于同一区间。故 57.4724 h 与 51.0906 h 不是特定网格、求解器或判据的产物。

13 灵敏度分析与不确定性

13.1 不确定性的来源分类

三类不确定性分别报告,不混为一谈:数值(网格、时间推进、阈值插值、容差)见 10.9 节;情景(环境延拓、 C_{eq} 、 h 与 β 、潜热、端面)见 13.2—13.6;参数(扩散前因子、导热系数、尾段温度)见 13.3。

边界声明:本题无内部实测真值,上述区间都是情景范围或设置差异,不是统计置信区间,也不是实测误差。

13.2 环境延拓的不确定性

报告时间落在附件 1 观测窗(4 h)外的比例:问题三 93.04%(57.4724 h 中 53.4724 h 无观测)、问题四 92.17%(51.0906 h 中 47.0906 h 无观测)。

4 h 末点(50.165 °C / 0.04986)与平台(50 °C / 0.05)间有 -0.165 °C / $+0.00014$ 拼接跳变, $t=14400$ s 处分段求解使其数值可见,平台仍为假设 A05。尾段温度情景:49 °C 使问题二三临界时间 +3.25%、51 °C 使 -3.11%,属物理输入情景敏感性,不是统计误差条。

13.3 参数不确定性与"传质控制"结论

口径说明:本表为 $N=200$ 代理网格,与正式网格($N=3200/6400$)事件时间差 1.65 s(问题二三),只用于排序与相对幅度; $scale=1.00$ 处已自检复现代理基准到 10^{-6} h。

表 35 参数扫描结果($N=200$ 代理网格,仅用于排序与相对幅度)

扩散前因子尺度	问题二三事件 /h	问题四事件 /h
0.70	79.4187	70.4059
0.875	64.7520	57.5214
1.00(基准)	57.4728	51.0910
1.05	55.0567	48.9526
1.225	48.1888	42.8615
1.40	43.0796	38.3160

跨度:问题二三 36.34 h(63.2%),问题四 32.09 h(62.8%)。

导热系数尺度	问题二三事件 /h	问题四事件 /h
0.85	57.4752	51.0941
0.925	57.4739	51.0924
1.00(基准)	57.4728	51.0910
1.075	57.4718	51.0898
1.15	57.4710	51.0887

跨度:问题二三元 0.00418 h(0.0073%),问题四元 0.00535 h(0.0105%)。

本工况下模型是传质控制的:扩散前因子 -30%/+40% 使达标时间变化约 +38%/-25%,导热系数 $\pm 15\%$ 的影响不到万分之一。

Morris 筛选(仅排序):最敏感三参数为尾段温度、表面潜热比例、 $\beta(\mu^*=7.36、2.96、2.44$ h/单位);其 $Sd_{\{\text{scale}\}}$ 条目必须作废(见 12.6 节), $D、k$ 结论以表 33 的带自检单参数扫描为准。

13.4 物理闭合的情景包络

潜热未计入时 $L \in 0, 0.25, 0.5, 1$ 包络(表 27)使问题二三 +0—5.26%、问题四 +0—10.56%;气固平衡未闭合(隐含 $a_w \equiv 1$)时数据锚定的 p 族包络(表 28)使问题二三 +0—13.87%、问题四 +0—8.27%;两者同向指向真实时长更长。

结论:两缺口同向,均指向真实时长不小于 57.4724 h / 51.0906 h,故二者写作本模型族下界恰当;但不能把区间相加当作总区间。

13.5 传热传质系数与端面效应

(a)Sh 与 β :沿用 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $\beta = 8 \times 10^{-7} \text{ m/s}$ (A07),已各作 $\pm 10\%$ 、 $\pm 20\%$ 情景计算,报告为设置差异而非实测误差; β 敏感度远高于 h ,与"传质控制"一致。

(b)端面效应: $L/(2R_0) = 6.25$ 是一维径向近似的依据之一,但 $R_0/L = 0.08$ 不是 8% 的误差上界;二维轴对称对照给出 R^2 退化回归残差 3.46×10^{-18} 、端面对临界时间影响 7.6089 s(60×60 网格),仅支持"本组参数与输出量下影响较小",不能推广为"所有时刻与位置均可忽略"。

(c)辐射(仅量级核对,不进入模型):按"壁温等于气温 50 °C、表面 28 °C、发射率 0.9、等温黑环境全包围"假设,辐射换热系数 6.216461327 $\text{W}/(\text{m}^2 \cdot \text{K})$,辐射与对流热流之比 24.87%。该结果完全依赖上述壁温与发射率假设,是情景参数而不是实测值;题目明确 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 为对流换热系数,本文不声称其已包含辐射,也未求解含辐射的 PDE,不能把该通量比当作时长误差。

13.6 数值精度限制的如实记录

四位小数只是展示口径而非精度声明(10.11 节);问题三四的最坏差统计于"21 个固定半径 $\times$ 整数秒/60 s"的公共采样点,不是连续全域的严格上界(问题四真实表面 $x = 1$ 处差更大);系数保护下限不裁剪状态变量,本轨迹最小含水率远高于该下限而未被激活;本文不构造成 $\partial(BT)/\partial t$ 形式,故不出现"能量守恒检验通过"。

13.7 待解释口径汇总

表 36 七个待解释口径及其处理状态

编号	问题	本文的处理	状态
U01	烘房 kg/kg 的参考质量基准与材料平衡含水率的映射未明	按假设 A04 显式约定,并在 11.5 节构造数据锚定的吸附族	阻塞"无条件实测准确性"的声明
U02	附件 1 仅 4 h,而全过程 2—3 天	按 50 °C/0.05 平台延拓,并对照未值/末段均值/扰动(13.2 节)	无假设不能唯一预测 4 h 之后
U03	'result2.xlsx' 的终止时刻未在题面明确	保留至问题三达标后的复核秒 (206902 s),另导出前 3 h	不影响求解
U04	问题四半径不落在 0.1 cm 格点时表面与域外如何表达	固定物理距离列 + 独立"药材表面"列;域外留空	已在 'result4.xlsx' 实现
U05	题给 $\rho(C)$ 与严格干物质守恒不自动相容	有效热容量与干骨架守恒分开处理(11.1 节)	阻塞"全部严格守恒"的说法
U06	问题四若达标超过 72 h 则半径数据不足	本次 51.09 h < 72 h,未使用外推	不触发
U07	"严格低于 0.15"与四位显示 0.1500 的冲突	报告连续穿越时刻、严格达标的首个输出时刻与未舍入 $\max C$	不阻塞,已保留未舍入证据

13.8 本节"不能声称"清单

- 不能给出实测预测准确率、RMSE 或烘干时间置信区间(无内部实测数据),也不能声称连续时空的严格误差界(现有为有限采样口径的比较);
- 不能把所有四位小数的末位都称为稳定;
- 不能声称完整的多相物理已闭合(气固平衡、潜热、密度—体积相容均未闭合);
- 不能把情景范围写成置信区间,也不能把参数情景写成实测误差。

14 结论

本文建立圆柱形中药材热风烘干的轴对称一维径向热湿耦合模型(守恒有限体积、Kirchhoff 浓度势、隐式变阶变步长积分、解析 Jacobian)。

问题一 常热物性下预热 1800 s 时表面/中心温度 $36.7856^{\circ}\text{C}/33.5753^{\circ}\text{C}$, 表面含水率降至 1.5102 kg/kg 、距中心 1 cm 内基本不变;热与水分 Fourier 数之比约 34,温升远快于水分迁移(解析解对照偏差 $1.80 \times 10^{-6}\text{ K}$)。

问题二 附录 3 物性下温度约 2 h 趋于环境值,3 h 时表面 49.9664°C 、径向温差仅 0.1169°C ,而含水率梯度仍显著(中心 1.7662 、表面 1.0081 kg/kg) $\Rightarrow$ 预热后进入内部水分扩散控制的恒温干燥阶段。

问题三 以空间最大含水率严格低于 0.15 kg/kg 为判据,固定半径下需 57.4724 h (约 2.39 天),落在题目 2—3 天周期内;达标由药材中心控制(报告时刻中心未舍入 0.1499999 、表面 0.0527 kg/kg),事件由 ODE 事件与独立二分求根给出、相差 $5.8 \times 10^{-11}\text{ s}$ 。

问题四 计入附件 2 半径收缩并用附录 4 物性后需 51.0906 h (约 2.13 天),报告时刻半径收缩至 1.2000 cm 、未用 72 h 之后的外推;同物性对照表明收缩使达标时间由约 129.85 h 缩短至约 51.09 h (约 60.7%),并经独立 $(R_0/R)^2$ 尺度折算印证到 1.37% 以内。问题三、四同时改变物性与几何,时长差不能全归因于收缩。

模型的修正与创新 ① 题给 $\rho(C)$ 、半径轨迹、固定长度与严格干物质守恒不能同时精确成立,给出热物性闭合(必要性半径界 1.2112 cm);② 表面汽化耗能以潜热比例包络处理($L=1$:问题二三 +5.26%、问题四 +10.56%);③ 以 Kirchhoff 浓度势处理强非线性水分通量(调和平均在 $N=1600$ 仍偏离 $8.8 \times 10^{-3}\text{ h}$);④ 以材料坐标与干骨架守恒处理移动边界;⑤ 由附件 1 反推平台相对湿度 0.607,得锚点 $a_w(0.05)=0.607$,构造吸附闭合族($p=4$:+13.87% / +8.27%)。

稳健性与不确定性 达标时间在两个积分族与两种湿面离散格式下于三档网格一致到 10^{-6} s 量级,收缩效应与三阶段分界分别经尺度折算和四判据互验;本工况为传质控制(扩散前因子 -30%/+40% 使时长变化约 +38%/-25%,导热系数 $\pm 15\%$ 影响不到万分之一)。无内部实测数据,故本文为所列假设下的条件性预测、两个报告时长是闭合族下界,不作准确率或置信区间声明;报告时长中 92%—93% 落在环境观测窗之外。

可推广方向 ① 二维轴对称以显式处理端面;② 引入吸附/解吸等温线与表面蒸发能量平衡闭合气固界面;③ 以“变长度 + 变半径”骨架守恒替换同比收缩假设;④ 接工艺优化(品质约束下最小化能耗或时长)。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具,主要用于资料与规则核对、公式与代码复核、数值验证脚本编写、文稿一致性与格式检查,详细使用情况见支撑材料。

本参赛队对所提交作品的原创性、真实性和准确性负全部责任。本文的核心模型选择、关键闭合假设(等效平衡映射、有效显热容量、同比径向收缩)与最终数值结论均由本参赛队确定;AI 参与完成的公式、代码与文字内容已逐项人工审查与核实,凡与题目给定条件或计算结果不符者已予修改或弃用。用于支撑本文结论的全部源程序、输入数据、运行记录与校验证据一并提供于支撑材料。

参考文献

- [1] 2026 年高教社杯全国大学生数学建模竞赛 A 题:药材的烘干问题[Z]. 2026.(含附件 1、附件 2、附件 3、附录 2—4)
- [2] COMSOL. Heat Transfer Theory[EB/OL]. [2026-09-12].
https://doc.comsol.com/6.3/doc/com.comsol.help.comsol/comsol_ref_heattransfer.30.25.html
- [3] FAO. Crop Drying(Chapter 9: Moisture Content / Drying Theory)[EB/OL]. [2026-09-12].
<https://www.fao.org/4/s1250e/S1250E0u.htm>
- [4] COMSOL. Diffusion Coefficient[EB/OL]. [2026-09-12].
<https://www.comsol.com/multiphysics/diffusion-coefficient>
- [5] COMSOL. How to Model Heat and Moisture Transport in Porous Media[EB/OL]. [2026-09-12].
<https://www.comsol.com/blogs/how-to-model-heat-and-moisture-transport-in-porous-media-with-comsol>
- [6] COMSOL. Deformed Meshes[EB/OL]. [2026-09-12].
https://doc.comsol.com/6.3/doc/com.comsol.help.comsol/comsol_ref_deformedmeshes.34.04.html
- [7] ASHRAE. ASHRAE Handbook—Fundamentals, Chapter 1: Psychrometrics[EB/OL]. [2026-09-12].
https://handbook.ashrae.org/Handbooks/F25/SI/F25_Ch01/F25_Ch01_si.aspx
- [8] Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python[J]. Nature Methods, 2020, 17(3): 261-272.
- [9] SciPy. scipy.integrate.solve_ivp[EB/OL]. [2026-09-12].
https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html
- [10] NIST. NIST Digital Library of Mathematical Functions(指数积分与 Bessel 函数章节)[EB/OL]. [2026-09-12]. <https://dlmf.nist.gov/>
- [11] MathWorks. ode15s:Solve stiff differential equations and DAEs[EB/OL]. [2026-09-12].
<https://www.mathworks.com/help/matlab/ref/ode15s.html>
- [12] 杨世铭, 陶文铨. 传热学[M]. 4 版. 北京: 高等教育出版社, 2006.
- [13] Crank J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [14] Carslaw H S, Jaeger J C. Conduction of Heat in Solids[M]. 2nd ed. Oxford: Oxford University Press, 1959.

正文引用位置对照

文献	正文引用位置
[1]	1.1、1.2、第 3.1 节(数据来源)、第 5.7 节(题给经验式)
[2]	5.3.1(傅里叶定律)、5.4.1(热方程)
[3]	5.1.2(干基/湿基定义)、5.7.4(a)(比热容加权形式)
[4]	5.3.2(扩散系数与 Arrhenius 关系)
[5]	5.6.4(气固界面与相变)、11.2(潜热闭合)
[6]	5.5(材料坐标与移动网格)
[7]	5.6.4(含湿量 $\leftrightarrow$ 蒸汽分压换算)、11.5(式 (68))
[8] [9]	10.3.1(BDF/NDF 与刚性 ODE 求解)
[10]	10.2.3(指数积分 Ei)、10.4(Bessel 函数)
[11]	10.4 检验五(MATLAB 独立交叉核验)
[12]	2.3(Biot 数)、5.8(无量纲数)
[13]	5.4.2(圆柱内扩散)
[14]	10.4 检验一(圆柱 Bessel 级数解)

附录

附录 A 支撑材料文件清单

序号	文件/目录	说明
1	`AI 工具使用详情.pdf`	按 AI 规定第 4 条撰写的详情说明 (工具与版本、使用目的与环节、主要提示方式与过程、采纳与人工修改核验情况)
2	`result1.xlsx`	问题一完整结果(1800 s 内每 1 s、每 0.1 cm)
3	`result2.xlsx`	问题二完整结果(全过程每 1 s、每 0.1 cm)
4	`result3.xlsx`	问题三完整结果(每 60 s、每 0.1 cm)
5	`result4.xlsx`	问题四完整结果(每 60 s、每 0.1 cm, 含药材表面列)
6	`code/`	建模与求解全部 Python 源程序(正式入口、核心模块、解析 Jacobian、绘图与导出脚本)
7	`code/matlab/`	MATLAB 独立交叉核验脚本
8	`docs/`	公式与算法说明、参数与运行设置说明
9	`figures/`	正文图件的矢量版本(PDF/SVG)与绘图数据
10	`evidence/`	运行记录、校验输出、交叉验证结果

附录 B 完整可运行源程序代码

源码在支撑包中的目录与下表一致(`code/` 为经人工审查的交付版,`code/data`,`code/modeling`,`code/verification` 为产生正文全部数值的生产与验证脚本),共 40 个 Python 文件与 1 个 MATLAB 文件。

B.1 问题求解核心模块(`code/`)

文件	作用
`dryingCore.py`	物理模型与控制方程右端项:物性函数、内部面通量(调和平均与 Kirchhoff 势两种格式)、表面与中心边界、问题一至问题四的设置装配
`analyticJacobian.py`	解析稀疏 Jacobian 的装配,含式 (55)–(59) 的全部导数项与交叉项
`diskDense.py`	稠密输出的磁盘存储与任意时刻插值求值(式 (52) 后的说明)
`runDelivery.py`	正式求解入口:读入清洗后数据、逐问求解、事件定位与严格复核、导出四份结果工作簿与正文表格 CSV
`exportOutputs.py`	按附件 3 模板导出 `result1—4.xlsx`;固定物理半径取样、域外留空、真实表面独立列与四位小数舍入
`q1Model.py`——`q4Model.py`	四个问题的模型装配入口(物性、几何与判据的切换)

B.2 数据准备与生产流水线(`code/data/`,`code/modeling/`)

文件	作用
`prepare_a_data.py`	附件 1、附件 2 的读取、单位换算与分段线性插值,生成清洗后数据
`run_modeling.py`	生产求解驱动:按问题装配设置、调用核心模块、落盘全部数值摘要
`drying_core.py`,`analytic_jacobian.py`,`disk_dense.py`,`export_outputs.py`	与 B.1 交付版同源的生产实现(蛇形命名)
`publication_plots.py`	生成正文图件的矢量版本与绘图数据
`validate_bessel.py`	独立重写的圆柱 Bessel 级数解析解与节点 / 控制体平均两种口径的判别(式 (60)、式 (61))
`verify_convergence.py`	相邻网格在公共采样点上的收敛性检验
`verify_time_accuracy.py`	基准与更紧时间推进设置下的解差异与事件差
`compare_analytic.py`,`axisymmetric_check.py`	解析解对照与二维轴对称对照(端面效应)

B.3 验证与交叉核验脚本(`code/verification/`)

文件	作用
`crossvalidate_solver.py`	积分器互换(BDF/NDF 与 Radau IIA)与两种湿面离散格式对照
`method_comparison.py`	同方程两种离散(Kirchhoff 势与二阶守恒中点差分)对照,含 `fluxPathUsed` 守卫
`threshold_and_scaling_checks.py`	阈值事件的独立二分求根与 $(R_0/R)^2$ 尺度折算(式 (67))
`latent_heat_scenarios.py`	潜热情景包络(式 (65) 与表 27)
`isotherm_activity_closure.py`	数据锚定的吸附闭合情景族(式 (68)–(71) 与表 28)
`sensitivity_analysis.py`	环境延拓、 C_{eq} 、 h 、 β 、扩散前因子与导热系数的情景扫描(表 33)
`crossvalidate_series.py`、`compile_crossvalidation_report.py`	三阶段分界的多判据交叉识别与其汇总
`analytic_metric_check.py`、`energy_balance_check.py`	解析度量口径核对与能量恒等式构造限制的诊断(10.9 节检验二)

B.4 MATLAB 独立实现(`code/matlab/`)

文件	作用
`runCrossCheck.m`	独立有限体积离散 + `ode15s` 的跨实现交叉核验(10.9 节检验五)

@@CODE_BLOCK_START@@

下列清单为产生与核验本文全部数值的完整源程序(人工审查交付版、生产核心模块、验证与交叉核验脚本、MATLAB 独立实现),共 19 个文件、4868 行;完整工作目录(含数据准备、绘图与其余诊断脚本)一并提供于支撑材料。

B.5 问题求解交付版(人工审查版)

`review\_delivery/runDelivery.py` (正式求解入口:读入清洗数据、逐问求解、事件定位与严格复核、导出四份结果工作簿)

"""A 题四问可审查入口:求解、导出、回读、回归核验和运行留痕。

默认 final 配置重算 N3200/N3200/N6400; audit 配置采用 N40 对照旧实现。

本入口产生独立版本目录。GUI 操作记录和用户人工审查由人另行填写。

"""

```
from __future__ import annotations
```

```
import argparse
from contextlib import redirect_stderr, redirect_stdout
from datetime import datetime, timezone
import gc
import gzip
import hashlib
import importlib
import importlib.metadata
import json
import os
from pathlib import Path
import re
import sys
import time
import traceback
import warnings
```

```
# 必须在加载 NumPy/SciPy 前限制线程,避免三个大网格争用内存和 CPU。
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["OMP_NUM_THREADS"] = "1"
sys.dont_write_bytecode = True
projectRoot = Path(__file__).resolve().parents[3]
codeRoot = Path(__file__).resolve().parent
os.environ["MPLCONFIGDIR"] = str(projectRoot / "tmp/cache/matplotlib")
os.environ["TMPDIR"] = str(projectRoot / "tmp/cache/review_delivery")
os.environ["TEMP"] = os.environ["TMPDIR"]
os.environ["TMP"] = os.environ["TMPDIR"]
Path(os.environ["TMPDIR"]).mkdir(parents=True, exist_ok=True)
```

```

def utcNow():
    """UTC ISO 时间戳；北京时间可在日志中按 +08:00 换算。"""
    return datetime.now(timezone.utc).isoformat()

def writeJson(filePath, content):
    filePath.parent.mkdir(parents=True, exist_ok=True)
    filePath.write_text(json.dumps(content, ensure_ascii=False, indent=2,
                                   allow_nan=False), encoding="utf-8")

def fileRecord(filePath):
    """分块哈希避免把大型原精度文件整体读入内存。"""
    digest = hashlib.sha256()
    with filePath.open("rb") as fileHandle:
        for block in iter(lambda: fileHandle.read(1024 * 1024), b''):
            digest.update(block)
    return {"path": filePath.relative_to(projectRoot).as_posix(),
            "bytes": filePath.stat().st_size, "sha256": digest.hexdigest()}

class TeeStream:
    """同时保留 IDE 可见输出和 UTF-8 持久日志。"""
    def __init__(self, screen, logFile):
        self.screen, self.logFile = screen, logFile

    def write(self, text):
        self.screen.write(text)
        self.logFile.write(text)
        self.flush()
        return len(text)

    def flush(self):
        self.screen.flush()
        self.logFile.flush()

def compareArrays(actualPath, baselinePath):
    """独立读取两份 NPZ；要求同名数组逐值相等，含 NaN 位置。"""
    import numpy as np
    checks = []
    with np.load(actualPath, allow_pickle=False) as actual, np.load(
        baselinePath, allow_pickle=False) as baseline:
        if set(actual.files) != set(baseline.files):
            raise AssertionError("NPZ 字段不一致")
        for key in baseline.files:
            left, right = actual[key], baseline[key]
            equal = left.shape == right.shape and np.array_equal(left, right, equal_nan=True)
            if not equal:
                raise AssertionError(f"重命名后数值改变: {actualPath.name}/{key}")
            checks.append({"array": key, "shape": list(left.shape), "exactEqual": True})
    return {"actual": fileRecord(actualPath), "baseline": fileRecord(baselinePath),
            "status": "PASS", "arrays": checks}

def compareArchives(actualPath, baselinePath):
    """比较解压后的逐秒原精度 CSV，忽略 gzip 容器时间戳。"""
    checkedBytes = 0
    with gzip.open(actualPath, "rb") as actual, gzip.open(baselinePath, "rb") as baseline:
        while True:
            left, right = actual.read(1024 * 1024), baseline.read(1024 * 1024)
            if left != right:
                raise AssertionError(f"原精度 CSV 与冻结结果不同: {actualPath.name}")
            checkedBytes += len(left)
            if not left:
                break
    return {"status": "PASS", "uncompressedBytes": checkedBytes,
            "actual": fileRecord(actualPath), "baseline": fileRecord(baselinePath)}

```

```

def verifyFrozenBaseline():
    """以历史实际退出0的账本核对冻结输入/输出，避免拿变动文件作基准。"""
    manifestPath = projectRoot / "paper_output/results/production/final_v6a/run_manifest.json"
    manifest = json.loads(manifestPath.read_text(encoding="utf-8-sig"))
    frozenRun = manifest["runs"][0]
    if manifest["status"] != "PASS" or frozenRun["returncode"] != 0:
        raise RuntimeError("历史基准没有有效的生产退出记录")
    checkedRecords = []
    for record in frozenRun["input_files"] + frozenRun["output_artifacts"]:
        actualRecord = fileRecord(projectRoot / record["path"])
        if actualRecord["sha256"] != record["sha256"]:
            raise AssertionError("冻结基准哈希改变: " + record["path"])
        checkedRecords.append(actualRecord)
    return {"status": "PASS", "manifest": fileRecord(manifestPath), "checkedRecords": checkedRecords}

def runQuestion(questionKey, intervalCount, outputDirectory, profile):
    """一个轨迹求解一次；Q2和Q3共享Q23轨迹，导出后立即释放缓存。"""
    import numpy as np
    import dryingCore
    import q3Model
    moduleName = {"Q1": "q1Model", "Q23": "q2Model", "Q4": "q4Model"}[questionKey]
    questionModule = importlib.import_module(moduleName)
    questionDirectory = outputDirectory / questionKey
    baselineRoot = projectRoot / "paper_output/results/production/final_v6a"
    solutionRun = None
    try:
        print(f"{utcNow()} SOLVE {questionKey} N={intervalCount}", flush=True)
        # 人工审查断点：下一行进入逐问入口，再进入 solveCase / rhs.
        with warnings.catch_warnings(record=True) as caughtWarnings:
            warnings.simplefilter("always")
            solutionRun = questionModule.solve(intervalCount)
        if caughtWarnings:
            raise RuntimeError("求解器警告: " + "; ".join(str(item.message) for item in caughtWarnings))
        summary = dryingCore.saveRun(solutionRun, questionDirectory)
        summary["warningCount"] = len(caughtWarnings)
        if questionKey != "Q1":
            summary["completion"] = q3Model.completion(solutionRun)
        sampleTimes = np.unique(np.array([0., min(1800., solutionRun.endS),
            min(10800., solutionRun.endS), solutionRun.endS] + ([[] if questionKey == "Q1"
            else [summary["completion"]]["reported_time_s"]]))))
        sampleCoordinates = np.array([0., .25, .5, .75, 1.])
        temperature, moisture = solutionRun.fields(sampleTimes, materialX=sampleCoordinates)
        summary["crosslanguageSamples"] = {"timesS": sampleTimes.tolist(),
            "materialX": sampleCoordinates.tolist(), "temperatureK": temperature.tolist(),
            "moistureDryBasis": moisture.tolist()}
        writeJson(questionDirectory / "summary.json", summary)
        print(json.dumps({"question": questionKey, "diagnostics": summary["diagnostics"]},
            ensure_ascii=False), flush=True)
        comparison = {}
        if profile == "final":
            comparison["sampledSolution"] = compareArrays(questionDirectory / "sampled_solution.npz",
                baselineRoot / questionKey / "sampled_solution.npz")
            frozenSummary = json.loads((baselineRoot / "numerical_summaries.json").read_text(encoding="utf-8"))
        [questionKey]
        if summary["diagnostics"]["event_s"] != frozenSummary["diagnostics"]["event_s"]:
            raise AssertionError("连续临界事件与冻结基线不同")
        if questionKey != "Q1" and summary["completion"] != frozenSummary["completion"]:
            raise AssertionError("严格达标时刻与冻结基线不同")
        import exportOutputs
        comparison["exports"] = []
        questionIds = ["Q2", "Q3"] if questionKey == "Q23" else [questionKey]
        for questionId in questionIds:
            print(f"{utcNow()} EXPORT_AND_READBACK {questionId}", flush=True)
            exportResult = exportOutputs.exportQuestion(solutionRun, questionId,
                outputDirectory / "outputs")
            validation = exportOutputs.validateExports([exportResult], runs={questionId: solutionRun})
            if validation.get("status") != "PASS" or not validation.get("fully_verified_with_live_Run"):
                raise AssertionError("工作簿/原精度文件/live Run 核对失败")
            rawName = f"result{questionId[-1]}_unrounded.csv.gz"

```

```

        archiveCheck = compareArchives(outputDirectory / "outputs" / rawName,
                                       baselineRoot / "outputs" / rawName)

        tableChecks = []
        for tablePath in sorted((outputDirectory / "outputs").glob(questionId.lower() + "_paper_*.csv")):
            referencePath = baselineRoot / "outputs" / tablePath.name
            if tablePath.read_bytes() != referencePath.read_bytes():
                raise AssertionError("正文 CSV 与冻结结果不同: " + tablePath.name)
            tableChecks.append({"file": fileRecord(tablePath), "exactEqual": True})
        comparison["exports"].append({"questionId": questionId, "validation": validation,
                                     "archiveCheck": archiveCheck, "paperTables": tableChecks})

    writeJson(questionDirectory / "comparison.json", comparison)
    return summary, comparison
finally:
    if solutionRun is not None:
        solutionRun.close()
        solutionRun = None
    gc.collect()
    print(f"{utcNow()} RELEASED {questionKey}", flush=True)

def runLegacyAudit(outputDirectory):
    """固定评价器：N40 同参数旧源码真实重算，与本轮副本逐值比较。"""
    legacyRoot = projectRoot / "paper_output/code/modeling"
    sys.path.insert(0, str(legacyRoot))
    import drying_core as legacyCore
    checks = []
    for questionKey, moduleName in [("Q1", "q1_model"), ("Q23", "q2_model"), ("Q4", "q4_model")]:
        legacyRun = None
        try:
            print(f"{utcNow()} AUTORESEARCH_BASELINE {questionKey} N=40", flush=True)
            legacyRun = importlib.import_module(moduleName).solve(40)
            legacyDirectory = outputDirectory / "legacyBaseline" / questionKey
            legacyCore.save_run(legacyRun, legacyDirectory)
            checks.append(compareArrays(outputDirectory / questionKey / "sampled_solution.npz",
                                       legacyDirectory / "sampled_solution.npz"))
        finally:
            if legacyRun is not None:
                legacyRun.close()
            gc.collect()
    return {"status": "KEEP", "primaryMetric": "maximum absolute sampled field difference",
           "primaryMetricValue": 0., "criterion": "exact equality at fixed N40, same settings and inputs",
           "randomSeed": None, "deterministic": True, "checks": checks,
           "heldOutCheck": "final grid and all unrounded output rows against frozen final_v6a"}

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--profile", choices=["final", "audit"], default="final")
    parser.add_argument("--run-id", dest="runId", default=None)
    arguments = parser.parse_args()
    runId = arguments.runId or (arguments.profile + "_" + datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S"))
    if re.fullmatch(r"[A-Za-z0-9][A-Za-z0-9_-]{0,79}", runId) is None:
        raise ValueError("run-id 仅可含字母、数字、下划线和连字符")
    if Path.cwd().resolve() != projectRoot:
        raise RuntimeError("请在 2026CUMCM 根目录启动，或使用已配置的 VS 工程")
    outputDirectory = projectRoot / "paper_output/results/code_delivery" / runId
    outputDirectory.mkdir(parents=True, exist_ok=False)
    startedAt, timer = utcNow(), time.perf_counter()
    sourcePaths = sorted(codeRoot.glob("*.py"))
    dataPaths = sorted((projectRoot / "paper_output/data_cleaned").glob("A_*observed.csv"))
    templatePaths = sorted((projectRoot / "problem_files/CUMCM2026Problems/A 题/附件/附件 3").glob("result*.xlsx"))
    inputRecords = [fileRecord(filePath) for filePath in sourcePaths + dataPaths + templatePaths]
    launchRecord = {"startedAtUtc": startedAt, "runId": runId, "profile": arguments.profile,
                   "command": [sys.executable] + sys.argv, "cwd": str(Path.cwd()), "python": sys.version,
                   "libraries": {name: importlib.metadata.version(name) for name in ["numpy", "scipy", "openpyxl"]},
                   "inputRecords": inputRecords, "blasThreads": 1, "humanReview": "pending", "guiObserved": False,
                   "baseline": "paper_output/results/production/final_v6a", "seed": None,
                   "seedReason": "deterministic PDE solve; no pseudorandom numbers", "experimentBudgetSeconds": 1800}
    writeJson(outputDirectory / "launch.json", launchRecord)
    result = {"runId": runId, "startedAtUtc": startedAt, "status": "RUNNING",
             "humanReview": "pending", "guiObservation": "separate external evidence required",
             "actualProcessExitCode": None}

```

```

with (outputDirectory / "stdout.log").open("w", encoding="utf-8") as logFile:
    with redirect_stdout(TeeStream(sys.stdout, logFile)), redirect_stderr(TeeStream(sys.stderr, logFile)):
        try:
            print(f"{{startedAt}} START {{runId}} {{arguments.profile}}", flush=True)
            result["frozenBaselineVerification"] = verifyFrozenBaseline()
            summaries, comparisons = {}, {}
            for questionKey, finalIntervals in [("Q1", 3200), ("Q23", 3200), ("Q4", 6400)]:
                intervalCount = 40 if arguments.profile == "audit" else finalIntervals
                summaries[questionKey], comparisons[questionKey] = runQuestion(
                    questionKey, intervalCount, outputDirectory, arguments.profile)
            result["summaries"], result["comparisons"] = summaries, comparisons
            if arguments.profile == "audit":
                result["autoresearch"] = runLegacyAudit(outputDirectory)
                writeJson(outputDirectory / "autoresearch.json", result["autoresearch"])
            if inputRecords != [fileRecord(filePath) for filePath in sourcePaths + dataPaths + templatePaths]:
                raise RuntimeError("运行期间源码或输入发生改变, 结果不能签核")
            result["status"] = "PASS"
            print(f"{{utcNow()}} ALL_Q1_Q4_SOLVES_AND_CHECKS_COMPLETED", flush=True)
        except BaseException as error:
            result["status"], result["error"] = "FAIL", repr(error)
            traceback.print_exc()
            raise
        finally:
            result["finishedAtUtc"], result["elapsedSeconds"] = utcNow(), time.perf_counter() - timer
            writeJson(outputDirectory / "runResult.json", result)
            stablePaths = [filePath for filePath in sorted(outputDirectory.rglob("*"))
                           if filePath.is_file() and filePath.name not in ["stdout.log", "artifactManifest.json"]]
            writeJson(outputDirectory / "artifactManifest.json", {"status": result["status"],
                                                                  "createdAtUtc": utcNow(), "artifacts": [fileRecord(filePath) for filePath in stablePaths]})

if __name__ == "__main__":
    main()

`review_delivery/dryingCore.py` (物理模型与控制方程右端项、物性函数、内部面通量与边界条件)
# 本文件为冻结求解源码的驼峰审查副本; 来源、改名与 AST 核验见 tools/coreRenameReport.json.
"""2026 A: radial heat and dry-basis moisture transport on a material mesh.

Units: s, m, K, kg water / kg dry matter. See numerical_design.md for derivation.
The supplied empirical rho*cp is an effective thermal capacity. Dry-solid mass
is conserved separately on uniformly shrinking material control volumes.
No latent heat in the baseline; optional surface-latent scenario is labelled.
No clipping of solution values. Coefficients use a positive continuation only
for integrator Newton probes; all accepted states are checked independently.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path
import hashlib
import io
import json
import time

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import lil_matrix
from scipy.special import expi
import analyticJacobian
import diskDense

ROOT = Path(__file__).resolve().parents[3]
LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
C0, T0, R0, LENGTH = 2.55, 301.15, 0.02, 0.25

# 原始环境时间单位为秒, 温度为 K, 含水率为 kg 水 / kg 干物质; 记录输入哈希。
def loadInputs(withRecords=False):
    arrays, records = [], []
    for name in ['A_environment_observed.csv', 'A_radius_observed.csv']:
        path = ROOT / 'paper_output/data_cleaned' / name

```

```

        content = path.read_bytes()
        arrays.append(np.genfromtxt(io.StringIO(content.decode('utf-8-sig')),
                                    delimiter=',', names=True))
        records.append({'path':path.relative_to(ROOT).as_posix(), 'bytes':len(content),
                        'sha256':hashlib.sha256(content).hexdigest(), 'exists':True})
    return (*arrays, records) if withRecords else tuple(arrays)

@dataclass(frozen=True)
# 数值参数集中配置；四问正式设置由 q1Model/q2Model/q4Model 提供。
class Settings:
    question: str = 'Q23'
    intervals: int = 100
    rtol: float = 1e-7
    atolTemperature: float = 1e-7
    atolMoisture: float = 1e-9
    maxStepS: float = 600.0
    earlyMaxStepS: float = 30.0
    horizonH: float = 240.0
    shrink: bool = False
    boundaryExtension: str = 'nominal'
    tailTemperatureC: float = 50.0
    tailEquilibrium: float = 0.05
    h: float = 25.0
    beta: float = 8e-7
    equilibriumScale: float = 1.0
    surfaceLatentFraction: float = 0.0
    latentJKg: float = 2.4e6
    constantD: float | None = None
    constantThermal: bool = False
    method: str = 'BDF'
    faceScheme: str = 'harmonic'
    jacobianMode: str = 'analytic'
    denseStorage: str = 'disk'

class RadialModel:
    def __init__(self, settings: Settings):
        self.settings = settings
        if settings.intervals < 2 or settings.jacobianMode not in ('analytic', 'finite_difference'):
            raise ValueError('At least two intervals and a supported Jacobian mode are required')
        self.env, self.rad, self.inputRecords = loadInputs(withRecords=True)
        # x=r/R(t) 是无量纲材料坐标；控制体权重来自圆柱半径方向的积分。
        self.x = np.linspace(0., 1., settings.intervals + 1)
        self.dx = 1. / settings.intervals
        faces = np.r_[0., (self.x[1:] + self.x[:-1]) / 2., 1.]
        self.w = np.diff(faces ** 2) / 2.
        self.internalFaces = faces[1:-1]
        self.n = len(self.x)
        self.rhoD0 = (760 + 90 * C0) / (1 + C0) if settings.question == 'Q4' else (
            820. / (1 + C0) if settings.question == 'Q1' else (650 + 128 * C0) / (1 + C0))
        self.evaluations = 0
        self.jacPattern = self.buildSparsity()

    def radius(self, t):
        if self.settings.shrink:
            return np.interp(t, self.rad['time_s'], self.rad['radius_m'])
        return np.asarray(t) * 0. + R0

    # 4 h 之后采用已声明的平台延拓；不能把外推段称为实测环境。
    def environment(self, t):
        s = self.settings
        tair = np.interp(t, self.env['time_s'], self.env['temperature_K'])
        ceq = np.interp(t, self.env['time_s'], self.env['air_moisture_kg_per_kg'])
        after = np.asarray(t) > self.env['time_s'][-1]
        if s.boundaryExtension == 'nominal':
            tair = np.where(after, s.tailTemperatureC + 273.15, tair)
            ceq = np.where(after, s.tailEquilibrium, ceq)
        elif s.boundaryExtension == 'tail_mean':
            tail = self.env['time_s'] >= 10800
            tair = np.where(after, self.env['temperature_K'][tail].mean(), tair)

```

```

        ceq = np.where(after, self.env['air_moisture_kg_per_kg'][tail].mean(), ceq)
    elif s.boundaryExtension != 'last':
        raise ValueError('Unknown boundary extension')
    return tair, ceq * s.equilibriumScale

# rho*cp 为有效显热体积容量；干物质量通过独立的积分守恒式约束。
def properties(self, T, C):
    s = self.settings
    # 仅延拓 Newton 试探点的系数；不裁剪被接受的温度或含水率状态。
    positiveC = np.maximum(C, 1e-12) # coefficient continuation, never state clipping
    if np.any(T <= 0):
        raise FloatingPointError('Nonpositive absolute temperature')
    wet = positiveC / (1. + positiveC)
    if s.question == 'Q1':
        rho = np.full_like(C, 820.)
        cp = np.full_like(C, 2600.)
        k = np.full_like(C, .36)
        D = 7e-9 * np.exp(-.89 / positiveC)
    elif s.question in ('Q2', 'Q3', 'Q23'):
        rho, cp, k = 650 + 128 * positiveC, 1450 + 2736 * wet, .21 + .38 * wet
        D = 2.4e-3 * np.exp(-.45 / positiveC - 3850 / T)
    elif s.question == 'Q4':
        rho, cp, k = 760 + 90 * positiveC, 1850 + 2150 * wet, .12 + .20 * wet
        D = 4.2e-4 * np.exp(-.30 / positiveC - 3850 / T)
    else:
        raise ValueError(s.question)
    if s.constantD is not None:
        D = np.full_like(C, s.constantD)
    if s.constantThermal:
        rho, cp, k = np.full_like(C, 820.), np.full_like(C, 2600.), np.full_like(C, .36)
    return rho, cp, k, D

@staticmethod
def harmonic(a):
    return 2 * a[:-1] * a[1:] / np.maximum(a[:-1] + a[1:], np.finfo(float).tiny)

def buildSparsity(self):
    p = lil_matrix((2 * self.n + 1, 2 * self.n + 1), dtype=int)
    for i in range(self.n):
        for j in range(max(0, i - 1), min(self.n, i + 2)):
            p[2*i:2*i+2, 2*j:2*j+2] = 1
    p[-1, 2 * (self.n - 1) + 1] = 1
    return p.tocsr()

# Kirchhoff 假设 D(C), 温度因子在同一面上取值。
def waterInternalFlux(self, T, C, D):
    if self.settings.faceScheme == 'harmonic' or self.settings.constantD is not None:
        return self.internalFaces * self.harmonic(D) * np.diff(C) / self.dx
    if self.settings.faceScheme != 'kirchhoff':
        raise ValueError('Unknown nonlinear face scheme')
    a, D0 = {'Q1':(.89,7e-9), 'Q23':(.45,2.4e-3), 'Q2':(.45,2.4e-3),
            'Q3':(.45,2.4e-3), 'Q4':(.30,4.2e-4)}[self.settings.question]
    cc = np.maximum(C, 1e-12)
    # 势 F(C)=C*exp(-a/C)+a*Ei(-a/C), 导数为 exp(-a/C)。
    potential = cc * np.exp(-a/cc) + a * expi(-a/cc)
    difference = np.diff(potential)
    small = np.abs(np.diff(cc)) < 1e-7 * np.maximum((cc[:-1]+cc[1:])/2, 1e-3)
    difference[small] = (np.exp(-a/((cc[:-1][small]+cc[1:][small])/2)) * np.diff(cc)[small])
    # 不能对温度因子乘势后的整体作差，否则会引入题设没有的交叉扩散通量。
    thermalFactor = 1. if self.settings.question == 'Q1' else np.exp(-3850/((T[:-1]+T[1:])/2))
    # Do not difference thermal_factor*potential: that would add a false Soret flux.
    return self.internalFaces * D0 * thermalFactor * difference / self.dx

# 状态交错排列 T0,C0,T1,C1,..., 最后一项累计平均失水；共享面通量保证离散守恒。
def rhs(self, t, state):
    """REVIEW: actual material-control-volume balance, no extra mesh advection."""
    self.evaluations += 1
    T, C = state[:-1:2], state[1:-1:2]
    rho, cp, k, D = self.properties(T, C)
    radius = float(self.radius(t))
    tair, ceq = self.environment(t)
    heatG = np.zeros(self.n + 1)

```

```

waterG = np.zeros(self.n + 1)
heatG[1:-1] = self.internalFaces * self.harmonic(k) * np.diff(T) / self.dx
waterG[1:-1] = self.waterInternalFlux(T, C, D)
# 表面对流传质和传热采用外法向流出约定；中心面面积为零。
waterG[-1] = -self.settings.beta * radius * (C[-1] - ceq)
heatG[-1] = -self.settings.h * radius * (T[-1] - tair)
if self.settings.surfaceLatentFraction:
    # Scenario: all selected outgoing water vaporizes at the surface.
    rhoD = self.rhoD0 * (R0 / radius) ** 2
    jEvap = rhoD * self.settings.beta * (C[-1] - ceq)
    heatG[-1] -= (radius * self.settings.surfaceLatentFraction *
                  self.settings.latentJKg * jEvap)
derivative = np.empty_like(state)
# 除以 R(t)^2 与控制权重得到材料导数；同比收缩无需额外网格对流项。
derivative[:-1:2] = np.diff(heatG) / (radius ** 2 * self.w * rho * cp)
derivative[1:-1:2] = np.diff(waterG) / (radius ** 2 * self.w)
derivative[-1] = 2 * self.settings.beta / radius * (C[-1] - ceq)
return derivative

def initial(self):
    state = np.empty(2 * self.n + 1)
    state[1:-1:2], state[1:-1:2], state[-1] = T0, C0, 0.
    return state

class Run:
    def __init__(self, model, pieces, elapsed, eventS, cache=None):
        self.model, self.pieces = model, pieces
        self.cache = cache
        self.elapsedS, self.eventS = elapsed, eventS
        self.endS = float(pieces[-1].t[-1])

    def close(self):
        """Release this Run after its exports/checks; further queries are invalid."""
        self.pieces.clear()
        if self.cache is not None:
            self.cache.close()

    def state(self, times):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        if np.min(tt) < -1e-10 or np.max(tt) > self.endS + 1e-7:
            raise ValueError('Requested time outside solved interval')
        out = np.empty((2 * self.model.n + 1, len(tt)))
        remaining = np.ones(len(tt), dtype=bool)
        for result in self.pieces:
            select = remaining & (tt >= result.t[0]-1e-7) & (tt <= result.t[-1]+1e-7)
            if np.any(select):
                out[:, select] = result.sol(tt[select])
                remaining[select] = False
        if np.any(remaining):
            raise RuntimeError('Missing dense solution segment')
        return out

# materialX 查询材料坐标；radiiM 查询实际米制半径，收缩域外返回 NaN。
def fields(self, times, radiiM=None, materialX=None):
    tt = np.atleast_1d(np.asarray(times, dtype=float))
    state = self.state(tt)
    Ts, Cs = state[:-1:2].T, state[1:-1:2].T
    if materialX is not None:
        points = np.asarray(materialX, dtype=float)
        if np.any(~np.isfinite(points)) or np.any((points < 0.) | (points > 1.)):
            raise ValueError('Material coordinates must be finite and within [0,1]')
        return np.array([np.interp(points, self.model.x, row) for row in Ts]), np.array([
            np.interp(points, self.model.x, row) for row in Cs])
    if radiiM is None:
        return Ts, Cs
    radial = np.asarray(radiiM)
    Tout, Cout = [], []
    for i, t in enumerate(tt):
        xx = radial / self.model.radius(t)
        Tout.append(np.interp(xx, self.model.x, Ts[i], left=np.nan, right=np.nan))

```

```

        Cout.append(np.interp(xx, self.model.x, Cs[i], left=np.nan, right=np.nan))
    return np.asarray(Tout), np.asarray(Cout)

# 每块至多 256 个被接受时刻，避免对高网格状态和物性数组再做整域复制。
def diagnostics(self):
    # Reduce in bounded blocks: a fine full-domain run may contain tens of
    # millions of accepted state values. Diagnostics must not duplicate all
    # of them and four property arrays at the same time.
    minimumC = minimumT = minimumD = minimumProperty = np.inf
    maximumC = maximumT = maximumD = maximumRadialIncrease = -np.inf
    maximumMassResidual = 0.
    for piece in self.pieces:
        for first in range(0, len(piece.t), 256):
            raw = piece.y[:, first:first+256]
            T, C = raw[:-1:2], raw[1:-1:2]
            residual = 2*self.model.w*C + raw[-1] - C0
            rho, cp, k, D = self.model.properties(T.ravel(), C.ravel())
            minimumC, maximumC = min(minimumC, C.min()), max(maximumC, C.max())
            minimumT, maximumT = min(minimumT, T.min()), max(maximumT, T.max())
            minimumD = min(minimumD, D.min())
            maximumD = max(maximumD, D.max())
            minimumProperty = min(minimumProperty, rho.min(), cp.min(), k.min(), D.min())
            maximumRadialIncrease = max(maximumRadialIncrease, np.max(np.diff(C, axis=0)))
            maximumMassResidual = max(maximumMassResidual, np.max(np.abs(residual)))
    final = self.state([self.endsS])[:, 0]
    finalC = final[1:-1:2]
    return {
        'event_s': self.eventS, 'event_h': None if self.eventS is None else self.eventS/3600,
        'end_s': self.endsS, 'elapsed_s': self.elapsedS,
        'end_time_convention': 'ceil(critical_event_s)+1: conservative post-crossing verification second; not
claimed earliest integer second',
        'max_mass_balance_abs_kg_per_kg': float(maximumMassResidual),
        'min_C': float(minimumC), 'max_C': float(maximumC),
        'min_T_K': float(minimumT), 'max_T_K': float(maximumT),
        'min_D': float(minimumD), 'max_D': float(maximumD),
        'positive_properties': bool(minimumProperty > 0),
        'max_radial_C_increase': float(maximumRadialIncrease),
        'final_max_C': float(finalC.max()), 'final_surface_C': float(finalC[-1]),
        'strictly_dry_at_end': bool(finalC.max() < .15),
        'radius_end_m': float(self.model.radius(self.endsS)),
        'radius_extrapolation_used': bool(self.model.settings.shrink and self.endsS > 259200),
        'rhs_evaluations': self.model.evaluations,
        'accepted_time_points': sum(len(p.t) for p in self.pieces),
        'nfev': sum(p.nfev for p in self.pieces),
        'njev': sum(p.njev for p in self.pieces), 'nlu': sum(p.nlu for p in self.pieces),
        'solver_success': all(p.success for p in self.pieces),
        'dense_storage': self.model.settings.denseStorage,
        'dense_coefficient_bytes': 0 if self.cache is None else self.cache.bytesWritten,
        'dense_polynomial_count': 0 if self.cache is None else self.cache.polynomialCount,
    }

def solveCase(settings: Settings) -> Run:
    started = time.perf_counter()
    if settings.denseStorage not in ('memory', 'disk'):
        raise ValueError('Dense storage must be memory or disk')
    if settings.denseStorage == 'disk' and settings.method != 'BDF':
        raise ValueError('Exact disk dense storage currently supports BDF only')
    cache = diskDense.DenseCache(ROOT) if settings.denseStorage == 'disk' else None
    try:
        return solveCaseImpl(settings, cache, started)
    except BaseException as error:
        if cache is not None:
            try:
                cache.close()
            except BaseException as cleanup_error:
                error.add_note('Private cache cleanup also failed: '+repr(cleanup_error))
        raise

def solveCaseImpl(settings, cache, started):
    model = RadialModel(settings)

```

```

method = diskDense.DiskBDF if cache is not None else settings.method
jacobianOptions = ({'jac': lambda t, y: analyticJacobian.jacobian(model, t, y)}
    if settings.jacobianMode == 'analytic' else {'jac_sparsity': model.jacPattern})
if cache is not None:
    jacobianOptions['dense_cache'] = cache
# 连续事件取整个离散材料域 max(C)=0.15 ; 严格达标还须在事件后重新检查。
def dryEvent(t, y):
    return float(np.max(y[1:-1:2]) - .15)
dryEvent.terminal, dryEvent.direction = True, -1
atol = np.empty(2 * model.n + 1)
atol[1:-1:2], atol[1:-1:2], atol[-1] = settings.atolTemperature, settings.atolMoisture, settings.atolMoisture
horizon = 1800. if settings.question == 'Q1' else settings.horizonH * 3600.
pieces, initial, eventS = [], model.initial(), None
# A separate segment at 4 h makes the modelling extension explicit.
endpoints = [0., min(14400., horizon)]
if horizon > 14400.:
    endpoints.append(horizon)
for left, right in zip(endpoints[:-1], endpoints[1:]):
    piece = solve_ivp(model.rhs, (left, right), initial, method=method,
        rtol=settings.rtol, atol=atol, **jacobianOptions,
        max_step=settings.earlyMaxStepS if left < 14400. else settings.maxStepS,
        events=None if settings.question == 'Q1' else dryEvent, dense_output=True)
    if cache is not None:
        diskDense.alignBdfSegments(piece)
    pieces.append(piece)
    if not piece.success:
        raise RuntimeError(piece.message)
    initial = piece.y[:, -1].copy()
    if cache is not None:
        piece.y = cache.storeAccepted(piece.y)
    if piece.t_events is not None and len(piece.t_events[0]):
        eventS = float(piece.t_events[0][0])
        # Continue to a genuine post-crossing integer second, not an extrapolation.
        # 从实际事件状态继续积分至 ceil(event)+1 秒, 不依赖外推或四位舍入。
        end = float(np.ceil(eventS) + 1)
        tail = solve_ivp(model.rhs, (eventS, end), initial, method=method,
            rtol=settings.rtol, atol=atol, **jacobianOptions,
            max_step=1., dense_output=True)
        if cache is not None:
            diskDense.alignBdfSegments(tail)
        if not tail.success:
            raise RuntimeError(tail.message)
        if cache is not None:
            tail.y = cache.storeAccepted(tail.y)
        pieces.append(tail)
        break
run = Run(model, pieces, time.perf_counter() - started, eventS, cache)
diagnostic = run.diagnostics()
if diagnostic['min_C'] < -1e-8 or not diagnostic['positive_properties']:
    raise FloatingPointError('Physical range/positive property check failed')
if diagnostic['max_mass_balance_abs_kg_per_kg'] > 1e-6:
    raise FloatingPointError('Dry-basis mass balance failed')
return run

def fileRecord(path):
    p = Path(path)
    return {'path': p.relative_to(ROOT).as_posix(), 'bytes': p.stat().st_size,
        'sha256': hashlib.sha256(p.read_bytes()).hexdigest(), 'exists': True}

# 保存 60 s 等审查采样、源码/输入哈希和状态; 整秒题表由 exportOutputs 直接查询 live Run。
def saveRun(run: Run, directory: Path):
    codeRecord = fileRecord(Path(__file__))
    if codeRecord['sha256'] != LOADED_CODE_SHA256:
        raise RuntimeError('Solver file changed after import; restart to obtain valid provenance')
    jacobianRecord = fileRecord(Path(analyticJacobian.__file__))
    if jacobianRecord['sha256'] != analyticJacobian.LOADED_CODE_SHA256:
        raise RuntimeError('Jacobian file changed after import; restart to obtain valid provenance')
    storageRecord = fileRecord(Path(diskDense.__file__))
    if storageRecord['sha256'] != diskDense.LOADED_CODE_SHA256:
        raise RuntimeError('Dense storage file changed after import; restart for valid provenance')

```

```

for record in run.model.inputRecords:
    if fileRecord(ROOT/record['path'])['sha256'] != record['sha256']:
        raise RuntimeError('Input changed after being loaded; retain failure and rerun')
directory.mkdir(parents=True, exist_ok=True)
times = np.unique(np.r_[np.arange(0., run.endS, 60.),
    [t for t in [100., 300., 600., 900., 1200., 1500., 1800., 3600., 5400., 7200., 9000., 10800.] if t
<= run.endS],
    run.endS, [] if run.eventS is None else [run.eventS]])
x = np.linspace(0., 1., 21)
temperature, moisture, means, losses = [], [], [], []
for first in range(0, len(times), 128):
    blockTimes = times[first:first+128]
    Tb, Cb = run.fields(blockTimes, materialX=x)
    raw = run.state(blockTimes)
    temperature.append(Tb); moisture.append(Cb)
    means.append(2*run.model.w@raw[1:-1:2]); losses.append(raw[-1].copy())
T, C = np.vstack(temperature), np.vstack(moisture)
np.savez_compressed(directory/'sampled_solution.npz', times_s=times, material_x=x,
    T_K=T, C=C, radius_m=run.model.radius(times), mean_C=np.concatenate(means),
    cumulative_loss=np.concatenate(losses))
summary = {'settings': asdict(run.model.settings), 'diagnostics': run.diagnostics(),
    'code': codeRecord,
    'jacobian_code': jacobianRecord,
    'dense_storage_code': storageRecord,
    'inputs': run.model.inputRecords,
    'human_review_status': 'pending', 'gui_reproduced': False}
(directory/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
return summary

`review_delivery/analyticJacobian.py` (解析稀疏 Jacobian 装配(式 (55)—(59)))
# 本文件为冻结求解源码的驼峰审查副本; 来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Analytic sparse Jacobian for drying_core.RadialModel.rhs.

```

The ordering is T0,C0,...,TN,CN,A. A is a passive cumulative loss variable; its entire column is exactly zero and must not use adaptive numdiff factors. This file does not modify the core or select a production configuration.

Self-check from the contest root:

```

C:\Python314\python.exe -B paper_output/code/modeling/analytic_jacobian.py --self-test
"""

```

```

from __future__ import annotations

```

```

import argparse
from dataclasses import asdict
from datetime import datetime, timezone
import hashlib
import json
from pathlib import Path
import sys
import time
import warnings

```

```

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import coo_matrix
from scipy.special import expi

```

```

LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

```

调和平均面对左右节点的解析偏导，供热通量与调和水通量使用。

```

def harmonicPartials(left, right):
    """Positive-coefficient partials; respect the core's tiny denominator guard."""
    total = left + right
    tiny = np.finfo(float).tiny
    denominator = np.maximum(total, tiny)
    ordinary = total > tiny
    dl = np.where(ordinary, 2 * (right / denominator) ** 2, 2 * right / denominator)
    dr = np.where(ordinary, 2 * (left / denominator) ** 2, 2 * left / denominator)
    return dl, dr

```

```
# 稀疏 Jacobian 按交错 T/C 状态组装；保留物性、水通量与容量分母的全部链式法则项。
def jacobian(model, t, y):
    """Return d(rhs)/d(y) as CSC without calling rhs or numerical differentiation.
```

Each internal face contributes opposite flux derivatives to its two cells.
 For heat, differentiating $1/(\rho \cdot c_p)$ adds $-T_{dot} \cdot c_p / c_p$ locally.
 Kirchhoff's temperature factor is frozen at the symmetric face temperature;
 the primitive's close-concentration branch is differentiated exactly as coded.

```
"""
    state = np.asarray(y, dtype=float)
    n = model.n
    if state.ndim != 1 or state.size != 2*n + 1:
        raise ValueError("Jacobian requires interleaved 1D state of length 2*n+1")
    s = model.settings
    T, C = state[:-1:2], state[1:-1:2]
    cc = np.maximum(C, 1e-12)
    active = (C > 1e-12).astype(float)
    rho, cp, k, D = model.properties(T, C)
    if s.question == 'Q1':
        a, d0, tempConstant = .89, 7e-9, 0.
        rhoC, cpC, kC = np.zeros(n), np.zeros(n), np.zeros(n)
    elif s.question in ('Q2', 'Q3', 'Q23'):
        a, d0, tempConstant = .45, 2.4e-3, 3850.
        rhoC = np.full(n, 128.) * active
        cpC = 2736. / (1. + cc)**2 * active
        kC = .38 / (1. + cc)**2 * active
    elif s.question == 'Q4':
        a, d0, tempConstant = .30, 4.2e-4, 3850.
        rhoC = np.full(n, 90.) * active
        cpC = 2150. / (1. + cc)**2 * active
        kC = .20 / (1. + cc)**2 * active
    else:
        raise ValueError(s.question)
    if s.constantThermal:
        rhoC, cpC, kC = np.zeros(n), np.zeros(n), np.zeros(n)
    capacity = rho * cp
    capacityC = rhoC * cp + rho * cpC
    if s.constantD is None:
        dC = D * (a / cc**2) * active
        dT = D * tempConstant / T**2
    else:
        dC, dT = np.zeros(n), np.zeros(n)

    radius = float(model.radius(t))
    if radius <= 0:
        raise ValueError("Nonpositive radius")
    tair, ceq = model.environment(t)
    geom = model.internalFaces / model.dx
    deltaT, deltaC = np.diff(T), np.diff(C)

    # Derivative columns for each face are (T_left, C_left, T_right, C_right).
    kHarm = model.harmonic(k)
    khL, khR = harmonicPartials(k[:-1], k[1:])
    # 每个面的四列依次为 T左、C左、T右、C右，对相邻两个控制体施加相反符号。
    heatDeriv = np.column_stack((
        -geom * kHarm,
        geom * khL * kC[:-1] * deltaT,
        geom * kHarm,
        geom * khR * kC[1:] * deltaT,
    ))
    heatG = np.zeros(n + 1)
    heatG[1:-1] = geom * kHarm * deltaT
    heatG[-1] = -s.h * radius * (T[-1] - tair)

    if s.faceScheme == 'harmonic' or s.constantD is not None:
        dh = model.harmonic(D)
        dhL, dhR = harmonicPartials(D[:-1], D[1:])
        waterDeriv = np.column_stack((
            geom * dhL * dT[:-1] * deltaC,
```

```

        geom * (dhL * dC[:-1] * deltaC - dh),
        geom * dhR * dT[1:] * deltaC,
        geom * (dhR * dC[1:] * deltaC + dh),
    ))
elif s.faceScheme == 'kirchhoff':
    primitive = cc * np.exp(-a / cc) + a * expi(-a / cc)
    primitiveDelta = np.diff(primitive)
    meanC = (cc[:-1] + cc[1:]) / 2.
    meanT = (T[:-1] + T[1:]) / 2.
    deltaCc = np.diff(cc)
    small = np.abs(deltaCc) < 1e-7 * np.maximum(meanC, 1e-3)
    expLeft, expRight = np.exp(-a/cc[:-1]), np.exp(-a/cc[1:])
    primitiveL = -expLeft * active[:-1]
    primitiveR = expRight * active[1:]
    # 近等浓度时对实际 RHS 使用的中点分支求导, 避免与求解器分支不一致。
    if np.any(small):
        fMid = np.exp(-a / meanC[small])
        fPrimeMid = fMid * a / meanC[small]**2
        primitiveDelta[small] = fMid * deltaCc[small]
        primitiveL[small] = (0.5*fPrimeMid*deltaCc[small]-fMid)*active[:-1][small]
        primitiveR[small] = (0.5*fPrimeMid*deltaCc[small]+fMid)*active[1:][small]
    temperatureFactor = np.exp(-tempConstant / meanT)
    coefficient = geom * d0 * temperatureFactor
    waterG = coefficient * primitiveDelta
    waterT = waterG * tempConstant / (2 * meanT**2)
    waterDeriv = np.column_stack((waterT, coefficient*primitiveL,
                                   waterT, coefficient*primitiveR))
else:
    raise ValueError('Unknown nonlinear face scheme')

heatScale = 1. / (radius**2 * model.w * capacity)
waterScale = 1. / (radius**2 * model.w)
surfaceHeatC = 0.
if s.surfaceLatentFraction:
    rhoD = model.rhoD0 * (.02 / radius)**2
    surfaceHeatC = -radius*s.surfaceLatentFraction*s.latentJKg*rhoD*s.beta
    heatG[-1] += surfaceHeatC * (C[-1] - ceq)
tempDerivative = np.diff(heatG) * heatScale

faceIndex = np.arange(n-1)
faceColumns = np.column_stack((2*faceIndex, 2*faceIndex+1,
                                2*faceIndex+2, 2*faceIndex+3)).ravel()
rows, columns, values = [], [], []

def addFace(rowIndex, derivatives, factor):
    rows.append(np.repeat(rowIndex, 4))
    columns.append(faceColumns)
    values.append((derivatives * factor[:, None]).ravel())

addFace(2*faceIndex, heatDeriv, heatScale[:-1])
addFace(2*faceIndex+2, heatDeriv, -heatScale[1:])
addFace(2*faceIndex+1, waterDeriv, waterScale[:-1])
addFace(2*faceIndex+3, waterDeriv, -waterScale[1:])
cellIndex = np.arange(n)
rows.append(2*cellIndex)
columns.append(2*cellIndex+1)
# 热容量随 C 改变, 必须保留 -Tdot*(容量对 C 偏导)/容量 这一局部项。
values.append(-tempDerivative * capacityC / capacity)
rows.append(np.array([2*n-2, 2*n-2, 2*n-1, 2*n]))
columns.append(np.array([2*n-2, 2*n-1, 2*n-1, 2*n-1]))
values.append(np.array([-s.h*radius*heatScale[-1],
                        surfaceHeatC*heatScale[-1],
                        -s.beta*radius*waterScale[-1], 2*s.beta/radius]))
# 面模板只有邻近耦合; COO 合并重复贡献后转 CSC, 交给 BDF 稀疏线性求解。
matrix = coo_matrix((np.concatenate(values),
                     (np.concatenate(rows), np.concatenate(columns))),
                    shape=(2*n+1, 2*n+1)).tocsc()
matrix.sum_duplicates()
matrix.eliminate_zeros()
return matrix

```

此历史自检入口仅验证导数和小网格接线，不能替代正式网格与人工代码审核。

```
def selfTest(outputDirectory):
    """Independent RHS perturbation checks, a conservation derivative, and tiny BDF runs."""
    import scipy
    from dryingCore import ROOT, Settings, RadialModel, LOADED_CODE_SHA256 as CORE_LOADED_CODE_SHA256

    started = time.perf_counter()
    rng = np.random.default_rng(20260910)
    relativeTolerance, absoluteTolerance = 5e-6, 5e-10
    cases = []
    for question in ['Q1', 'Q23', 'Q4']:
        for scheme in ['harmonic', 'kirchhoff']:
            for latent in [0., 1.]:
                cases.append(Settings(question=question, intervals=8, faceScheme=scheme,
                                     shrink=(question=='Q4'), surfaceLatentFraction=latent))
    for question in ['Q1', 'Q23', 'Q4']:
        for constantDiffusivity, constantThermal in [(2e-9, False), (None, True), (2e-9, True)]:
            cases.append(Settings(question=question, intervals=8, faceScheme='kirchhoff',
                                 shrink=(question=='Q4'), constantD=constantDiffusivity,
                                 constantThermal=constantThermal, surfaceLatentFraction=1.))

    records, failures = [], []
    for settings in cases:
        model = RadialModel(settings)
        x = model.x
        states = {
            'initial': (0., model.initial()),
            'nonuniform': (18000., model.initial()),
            'late_dry': (150000., model.initial()),
            'near_uniform_small_branch': (14401., model.initial()),
        }
        states['nonuniform'][1][:1:2] = 303. + 17.*x**2
        states['nonuniform'][1][1:-1:2] = 2.4 - 2.0*x**2
        states['late_dry'][1][:1:2] = 321. + 2.0*x**2
        states['late_dry'][1][1:-1:2] = .175 - .115*x**2
        states['near_uniform_small_branch'][1][:1:2] = 303. + 17.*x**2
        states['near_uniform_small_branch'][1][1:-1:2] = .15 + 1e-10*x
        for stateName, (t, y) in states.items():
            with warnings.catch_warnings(record=True) as captured:
                warnings.simplefilter('always')
                matrix = jacobian(model, t, y)
                vectors = []
                for _ in range(4):
                    direction = rng.normal(size=y.size)
                    direction[:1:2] *= 1.0
                    direction[1:-1:2] *= .02
                    direction[-1] = .3
                    vectors.append(direction)
                directional = []
                for direction in vectors:
                    predicted = matrix @ direction
                    steps = []
                    for step in [1e-4, 3e-5, 1e-5]:
                        finiteDifference = (model.rhs(t, y+step*direction)-
                                           model.rhs(t, y-step*direction))/(2*step)
                        absoluteError = float(np.max(np.abs(predicted-finiteDifference)))
                        scale = max(float(np.max(np.abs(predicted))),
                                   float(np.max(np.abs(finiteDifference))), 1e-30)
                        scaled = float(np.max(np.abs(predicted-finiteDifference)/
                                                (absoluteTolerance + relativeTolerance*np.maximum(
                                                    np.abs(predicted), np.abs(finiteDifference))))))
                        steps.append({'step':step, 'max_abs_error':absoluteError,
                                    'relative_inf_error':absoluteError/scale,
                                    'max_component_tolerance_ratio':scaled})
                    best = min(steps, key=lambda item:item['max_component_tolerance_ratio'])
                    directional.append({'all_step_errors':steps, 'best':best})
                passive = np.zeros(y.size); passive[-1] = 1.
                passiveAnalytic = float(np.max(np.abs(matrix @ passive)))
                passiveFd = float(np.max(np.abs(model.rhs(t, y+passive)-model.rhs(t,y-passive))))
                massWeights = np.zeros(y.size)
                massWeights[1:-1:2] = 2*model.w
                massWeights[-1] = 1.
                conservation = float(np.max(np.abs(np.asarray(massWeights @ matrix))))
```

```

warningMessages = [str(w.message) for w in captured]
passed = (all(step['max_component_tolerance_ratio'] <= 1
               for d in directional for step in d['all_step_errors'])
          and passiveAnalytic == 0 and passiveFd == 0 and conservation < 1e-11
          and np.isfinite(matrix.data).all() and not warningMessages)
record = {'settings':asdict(settings), 'state':stateName, 'time_s':t,
          'status':'PASS' if passed else 'FAIL', 'matrix_shape':matrix.shape,
          'matrix_nnz':matrix.nnz, 'directional_checks':directional,
          'passive_column_analytic_abs':passiveAnalytic,
          'passive_column_finite_difference_abs':passiveFd,
          'mass_balance_derivative_abs':conservation, 'warnings':warningMessages}
records.append(record)
if not passed:

failures.append(f"{settings.question}/{settings.faceScheme}/latent={settings.surfaceLatentFraction}/{stateName}/
constant_D={settings.constantD}/constant_thermal={settings.constantThermal}")
smokeRecords = []
for scheme in ['harmonic', 'kirchhoff']:
    for question in ['Q23', 'Q4']:
        settings = Settings(question=question, intervals=8, faceScheme=scheme,
                             shrink=(question=='Q4'), surfaceLatentFraction=1.)
        model = RadialModel(settings)
        with warnings.catch_warnings(record=True) as captured:
            warnings.simplefilter('always')
            result = solve_ivp(model.rhs, (0., 2.), model.initial(), method='BDF',
                               jac=lambda t,y:jacobian(model,t,y), rtol=1e-10,
                               atol=1e-12, max_step=.2)
        msgs = [str(w.message) for w in captured]
        passed = bool(result.success and np.isfinite(result.y).all() and not msgs)
        smokeRecords.append({'question':question, 'face_scheme':scheme, 'interval_s':[0,2],
                             'status':'PASS' if passed else 'FAIL', 'warnings':msgs,
                             'nfev':result.nfev, 'njev':result.njev, 'nlu':result.nlu,
                             'purpose':'Short Jacobian/BDF wiring check, not production accuracy'})

        if not passed:
            failures.append(f"BDF_smoke/{question}/{scheme}")
            currentSourceHash = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
            currentCoreHash =
            hashlib.sha256((ROOT/'paper_output/code/review_delivery/dryingCore.py').read_bytes()).hexdigest()
            if currentSourceHash != LOADED_CODE_SHA256 or currentCoreHash != CORE_LOADED_CODE_SHA256:
                failures.append('Source file changed after module load during validation')
            report = {'schema_version':'1.0', 'created_utc':datetime.now(timezone.utc).isoformat(),
                     'status':'PASS' if not failures else 'FAIL', 'failures':failures,
                     'source_sha256':LOADED_CODE_SHA256,
                     'core_sha256':CORE_LOADED_CODE_SHA256,
                     'source_hash_policy':'SHA-256 frozen when each module is imported; files checked unchanged before
report save.',
                     'runtime':
{'python':sys.version,'executable':sys.executable,'numpy':np.__version__, 'scipy':scipy.__version__,
 'seed':20260910,'relative_component_tolerance':relativeTolerance,
 'absolute_component_tolerance':absoluteTolerance,
 'finite_difference_policy':'Central directional differences, three decreasing steps; every step must
satisfy the mixed absolute/relative component tolerance. Best comparison is supplementary only.',
 'case_state_count':len(records), 'direction_count':4*len(records),
 'checks':records, 'short_BDF_checks':smokeRecords,
 'elapsed_s':time.perf_counter()-started,
 'limitations':['No full production run or grid convergence performed here.',
                 'Tiny denominator/underflow extensions at nonphysical Newton probes are not
calibrated physical data.',
                 'Only this module and the short tests use analytic Jacobian until main agent hooks
core.',
                 'Visual Studio reproduction and human review remain pending.']}
            outputDirectory.mkdir(parents=True, exist_ok=True)
            (outputDirectory/'analytic_jacobian_selftest.json').write_text(
                json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
            brief = {key:report[key] for key in ['status','failures','case_state_count','direction_count','elapsed_s']}
            brief['max_best_component_tolerance_ratio'] = max(
                d['best']['max_component_tolerance_ratio'] for r in records for d in r['directional_checks'])
            brief['max_all_steps_component_tolerance_ratio'] = max(
                step['max_component_tolerance_ratio'] for r in records
                for d in r['directional_checks'] for step in d['all_step_errors'])
            brief['max_mass_balance_derivative_abs'] = max(r['mass_balance_derivative_abs'] for r in records)
            brief['warning_count'] = sum(len(r['warnings']) for r in records+smokeRecords)
            print(json.dumps(brief,ensure_ascii=False,indent=2))

```

```

    return 0 if not failures else 1

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--self-test', action='store_true')
    parser.add_argument('--output-dir', type=Path,
                        default=Path(__file__).resolve().parents[2]/'code/review_delivery/runtime/jacobianValidation')
    arguments = parser.parse_args()
    if not arguments.self_test:
        parser.error('Use --self-test, or import jacobian(model,t,y) from this module.')
    raise SystemExit(selfTest(arguments.output_dir))

`review_delivery/diskDense.py` (稠密输出的磁盘存储与任意时刻插值)
# 本文件为冻结求解源码的驼峰审查副本; 来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Exact BDF dense polynomials backed by a private, rebuildable disk cache.

This changes storage only: each accepted BDF polynomial is written as float64
bytes, then evaluated with SciPy's original BdfDenseOutput implementation.
No additional time/space interpolation and no solver restart are introduced.
"""

from __future__ import annotations
import hashlib
from pathlib import Path
import tempfile
import numpy as np
from scipy.integrate import OdeSolution
from scipy.integrate._ivp.bdf import BDF, BdfDenseOutput
from scipy.integrate._ivp.base import DenseOutput

LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

# 每个 Run 独享项目内可重建缓存; 存原始 float64 字节, 不降低插值阶数。
class DenseCache:
    def __init__(self, root):
        self.parent = (Path(root)/'tmp/cache/solver_runs').resolve()
        if not self.parent.is_relative_to(Path(root).resolve()):
            raise ValueError('Private solver cache must remain in this project')
        self.parent.mkdir(parents=True, exist_ok=True)
        self.directory = Path(tempfile.mkdtemp(prefix='bdf_', dir=self.parent)).resolve()
        try:
            self.handle = (self.directory/'polynomials.bin').open('w+b', buffering=0)
        except BaseException as error:
            try:
                self.directory.rmdir()
            except OSError as cleanup_error:
                error.add_note('Private cache directory cleanup failed: '+repr(cleanup_error))
            raise
        self.bytesWritten = 0
        self.polynomialCount = 0
        self.acceptedArrays = []
        self.closed = False

    def append(self, values):
        if self.closed:
            raise RuntimeError('Dense solution cache is closed')
        array = np.ascontiguousarray(values, dtype=np.float64)
        offset = self.bytesWritten
        self.handle.seek(offset)
        array.tofile(self.handle)
        self.bytesWritten += array.nbytes
        self.polynomialCount += 1
        return offset, array.shape

    def read(self, offset, shape):
        if self.closed:
            raise RuntimeError('Dense solution cache is closed')
        count = int(np.prod(shape))
        self.handle.seek(offset)
        values = np.fromfile(self.handle, dtype=np.float64, count=count)

```

```

        if values.size != count:
            raise IOError('Incomplete BDF coefficient cache')
        return values.reshape(shape)

# 被接受的整段状态写为磁盘映射数组，以限制长时高网格运行的常驻内存。
def storeAccepted(self, values):
    path = self.directory/f'accepted_{len(self.acceptedArrays):03d}.npy'
    mapped = np.lib.format.open_mmap(path, mode='w+', dtype=values.dtype, shape=values.shape)
    # Register before writing so failures can close this Windows mapping.
    self.acceptedArrays.append(mapped)
    mapped[:] = values
    mapped.flush()
    return mapped

# 先释放 Windows 文件映射，再仅清理当前对象创建并验证过的私有缓存。
def close(self):
    if self.closed:
        return
    self.handle.close()
    for array in self.acceptedArrays:
        array._mmap.close()
    self.acceptedArrays.clear()
    # Delete only the verified private cache created by this object.
    if self.directory.parent != self.parent or not self.directory.name.startswith('bdf_'):
        raise RuntimeError('Unexpected private cache path; cleanup refused')
    for path in self.directory.iterdir():
        if not path.is_file() or path.is_symlink():
            raise RuntimeError('Unexpected cache entry; cleanup refused')
        path.unlink()
    self.directory.rmdir()
    self.closed = True

class FileBdfDenseOutput(DenseOutput):
    def __init__(self, original, cache):
        super().__init__(original.t_old, original.t)
        self.order = original.order
        self.t_shift = original.t_shift.copy()
        self.denom = original.denom.copy()
        self.cache = cache
        self.offset, self.shape = cache.append(original.D)

# 保留 SciPy override 名，调用安装版本的原生 BDF 多项式求值器。
def _call_impl(self, t):
    # Reuse the installed SciPy evaluator with the exact recorded D bytes.
    dense = object.__new__(BdfDenseOutput)
    dense.D = self.cache.read(self.offset, self.shape)
    dense.t_shift, dense.denom = self.t_shift, self.denom
    return dense._call_impl(t)

# dense_cache 是 solve_ivp 注入此子类的协议参数，保留拼写以维持接口。
class DiskBDF(BDF):
    def __init__(self, *args, dense_cache, **kwargs):
        self.dense_cache = dense_cache
        super().__init__(*args, **kwargs)

    def _dense_output_impl(self):
        return FileBdfDenseOutput(super()._dense_output_impl(), self.dense_cache)

# 子类不命中 SciPy 对 BDF 的类身份判断，故恢复其接受断点右侧多项式选择。
def alignBdfSegments(result):
    """Restore the original BDF convention at accepted time breakpoints.

    SciPy solve_ivp tests the exact method class for alt_segment. A subclass
    otherwise selects the opposite polynomial at a shared knot. Reconstructing
    OdeSolution changes only this selection, never coefficients or integration.
    """

```

```

    if result.sol is not None:
        result.sol = OdeSolution(result.sol.ts, result.sol.interpolants, alt_segment=True)
    `review_delivery/exportOutputs.py` (按附件 3 模板导出 result1—4.xlsx(域外留空、真实表面独立
列、四位小数))

```

本文件为冻结求解源码的驼峰审查副本; 来源、改名与 AST 核验见 tools/coreRenameReport.json。
 """Stream contest A output directly from a live Run, then stream-read it.

The original contest templates have precedence over workbook design defaults.
 The requested openpyxl write_only method bounds authoring memory. No 60-second
 NPZ is used to fabricate per-second output. Unrounded values on the requested
 physical output grid are archived separately as a compressed CSV; all N solver
 nodes are deliberately not duplicated in this archive.

Usage by the final runner:
 record = export_question(run, 'Q2', output_directory)
 validation = validate_exports([record], runs={'Q2': run})

The archive is an internal reproducibility artifact, not automatically part of
 the size-limited submission package. Nothing is silently truncated or deleted.
 """

```

from __future__ import annotations

```

```

import argparse
import csv
from dataclasses import asdict
from datetime import datetime, timezone
import gzip
import hashlib
import inspect
import json
import math
from pathlib import Path
import re
import sys
import tempfile
import time

```

```

import numpy as np
from openpyxl import Workbook, load_workbook
from openpyxl.cell import WriteOnlyCell
from openpyxl.styles import Alignment, Font, PatternFill
from openpyxl.utils import get_column_letter

```

```

ROOT = Path(__file__).resolve().parents[3]
SOURCE = Path(__file__).resolve()
LOADED_EXPORTER_SHA256 = hashlib.sha256(SOURCE.read_bytes()).hexdigest()
TEMPLATE_DIR = ROOT / 'problem_files/CUMCM2026Problems/A 题/附件/附件 3'
RADII_M = np.arange(21, dtype=float)*0.001
RADII_CM = [i/10 for i in range(21)]
NUMBER_FORMAT = '0.0000'
SIZE_LIMIT_BYTES = 20_000_000 # report also MiB; do not reinterpret an ambiguous M upward

```

```

def sha256File(path):
    h=hashlib.sha256()
    with Path(path).open('rb') as fh:
        for block in iter(lambda:fh.read(1024*1024),b''):
            h.update(block)
    return h.hexdigest()

```

```

def exportFileRecord(path, role=None):
    path=Path(path).resolve()
    result={'path':path.relative_to(ROOT).as_posix(),'absolute_path':str(path),
            'bytes':path.stat().st_size,'sha256':sha256File(path),'exists':True}
    if role:
        result['role']=role
    return result

```

```

def writeJson(path,value):
    path.write_text(json.dumps(value,ensure_ascii=False,indent=2,allow_nan=False)+'\n',encoding='utf-8')

def normalizeQuestionId(questionId):
    value=str(questionId).upper()
    if value in ['1','2','3','4']:
        value='Q'+value
    if value not in ['Q1','Q2','Q3','Q4']:
        raise ValueError('Export requires Q1, Q2, Q3 or Q4')
    return value

def outputTimes(questionId,end,event):
    if questionId=='Q1':
        if end<1800-1e-7:
            raise ValueError('Q1 requires a solved interval covering 0..1800s')
        return np.arange(1801,dtype=float)
    if questionId=='Q2':
        if abs(end-round(end))>1e-7:
            raise ValueError('Q2 needs an integer post-crossing verification endpoint')
        return np.arange(int(round(end))+1,dtype=float)
    if event is None:
        raise ValueError('Q3/Q4 final export requires an actual critical drying event')
    if not 0<=event<=end:
        raise ValueError('Critical event must lie inside the solved interval')
    return np.unique(np.r_[np.arange(math.floor(end/60)+1,dtype=float)*60,event,end])

def roundFourDecimals(x):
    if x is None:
        return None
    if not math.isfinite(float(x)):
        raise ValueError('A non-finite value cannot be exported as a numerical result')
    return float(round(float(x),4))

def loadTemplate(questionId):
    path=TEMPLATE_DIR/f'result{questionId[-1]}.xlsx'
    wb=load_workbook(path,read_only=True,data_only=False)
    try:
        header=next(wb.worksheets[0].iter_rows(values_only=True))
        names=wb.sheetnames
        return {'record':exportFileRecord(path,'original_result_template'),'time_header':header[0],
                'sheet_names':names,'surface_header':header[-1] if questionId=='Q4' else None}
    finally:
        wb.close()

def runProvenance(run):
    module=sys.modules[run.model.__class__.__module__]
    core=Path(inspect.getfile(run.model.__class__)).resolve()
    loaded=getattr(module,'LOADED_CODE_SHA256',None)
    if loaded is not None and loaded!=sha256File(core):
        raise RuntimeError('Core file changed after this Run implementation was imported')
    if sha256File(SOURCE)!=LOADED_EXPORTER_SHA256:
        raise RuntimeError('Exporter changed after import; restart for consistent provenance')
    inputRecords=list(getattr(run.model,'inputRecords',[]))
    for rec in inputRecords:
        if sha256File(ROOT/rec['path'])!=rec['sha256']:
            raise RuntimeError('Observed input changed after solve: '+rec['path'])
    return {'core':exportFileRecord(core,'solver_source'),'exporter':exportFileRecord(SOURCE,'exporter_source'),
            'inputs':inputRecords,'settings':asdict(run.model.settings),
            'event_s':None if run.eventS is None else float(run.eventS),
            'end_s':float(run.endS),'source_kind':'live_Run_dense_solution',
            'per_second_values_are_not_interpolated_from_60s_npz':True}

```

```

# 固定 21 个物理半径逐块查询，Q4 域外保持空白；实际表面单独用 x=1 查询。
def fieldBlock(run,times,qid):
    T,C=run.fields(times,radiiM=RADII_M)
    radius=np.asarray(run.model.radius(times),dtype=float)
    if radius.ndim==0:
        radius=np.full(len(times),radius)
    surfaceC=None
    if qid=='Q4':
        _,surface=run.fields(times,materialX=np.array([1.0]))
        surfaceC=surface[:,0]
        inside=RADII_M[None,:]<=radius[:,None]+1e-12
        # A difference below 1e-12 m at a grid/surface coincidence is roundoff.
        nearBoundary=inside & (~np.isfinite(C))
        if np.any(nearBoundary):
            C=np.where(nearBoundary,surfaceC[:,None],C)
            T=np.where(inside,T,np.nan)
            C=np.where(inside,C,np.nan)
            if not np.all(np.isfinite(C[inside])) or not np.all(np.isfinite(surfaceC)):
                raise ValueError('Non-finite moisture inside Q4 domain')
    else:
        inside=np.ones_like(C,dtype=bool)
        if not np.all(np.isfinite(T)) or not np.all(np.isfinite(C)):
            raise ValueError('Non-finite output inside fixed domain')
    return T-273.15,C,radius,surfaceC,inside

def rawHeaders(qid):
    h=['time_s','radius_m']
    if qid in ['Q1','Q2']:
        h += [f'T_C_r_{i/10:.1f}_cm' for i in range(21)]
    h += [f'C_r_{i/10:.1f}_cm' for i in range(21)]
    if qid=='Q4':
        h += ['C_surface']
    return h

def rawRow(qid,t,T,C,R,surface,inside):
    row=[float(t),float(R)]
    if qid in ['Q1','Q2']:
        row+=list(map(float,T))
    row += [float(c) if valid else None for c,valid in zip(C,inside)]
    if qid=='Q4':
        row.append(float(surface))
    return row

def roundedSheets(qid,raw):
    t,R=raw[:2]
    if qid in ['Q1','Q2']:
        return {'温度':[roundFourDecimals(t)]+[roundFourDecimals(v) for v in raw[2:23]],
                '水分浓度':[roundFourDecimals(t)]+[roundFourDecimals(v) for v in raw[23:44]]}
    if qid=='Q3':
        return {'Sheet1':[roundFourDecimals(t)]+[roundFourDecimals(v) for v in raw[2:23]]}
    return {'Sheet1':[roundFourDecimals(t)]+[roundFourDecimals(v) for v in raw[2:24]],
            '半径':[roundFourDecimals(t),roundFourDecimals(R*100)]}

# 按官方表头设置工作簿；write_only 流式输出降低大题表的内存峰值。
def createSheet(wb,name,header):
    ws=wb.create_sheet(name)
    ws.freeze_panes='B2'
    ws.sheet_view.showGridLines=False
    ws.column_dimensions['A'].width=30
    for j in range(2,len(header)+1):
        ws.column_dimensions[get_column_letter(j)].width=13 if j<len(header) else 16
    ws.row_dimensions[1].height=34
    cells=[]
    for value in header:
        cell=WriteOnlyCell(ws,value=value)

```

```

        cell.font=Font(name='Arial',size=10,bold=True,color='FFFFFF')
        cell.fill=PatternFill(fill_type='solid',fgColor='334155')
        cell.alignment=Alignment(horizontal='center',vertical='center',wrap_text=True)
        if isinstance(value,(int,float)):
            cell.number_format='0.0'
        cells.append(cell)
# write_only's first append opens a temporary worksheet XML file. Keep
# those rebuildable files in this contest workspace instead of OS Temp.
cache=ROOT/'tmp/cache/openpyxl_exports'
cache.mkdir(parents=True,exist_ok=True)
previousTempdir=tempfile.tempdir
try:
    tempdir=tempfile.tempdir=str(cache)
    ws.append(cells)
finally:
    tempdir=tempfile.tempdir=previousTempdir
return ws

def appendNumeric(ws,row):
    cells=[]
    for value in row:
        if value is None:
            cells.append(None)
        else:
            cell=WriteOnlyCell(ws,value=value)
            cell.number_format=NUMBER_FORMAT
            cells.append(cell)
    ws.append(cells)

# 正文表独立生成,单位和显示位数跟随题目模板,原精度数值另存。
def paperTables(run,qid,directory):
    if qid=='Q1':
        ts=np.array([100,300,600,900,1200,1500,1800],dtype=float)
    elif qid=='Q2':
        ts=np.arange(1,7,dtype=float)*1800
        if run.endS<10800-1e-7:
            raise ValueError('Q2 paper tables require 3h of actual solution')
    else:
        from q3Model import completion
        reportedEnd=completion(run)['reported_time_s']
        ts=np.unique(np.r_[np.arange(21600.,reportedEnd+1e-8,21600.),reportedEnd])
    rs=np.array([0.,.005,.01,.015,.02])
    T,C=run.fields(ts,radiiM=rs)
    T=T-273.15
    R=np.asarray(run.model.radius(ts))
    headers=['时间/s' if qid=='Q1' else '时间/h']+[0,0.5,1,1.5,2]
    if qid=='Q4':
        headers += ['药材表面']
        _,Cs=run.fields(ts,materialX=[1.])
    outputs=[]
    fields=[('temperature',T),('moisture',C)] if qid in ['Q1','Q2'] else [('moisture',C)]
    for kind,values in fields:
        path=directory/f'{qid.lower()}_paper_{kind}.csv'
        with path.open('w',encoding='utf-8-sig',newline='') as fh:
            w=csv.writer(fh);w.writerow(headers)
            for i,t in enumerate(ts):
                row=[f'{t if qid=="Q1" else t/3600:.4f}']
                for j,r in enumerate(rs):
                    outside=qid=='Q4' and r>R[i]+1e-12
                    value=values[i,j]
                    if not outside and not np.isfinite(value) and qid=='Q4' and abs(r-R[i])<=1e-12:
                        value=Cs[i,0]
                    row.append(' ' if outside else f'{float(value):.4f}')
                if qid=='Q4':
                    row += [f'{Cs[i,0]:.4f}']
            w.writerow(row)
        outputs.append(exportFileRecord(path,'paper_table'))
    if qid=='Q4':
        path=directory/'q4_paper_radius.csv'
        with path.open('w',encoding='utf-8-sig',newline='') as fh:

```

```

        w=csv.writer(fh);w.writerow(['时间/h','表面半径/cm'])
        w.writerows([[f'{t/3600:.4f}',f'{r*100:.4f}'] for t,r in zip(ts,R)])
    outputs.append(exportFileRecord(path,'paper_surface_coordinates'))
return outputs

```

```
def validatePaperTables(manifest,run):
```

```
    """Re-read every paper CSV cell and query the live solution independently.
```

```

    Do not call _paper_tables or reuse its generated values. The question's
    schedule is reconstructed here. Physical points come from validated CSV
    headers; the final query uses completion's actual seconds, never the
    rounded hour label parsed back from the CSV.
    """

```

```

    qid=manifest['question_id']
    if run is None:
        return {'status':'NOT_REQUESTED','checked_cells':0,'files':[],
                'scope':'Paper CSV hashes only; no independent live Run available.'},[]
    errors=[];records=manifest['paper_tables'];completionInfo=None
    if qid=='Q1':
        times=np.asarray([100.,300.,600.,900.,1200.,1500.,1800.])
        expectedKinds=['temperature','moisture']
    elif qid=='Q2':
        times=1800.*np.arange(1,7)
        expectedKinds=['temperature','moisture']
    else:
        from q3Model import completion
        completionInfo=completion(run)
        finalS=float(completionInfo['reported_time_s'])
        # Integer multiples strictly before the report, then the report itself.
        # Construct afresh rather than reading generation-time arrays/labels.
        regular=[21600.*k for k in range(1,math.floor(finalS/21600.)+1)
                 if 21600.*k<finalS-1e-8]
        times=np.asarray([*regular,finalS],dtype=float)
        expectedKinds=['moisture']+(['radius'] if qid=='Q4' else [])
        stored=manifest.get('paper_table_time_contract',{})
        for key in ['reported_time_s','reported_drying_time_h']:
            if key not in stored or not numericEqual(stored[key],completionInfo[key]):
                errors.append('Paper CSV report-time metadata differs from independent completion: '+key)
        if not numericEqual(stored.get('reported_time_h'),completionInfo['reported_drying_time_h']):
            errors.append('Paper CSV reported_time_h alias differs from independent completion')
    if times[-1]>run.ends+1e-7:
        errors.append('Paper CSV contract exceeds the live solution horizon')
        return {'status':'FAIL','checked_cells':0,'files':[],errors}
    expectedNames={f'{qid.lower()}_paper_{kind}.csv':kind for kind in expectedKinds}
    actualNames=[Path(r['path']).name for r in records]
    if set(actualNames)!=set(expectedNames) or len(actualNames)!=len(expectedNames):
        errors.append('Missing, duplicate, or unexpected paper CSV artifact')
    files=[];totalCells=0
    for rec in records:
        path=ROOT/rec['path'];kind=expectedNames.get(path.name)
        if kind is None:continue
        beforeErrors=len(errors)
        with path.open('r',encoding='utf-8-sig',newline='') as fh:
            reader=csv.reader(fh);header=next(reader,None);rows=list(reader)
            desiredTime='时间/s' if qid=='Q1' else '时间/h'
            if kind=='radius':
                correctHeader=['时间/h','表面半径/cm']
                if header!=correctHeader:
                    errors.append(path.name+': radius/time units or header mismatch')
                    continue
                radii=None
                expected=np.asarray(run.model.radius(times),dtype=float).reshape(-1,1)*100.
                inside=np.ones_like(expected,dtype=bool)
                unit='cm';surfaceColumn=None
            else:
                wantedColumns=7 if qid=='Q4' else 6
                if header is None or len(header)!=wantedColumns or header[0]!=desiredTime:
                    errors.append(path.name+': time/column header mismatch');continue
                try:
                    radii=np.asarray([float(v)*.01 for v in header[1:6]])
                except ValueError:

```

```

        errors.append(path.name+': nonnumeric physical-radius header');continue
    if not np.all(np.isfinite(radii)) or not np.allclose(radii,[0.,.005,.01,.015,.02],rtol=0,atol=1e-12):
        errors.append(path.name+': physical-radius grid must be 0,0.5,1,1.5,2 cm');continue
    if qid=='Q4' and header[-1]!='药材表面':
        errors.append(path.name+': distinct material surface column missing');continue
    temperature,moisture=run.fields(times,radiiM=radii)
    expected=np.asarray(temperature)-273.15 if kind=='temperature' else np.asarray(moisture)
    inside=np.ones_like(expected,dtype=bool);surfaceColumn=None
    if qid=='Q4':
        physicalR=np.asarray(run.model.radius(times),dtype=float)
        inside=radii[None,:]<=physicalR[:,None]+1e-12
        _,surfaceValues=run.fields(times,materialX=np.asarray([1.]))
        # Only roundoff at a coincident physical/surface coordinate
        # may use the separately queried material surface value.
        coincidence=np.abs(radii[None,:]-physicalR[:,None])<=1e-12
        expected=np.where(coincidence & ~np.isfinite(expected),surfaceValues,expected)
        expected=np.column_stack([expected,surfaceValues[:,0]])
        inside=np.column_stack([inside,np.ones(len(times),dtype=bool)])
        surfaceColumn=7
    unit='degC' if kind=='temperature' else 'kg/kg'
    if len(rows)!=len(times):
        errors.append(f'{path.name}: expected {len(times)} data rows, read {len(rows)}')
    checked=0;blanks=0;maxFieldError=0.;maxTimeError=0.
    for i,(row,t) in enumerate(zip(rows,times)):
        if len(row)!=len(header):
            errors.append(f'{path.name}: column count mismatch at row {i+2}');continue
        timeValue=float(t if qid=='Q1' else t/3600.)
        if row[0]!=format(timeValue,'.4f'):
            errors.append(f'{path.name}: time label mismatch at row {i+2}; query time is {t:.17g}s')
        else:maxTimeError=max(maxTimeError,abs(float(row[0])-timeValue))
        checked+=1
        for j,cell in enumerate(row[1:]):
            checked+=1
            if not inside[i,j]:
                blanks+=1
                if cell!='':errors.append(f'{path.name}: domain outside must be blank at row {i+2}, column
{j+2}')
                continue
            value=float(expected[i,j])
            if not math.isfinite(value):
                errors.append(f'{path.name}: live value is not finite inside material');continue
            if not re.fullmatch(r'-?\d+\.\d{4}',cell):
                errors.append(f'{path.name}: finite four-decimal number required at row {i+2}, column {j+2}')
                continue
            actual=float(cell)
            if not math.isfinite(actual) or abs(actual-float(format(value,'.4f')))>1e-10:
                errors.append(f'{path.name}: independently queried value mismatch at row {i+2}, column {j+2}')
            maxFieldError=max(maxFieldError,abs(actual-value))
        totalCells+=checked
    files.append({'artifact':exportFileRecord(path),'kind':kind,'status':'PASS' if len(errors)==beforeErrors
else 'FAIL',
    'header':header,'data_rows':len(rows),'expected_data_rows':len(times),'checked_cells':checked,
    'source_query_times_s':times.tolist(),'display_time_unit':'s' if qid=='Q1' else 'h',
    'max_display_time_rounding_error':maxTimeError,'field_unit':unit,
    'max_field_rounding_error_from_live_value':maxFieldError,'outside_domain_blank_cells':blanks,
    'physical_radii_m':None if radii is None else radii.tolist(),
    'surface_column_1based':surfaceColumn,'surface_query':'material_x=1; never fixed 2 cm' if
surfaceColumn else None})
    return {'status':'FAIL' if errors else 'PASS','checked_cells':totalCells,'files':files,
    'completion':completionInfo,'completion_source':exportFileRecord(SOURCE.with_name('q3Model.py')) if
completionInfo else None,
    'scope':'Every paper CSV cell independently re-evaluated from live Run and fixed question contracts; no
generation arrays or 60s archives reused.',
    'rounding_limit':'CSV stores four decimals; matching rounded values cannot distinguish sub-rounding
perturbations of the original source value.'},errors

```

逐秒数据直接求值于 live Run 的 BDF 密集解; 不从 60 s NPZ 再插值生成。

```

def exportQuestion(run,questionId,outputDir,*,chunkRows=1000,overwrite=False):
    """Export a complete question and return records plus exact validation rules.

```

A fresh output directory/version is preferred. Existing files are rejected

```

unless overwrite=True is explicitly passed. This never edits raw templates.
"""
started=time.perf_counter()
qid=normalizeQuestionId(questionId)
directory=Path(outputDir).resolve()
if not directory.is_relative_to(ROOT) or directory.is_relative_to(ROOT/'problem_files'):
    raise ValueError('Outputs must be inside the competition workspace, outside problem_files')
if not 1<=int(chunkRows)<=2000:
    raise ValueError('chunk_rows must be between 1 and 2000')
directory.mkdir(parents=True,exist_ok=True)
stem='result'+qid[-1]
bookPath=directory/(stem+'.xlsx')
rawPath=directory/(stem+'_unrounded.csv.gz')
manifestPath=directory/(stem+'.export.json')
if not overwrite and any(p.exists() for p in [bookPath,rawPath,manifestPath]):
    raise FileExistsError('Export target already exists; use a new output version')
provenance=runProvenance(run)
allowed={'Q1':{'Q1'},'Q2':{'Q2','Q23'},'Q3':{'Q2','Q3','Q23'},'Q4':{'Q4'}}
if run.model.settings.question not in allowed[qid]:
    raise ValueError('Run physics/question do not match the requested output question')
if qid=='Q4' and not run.model.settings.shrink:
    raise ValueError('Formal Q4 output requires the shrinking-domain Run, not its fixed-radius control')
template=loadTemplate(qid)
times=outputTimes(qid,run.endS,run.events)
if len(times)+1>1_048_576:
    raise ValueError('Required full time grid exceeds the XLSX worksheet row limit; do not truncate')
workbook=Workbook(write_only=True)
workbook.properties.creator=''
workbook.properties.lastModifiedBy=''
workbook.properties.title=f'问题{qid[-1]}结果'
headers={name:[template['time_header']] + RADII_CM for name in template['sheet_names']}
if qid=='Q4':
    headers['Sheet1'].append(template['surface_header'])
    headers['半径']=['时间/s','药材表面半径/cm']
sheets={name:createSheet(workbook,name,header) for name,header in headers.items()}
counts={name:{'data_rows':0,'total_rows':1,'columns':len(header),'outside_domain_blank_cells':0,
    'first_time_s':None,'last_time_s':None,'header':header} for name,header in headers.items()}
samplesIdx=set(np.linspace(0,len(times)-1,min(13,len(times)),dtype=int).tolist())
samplesIdx.update([0,min(1,len(times)-1),len(times)-1])
if run.eventS is not None:
    samplesIdx.update(np.flatnonzero(times==run.events).tolist())
samples=[]
with gzip.open(rawPath,'wt',encoding='utf-8',newline='',compresslevel=6) as fh:
    rawWriter=csv.writer(fh);rawWriter.writerow(rawHeaders(qid))
    for start in range(0,len(times),int(chunkRows)):
        ts=times[start:start+int(chunkRows)]
        T,C,R,Cs,inside=fieldBlock(run,ts,qid)
        for i,t in enumerate(ts):
            raw=rawRow(qid,t,T[i],C[i],R[i],None if Cs is None else Cs[i],inside[i])
            rawWriter.writerow(['' if v is None else format(v,'.17g') for v in raw])
            values=roundedSheets(qid,raw)
            for name,row in values.items():
                appendNumeric(sheets[name],row)
                count=counts[name]
                count['data_rows']+=1;count['total_rows']+=1
                count['outside_domain_blank_cells']+=sum(v is None for v in row)
                if count['first_time_s'] is None:
                    count['first_time_s']=row[0]
                count['last_time_s']=row[0]
            if start+i in samplesIdx:
                samples.append({'data_row_index':start+i,'source_time_s':float(t),
                    'raw_values':raw,'rounded_sheets':values})
workbook.save(bookPath)
paperRecords=paperTables(run,qid,directory)
paperTimeContract={'unit':'s' if qid=='Q1' else 'h','stored_decimals':4,
    'query_uses_unrounded_seconds':True}
if qid in ['Q3','Q4']:
    from q3Model import completion
    completed=completion(run)
    paperTimeContract.update({key:completed[key] for key in
        ['reported_time_s','reported_drying_time_h','critical_event_s',
        'max_C_at_reported_time','conservative_post_verification_s']})
    paperTimeContract['reported_time_h']=completed['reported_drying_time_h']

```

```

        paperTimeContract['regular_step_s']=21600.
        paperTimeContract['endpoint_convention']='6-hour samples plus upward-reported strict-drying time; distinct
from workbook post-verification endpoint'
        endProvenance=runProvenance(run)
        if endProvenance!=provenance or sha256File(ROOT/template['record']['path'])!=template['record']['sha256']:
            raise RuntimeError('Source, settings or template changed during export')
        bookRecord=exportFileRecord(bookPath,'contest_result_workbook')
        archiveRecord=exportFileRecord(rawPath,'internal_unrounded_output_grid_archive')
        manifest={
            'schema_version':'1.0','question_id':qid,'generated_by':'paper_output/code/review_delivery/
exportOutputs.py',
            'generated_at':datetime.now(timezone.utc).isoformat(),'status':'EXPORTED_PENDING_STREAM_READBACK',
            'workbook':bookRecord,'unrounded_archive':archiveRecord,'paper_tables':paperRecords,
            'paper_table_time_contract':paperTimeContract,
            'provenance':provenance,'template':template['record'],'sheets':counts,
            'time_grid':{'start_s':0,'end_s':float(times[-1]),'regular_step_s':1 if qid in ['Q1','Q2'] else 60,
                'critical_event_s':provenance['event_s'] if qid in ['Q3','Q4'] else None,
                'include_post_verification_endpoint':qid!='Q1',
            }
            'count':len(times),'rounding_duplicate_time_count':int(np.sum(np.diff(np.round(times,4))==0)),
            'radius_grid':{'fixed_radii_cm':RADII_CM,'surface_column':qid=='Q4','surface_coordinate_sheet':'半径' if
qid=='Q4' else None},
            'rounding':{'decimal_places':4,'storage':'numeric rounded to four decimals','number_format':NUMBER_FORMAT,
                'raw_archive':'17 significant digits, source times in seconds and radius in metres',
                'threshold_judgement':'use original full precision event and max C, never rounded 0.1500'},
            'domain_rule':{'outside_values':'blank (None); never zero','comparison_tolerance_m':1e-12,
                'Q4_surface_is_not_fixed_2cm':True},
            'samples':samples,'elapsed_s':time.perf_counter()-started,
            'size':{'workbook_bytes':bookRecord['bytes'],'workbook_MiB':bookRecord['bytes']/2**20,
                'unrounded_archive_bytes':archiveRecord['bytes'],'warning_limit_bytes':SIZE_LIMIT_BYTES,
                'workbook_exceeds_20_decimal_MB':bookRecord['bytes']>SIZE_LIMIT_BYTES,
                'archive_is_internal_not_automatically_in_submission_zip':True,
                'complete_support_archive_still_requires_actual_size_check':True,
                'no_required_time_or_space_values_removed':True},
            'validation_required':'validate_exports([record], runs={question_id: run}); full workbook/archive
comparison, live Run workbook samples, and every paper CSV cell independently queried from live Run',
            'visual_studio_gui':'pending','human_review':'pending',
        }
        writeJson(manifestPath,manifest)
        return {'question_id':qid,'workbook_path':str(bookPath),'manifest_path':str(manifestPath),
            'artifacts':[bookRecord,archiveRecord]+paperRecords+
[exportFileRecord(manifestPath,'export_manifest')],
            'sheets':counts,'size':manifest['size'],'validation_status':'pending'}

def resolveManifestPath(item):
    if isinstance(item,dict):
        return Path(item['manifest_path'])
    path=Path(item)
    return path.with_suffix('.export.json') if path.suffix=='.xlsx' else path

def numericEqual(a,b,tolerance=1e-10):
    return a is None and b is None or (a is not None and b is not None and
        isinstance(a,(int,float)) and not isinstance(a,bool) and math.isfinite(float(a)) and abs(float(a)-
float(b))<=tolerance)

# 回读 XLSX/原精度压缩 CSV 并与同一个 live Run 独立查询比对。
def validateExports(paths,runs=None):
    """Stream every saved cell against the exact schedule and raw float archive.

    Pass runs={'Q1': run1, ...} to re-evaluate workbook sample points and every
    paper CSV cell from question-specific contracts and the live Run. With no
    live Run this reports static/archive consistency only, never full verification.
    Writes a sibling .validation.json for each export. Does not alter the XLSX.
    """
    if isinstance(paths,(str,Path,dict)):
        paths=[paths]
    runs={} if runs is None else runs
    results=[]

```

```

for item in paths:
    started=time.perf_counter()
    manifestPath=resolveManifestPath(item).resolve()
    manifest=json.loads(manifestPath.read_text(encoding='utf-8'))
    qid=manifest['question_id'];errors=[];checkedCells=0;checkedSheets={}
    for rec in [manifest['workbook'],manifest['unrounded_archive'],manifest['template']]
+manifest['paper_tables']:
        if not (ROOT/rec['path']).exists() or sha256File(ROOT/rec['path'])!=rec['sha256']:
            errors.append('Hash mismatch or missing artifact: '+rec['path'])
    if errors:
        raise RuntimeError('; '.join(errors))
    times=outputTimes(qid,manifest['provenance']['end_s'],manifest['provenance']['event_s'])
    wb=load_workbook(ROOT/manifest['workbook']['path'],read_only=True,data_only=False)
    if wb.sheetnames!=list(manifest['sheets']):
        errors.append('Sheet names/order differ from export contract')
    try:
        for name,spec in manifest['sheets'].items():
            ws=wb[name]
            rows=ws.iter_rows(min_row=1,max_col=spec['columns'])
            header=[cell.value for cell in next(rows)]
            if header!=spec['header']:
                errors.append(name+' : header/physical radius grid mismatch')
            count=0;blanks=0;first=None;last=None;maxRoundingError=0.
            with gzip.open(ROOT/manifest['unrounded_archive']['path'],'rt',encoding='utf-8',newline='') as
rawfh:
                sourceRows=csv.reader(rawfh)
                if next(sourceRows)!=rawHeaders(qid):
                    errors.append('Unrounded archive schema mismatch')
                for index,sourceRow in enumerate(sourceRows):
                    try:
                        cells=next(rows)
                    except StopIteration:
                        errors.append(name+' : workbook ends before raw archive');break
                    raw=[None if v=='' else float(v) for v in sourceRow]
                    if index>len(times) or abs(raw[0]-times[index])>1e-8:
                        errors.append(name+' : source time schedule mismatch at '+str(index))
                    if qid=='Q4':
                        for j,r in enumerate(RADII_M):
                            outside=r>raw[1]+1e-12
                            if (raw[2+j] is None)!=outside:
                                errors.append(name+' : incorrect moving-domain mask at '+str(index))
                    elif any(v is None for v in raw):
                        errors.append(name+' : unexpected blank in fixed domain')
                    expected=roundedSheets(qid,raw)[name]
                    for j,(cell,value) in enumerate(zip(cells,expected)):
                        if not numericEqual(cell.value,value):
                            errors.append(f'{name}: value mismatch at row {index+2}, column {j+1}')
                        if value is not None:
                            if cell.number_format!=NUMBER_FORMAT:
                                errors.append(f'{name}: four-decimal format missing at row {index+2}, column
{j+1}')
                            if not math.isfinite(float(cell.value)) or abs(float(cell.value)*1e4-
round(float(cell.value)*1e4))>1e-5:
                                errors.append(f'{name}: non-finite or non-four-decimal stored number')
                                maxRoundingError=max(maxRoundingError,abs(float(cell.value)-value))
                            else:
                                blanks+=1
                                checkedCells+=len(cells);count+=1
                                if first is None:first=cells[0].value
                                last=cells[0].value
                                if len(errors)>30:
                                    raise RuntimeError('Export validation failed: '+'; '.join(errors[:30]))
                                if next(rows,None) is not None:
                                    errors.append(name+' : workbook has uncontracted extra rows')
                    if count!=spec['data_rows'] or count!=len(times):
                        errors.append(name+' : row count differs from time grid')
                    if blanks!=spec['outside_domain_blank_cells']:
                        errors.append(name+' : blank count mismatch')
                    checkedSheets[name]={ 'data_rows':count,'total_rows':count+1,'first_time_s':first,
'last_time_s':last,'outside_domain_blank_cells':blanks,'value_comparison_max_abs_error':maxRoundingError}
            finally:
                wb.close()

```

```

run=runs.get(qid)
live={'status':'NOT_REQUESTED','samples':0,'max_abs_unrounded_difference':None}
if run is not None:
    current=runProvenance(run)
    if current!=manifest['provenance']:
        errors.append('Live Run provenance differs from the exported Run')
    samples=manifest['samples'];ts=np.array([s['source_time_s'] for s in samples])
    T,C,R,Cs,inside=fieldBlock(run,ts,qid)
    difference=0.
    for i,sample in enumerate(samples):
        actual=rawRow(qid,ts[i],T[i],C[i],R[i],None if Cs is None else Cs[i],inside[i])
        for a,b in zip(actual,sample['raw_values']):
            if a is None or b is None:
                if not (a is None and b is None):errors.append('Live Run sample domain mismatch')
            else:
                difference=max(difference,abs(a-b))
    if difference>1e-10:
        errors.append('Independent live Run sample mismatch exceeds 1e-10')
    live={'status':'PASS' if difference<=1e-10 else 'FAIL','samples':len(samples),
        'max_abs_unrounded_difference':difference}
    paperValidation,paperErrors=validatePaperTables(manifest,run)
    errors.extend(paperErrors)
    report={'schema_version':'1.0','question_id':qid,'generated_by':'paper_output/code/review_delivery/
exportOutputs.py',
        'generated_at':datetime.now(timezone.utc).isoformat(),
        'status':'FAIL' if errors else 'PASS','fully_verified_with_live_Run':not errors and run is not
None,
        'checked_cells':checkedCells,'sheets':checkedSheets,'live_Run_sample_validation':live,
        'paper_CSV_live_Run_validation':paperValidation,
        'errors':errors,'elapsed_s':time.perf_counter()-started,
        'workbook':manifest['workbook'],'export_manifest':exportFileRecord(manifestPath),
        'scope':'Full workbook/raw-archive validation; independent workbook live Run samples and every
paper CSV time/position/surface/radius cell when live Run is provided.',
        'four_decimals_are_storage_and_display_not_physical_accuracy_claim':True,
        'visual_render':'pending','visual_studio_gui':'pending','human_review':'pending'}
    reportPath=manifestPath.with_name(manifestPath.name.replace('.export.json','.validation.json'))
    writeJson(reportPath,report);report['report_path']=str(reportPath);results.append(report)
    if errors:
        raise RuntimeError('Export validation failed: '+'; '.join(errors[:30]))
    return {'status':'PASS' if all(r['status']=='PASS' for r in results) else 'FAIL',
        'fully_verified_with_live_Run':all(r['fully_verified_with_live_Run'] for r in
results),'exports':results}

def paperCsvSelfcheck(directory):
    """Small actual solves plus deliberate CSV defects, never final results."""
    from dryingCore import Settings,solveCase
    import copy
    directory=Path(directory).resolve()
    directory.mkdir(parents=True,exist_ok=True)
    summaries=[];negativeChecks=[]
    for qid in ['Q1','Q2','Q3','Q4']:
        settings={'question':Q23' if qid in ['Q2','Q3'] else qid,
            'intervals':40,'faceScheme':'kirchhoff','shrink':qid=='Q4',
            'rtol':1e-8,'atolTemperature':1e-8,'atolMoisture':1e-10,
            'earlyMaxSteps':5.}
        if qid=='Q2':
            settings.update(constantD=2e-8,beta=8e-6,horizonH=24.)
        print('PAPER_CSV_SELFHECK '+qid+' actual N40 solve',flush=True)
        run=solveCase(Settings(**settings))
        try:
            if qid=='Q2' and (run.eventS is None or run.endS<10800.):
                raise RuntimeError('Accelerated export test must still cover 3h and a real drying event')
            exported=exportQuestion(run,qid,directory/qid)
            checked=validateExports([exported],runs={qid:run})
            manifest=json.loads(Path(exported['manifest_path']).read_text(encoding='utf-8'))
            defects={'Q1':['temperature_value'],'Q2':['moisture_value'],
                'Q3':['last_time_label'],'Q4':['outside_nonblank','surface_missing','radius_value']}[qid]
            for defect in defects:
                changed=copy.deepcopy(manifest)
                kind='temperature' if defect=='temperature_value' else 'radius' if defect=='radius_value' else
'moisture'

```

```

        selected=next(r for r in changed['paper_tables'] if Path(r['path']).name==f'{qid.lower()}
_paper_{kind}.csv')
        originalPath=ROOT/selected['path']
        with originalPath.open('r',encoding='utf-8-sig',newline='') as fh:rows=list(csv.reader(fh))
        if defect in ['temperature_value','moisture_value']:
            rows[1][1]=format(float(rows[1][1])+.01, '.4f')
        elif defect=='last_time_label':
            rows[-1][0]=format(float(rows[-1][0])-.001, '.4f')
        elif defect=='outside_nonblank':
            ri,ci=next((i,j) for i in range(1,len(rows)) for j in range(1,6) if rows[i][j]=='')
            rows[ri][ci]='0.0000'
        elif defect=='surface_missing':rows[-1][-1]=''
        elif defect=='radius_value':rows[-1][1]=format(float(rows[-1][1])+.01, '.4f')
        target=directory/'deliberate_defects'/defect/originalPath.name
        target.parent.mkdir(parents=True,exist_ok=True)
        with target.open('w',encoding='utf-8-sig',newline='') as fh:csv.writer(fh).writerows(rows)
        selected.update(exportFileRecord(target,selected.get('role')))
        result,errors=validatePaperTables(changed,run)
        if not errors or result['status']!='FAIL':
            raise AssertionError('Independent paper validator missed deliberate defect: '+defect)
        negativeChecks.append({'question_id':qid,'defect':defect,'correctly_rejected':True,
            'errors':errors,'artifact':exportFileRecord(target)})
        summaries.append({'question_id':qid,'settings':asdict(run.model.settings),'export':exported,
            'validation':checked,'event_s':run.eventS,'end_s':run.endS,
            'Q2_is_accelerated_constant_D_export_test_only':qid=='Q2'})
        print(json.dumps({'question_id':qid,'status':'PASS','end_s':run.endS,
            'paper_cells':checked['exports'][0]['paper_CSV_live_Run_validation']
            ['checked_cells']},ensure_ascii=False),flush=True)
        finally:
            run.close()
        report={'status':'PASS','generated_at':datetime.now(timezone.utc).isoformat(),
            'scope':'N40 exporter fidelity only. Q2 uses constant_D=2e-8 and beta=8e-6 solely to shorten the real
            full-event export test; these are not formal physical results. Q1/Q3/Q4 use their nominal physics.',
            'source':exportFileRecord(SOURCE),'exports':summaries,'negative_checks':negativeChecks,
            'negative_test_scope':'Copied paper CSVs only; six deliberate defects must be rejected even after
            their recorded hashes are updated.',
            'visual_studio_gui':'pending','human_review':'pending'}
        writeJson(directory/'paper_csv_selfcheck_report.json',report)
        print(json.dumps({'status':'PASS','questions':4,'deliberate_defects_rejected':len(negativeChecks),
            'report':str(directory/'paper_csv_selfcheck_report.json')},ensure_ascii=False),flush=True)
        return 0

def main():
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--selfcheck',action='store_true')
    parser.add_argument('--paper-selfcheck',action='store_true')
    parser.add_argument('--output-dir',default='paper_output/code/review_delivery/runtime/exportSelfcheck')
    args=parser.parse_args()
    if not args.selfcheck and not args.paper_selfcheck:
        parser.error('Use the Python API with final Runs, or --selfcheck for a real Q1 validation export')
    if Path.cwd().resolve()!=ROOT:
        raise RuntimeError('Use the competition workspace as cwd')
    if args.paper_selfcheck:
        return paperCsvSelfcheck(ROOT/args.output_dir)
    from dryingCore import Settings,solveCase
    # Small spatial grid is intentional: this validates exporter fidelity, not
    # four-decimal physical accuracy, and is never a final result1 replacement.
    run=solveCase(Settings(question='Q1',intervals=40,faceScheme='kirchhoff',
        rtol=1e-8,atolTemperature=1e-8,atolMoisture=1e-10,earlyMaxStepS=5.))
    record=exportQuestion(run,'Q1',ROOT/args.output_dir)
    checks=validateExports([record],runs={'Q1':run})
    output=Path(args.output_dir).resolve()
    note={'status':checks['status'],'scope':'Actual Q1 export selfcheck only, not final production or numerical-
    accuracy acceptance',
        'settings':asdict(run.model.settings),'export':record,'validation':checks}
    writeJson(output/'selfcheck_summary.json',note)

    print(json.dumps({'status':checks['status'],'fully_verified_with_live_Run':checks['fully_verified_with_live_Run'],
        'output':str(output),'size':record['size'],'sheets':record['sheets']},ensure_ascii=False,indent=2))
    return 0

```

```

if __name__ == '__main__':
    raise SystemExit(main())

`review_delivery/q1Model.py` (问题一模型装配入口)
# 本文件为冻结求解源码的驼峰审查副本; 来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Q1 uses Appendix 2 throughout its 1800-second interval."""
from dryingCore import Settings, solveCase

# Q1 全部 1800 秒均采用附录 2 参数, 正式空间区间数默认 3200。
def solve(intervals=3200):
    return solveCase(Settings(question='Q1', intervals=intervals, faceScheme='kirchhoff',
                              rtol=1e-10, atolTemperature=1e-10, atolMoisture=1e-12,
                              earlyMaxStepS=2., maxStepS=120.))

`review_delivery/q2Model.py` (问题二模型装配入口)
# 本文件为冻结求解源码的驼峰审查副本; 来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Q2 uses Appendix 3 from t=0; it does not splice the Q1 trajectory."""
from dryingCore import Settings, solveCase

# Q2/Q3 从 t=0 采用附录 3, Q3 不能拼接 Q1。
def solve(intervals=3200):
    return solveCase(Settings(question='Q23', intervals=intervals, faceScheme='kirchhoff',
                              rtol=1e-10, atolTemperature=1e-10, atolMoisture=1e-12,
                              earlyMaxStepS=2., maxStepS=120.))

`review_delivery/q3Model.py` (问题三模型装配入口与达标判据)
# 本文件为冻结求解源码的驼峰审查副本; 来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Q3 is a threshold functional of the exact same Run used by Q2."""
import math
import numpy as np

# Q3 是 Q2 场解的阈值泛函; 先连续定位, 再向上取 0.0001 h 并验原精度 max(C)<0.15。
def completion(run):
    if run.eventS is None:
        raise ValueError('No full-domain drying event within the solved horizon')
    # At the continuous root the maximum equals 0.15. Report upward on the
    # required 0.0001-hour grid and verify the unrounded state there.
    count = math.ceil(run.eventS / 3600. * 10000.)
    # 即使四位显示为 0.1500, 也只能依据未舍入含水率判定严格干燥。
    while True:
        reportH = count/10000.
        t = reportH*3600.
        if t > run.endS:
            raise RuntimeError('Four-decimal reporting time is outside the verified trajectory')
        values = run.state([t])[1:-1:2,0]
        if float(np.max(values)) < .15:
            break
        count += 1
    return {'critical_event_s':run.eventS,'critical_event_h':run.eventS/3600.,
            'reported_drying_time_h':reportH,'reported_time_s':t,
            'max_C_at_reported_time':float(np.max(values)),
            'slowest_material_coordinate':float(run.model.x[np.argmax(values)]),
            'conservative_post_verification_s':run.endS,
            'rounding_convention':'upward on 0.0001 h grid, followed by actual strict threshold check',
            'interpretation':'A conditional numerical event, not a confidence bound on physical drying time.'}

`review_delivery/q4Model.py` (问题四模型装配入口)
# 本文件为冻结求解源码的驼峰审查副本; 来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Q4 switches all properties to Appendix 4 and follows observed radial shrinkage."""
from dryingCore import Settings, solveCase

# Q4 从 t=0 使用整组附录 4 系数, 并按观测 R(t) 同比径向收缩, 固定长度。
def solve(intervals=6400):
    return solveCase(Settings(question='Q4', intervals=intervals, faceScheme='kirchhoff',
                              shrink=True, rtol=1e-10, atolTemperature=1e-10, atolMoisture=1e-12,

```

```
earlyMaxStepS=2., maxStepS=120.))
```

B.6 生产核心模块(蛇形命名同源实现)

`modeling/drying\_core.py` (与 B.5 同源的生产实现)

```
"""2026 A: radial heat and dry-basis moisture transport on a material mesh.
```

Units: s, m, K, kg water / kg dry matter. See numerical_design.md for derivation.

The supplied empirical rho*cp is an effective thermal capacity. Dry-solid mass is conserved separately on uniformly shrinking material control volumes.

No latent heat in the baseline; optional surface-latent scenario is labelled.

No clipping of solution values. Coefficients use a positive continuation only for integrator Newton probes; all accepted states are checked independently.

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import asdict, dataclass
```

```
from pathlib import Path
```

```
import hashlib
```

```
import io
```

```
import json
```

```
import time
```

```
import numpy as np
```

```
from scipy.integrate import solve_ivp
```

```
from scipy.sparse import lil_matrix
```

```
from scipy.special import expi
```

```
import analytic_jacobian
```

```
import disk_dense
```

```
ROOT = Path(__file__).resolve().parents[3]
```

```
LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
```

```
C0, T0, R0, LENGTH = 2.55, 301.15, 0.02, 0.25
```

```
def load_inputs(with_records=False):
```

```
    arrays, records = [], []
```

```
    for name in ['A_environment_observed.csv', 'A_radius_observed.csv']:
```

```
        path = ROOT / 'paper_output/data_cleaned' / name
```

```
        content = path.read_bytes()
```

```
        arrays.append(np.genfromtxt(io.StringIO(content.decode('utf-8-sig')),  
                                   delimiter=',', names=True))
```

```
        records.append({'path':path.relative_to(ROOT).as_posix(), 'bytes':len(content),  
                       'sha256':hashlib.sha256(content).hexdigest(), 'exists':True})
```

```
    return (*arrays, records) if with_records else tuple(arrays)
```

```
@dataclass(frozen=True)
```

```
class Settings:
```

```
    question: str = 'Q23'
```

```
    intervals: int = 100
```

```
    rtol: float = 1e-7
```

```
    atol_temperature: float = 1e-7
```

```
    atol_moisture: float = 1e-9
```

```
    max_step_s: float = 600.0
```

```
    early_max_step_s: float = 30.0
```

```
    horizon_h: float = 240.0
```

```
    shrink: bool = False
```

```
    boundary_extension: str = 'nominal'
```

```
    tail_temperature_C: float = 50.0
```

```
    tail_equilibrium: float = 0.05
```

```
    h: float = 25.0
```

```
    beta: float = 8e-7
```

```
    equilibrium_scale: float = 1.0
```

```
    surface_latent_fraction: float = 0.0
```

```
    latent_J_kg: float = 2.4e6
```

```
    constant_D: float | None = None
```

```
    constant_thermal: bool = False
```

```
    method: str = 'BDF'
```

```
    face_scheme: str = 'harmonic'
```

```
    jacobian_mode: str = 'analytic'
```

```
dense_storage: str = 'disk'
```

```
class RadialModel:
```

```
    def __init__(self, settings: Settings):
        self.settings = settings
        if settings.intervals < 2 or settings.jacobian_mode not in ('analytic', 'finite_difference'):
            raise ValueError('At least two intervals and a supported Jacobian mode are required')
        self.env, self.rad, self.input_records = load_inputs(with_records=True)
        self.x = np.linspace(0., 1., settings.intervals + 1)
        self.dx = 1. / settings.intervals
        faces = np.r_[0., (self.x[1:] + self.x[:-1]) / 2., 1.]
        self.w = np.diff(faces ** 2) / 2.
        self.internal_faces = faces[1:-1]
        self.n = len(self.x)
        self.rho_d0 = (760 + 90 * C0) / (1 + C0) if settings.question == 'Q4' else (
            820. / (1 + C0) if settings.question == 'Q1' else (650 + 128 * C0) / (1 + C0))
        self.evaluations = 0
        self.jac_pattern = self._sparsity()
```

```
    def radius(self, t):
        if self.settings.shrink:
            return np.interp(t, self.rad['time_s'], self.rad['radius_m'])
        return np.asarray(t) * 0. + R0
```

```
    def environment(self, t):
        s = self.settings
        tair = np.interp(t, self.env['time_s'], self.env['temperature_K'])
        ceq = np.interp(t, self.env['time_s'], self.env['air_moisture_kg_per_kg'])
        after = np.asarray(t) > self.env['time_s'][-1]
        if s.boundary_extension == 'nominal':
            tair = np.where(after, s.tail_temperature_C + 273.15, tair)
            ceq = np.where(after, s.tail_equilibrium, ceq)
        elif s.boundary_extension == 'tail_mean':
            tail = self.env['time_s'] >= 10800
            tair = np.where(after, self.env['temperature_K'][tail].mean(), tair)
            ceq = np.where(after, self.env['air_moisture_kg_per_kg'][tail].mean(), ceq)
        elif s.boundary_extension != 'last':
            raise ValueError('Unknown boundary extension')
        return tair, ceq * s.equilibrium_scale
```

```
    def properties(self, T, C):
        s = self.settings
        positive_C = np.maximum(C, 1e-12) # coefficient continuation, never state clipping
        if np.any(T <= 0):
            raise FloatingPointError('Nonpositive absolute temperature')
        wet = positive_C / (1. + positive_C)
        if s.question == 'Q1':
            rho = np.full_like(C, 820.)
            cp = np.full_like(C, 2600.)
            k = np.full_like(C, .36)
            D = 7e-9 * np.exp(-.89 / positive_C)
        elif s.question in ('Q2', 'Q3', 'Q23'):
            rho, cp, k = 650 + 128 * positive_C, 1450 + 2736 * wet, .21 + .38 * wet
            D = 2.4e-3 * np.exp(-.45 / positive_C - 3850 / T)
        elif s.question == 'Q4':
            rho, cp, k = 760 + 90 * positive_C, 1850 + 2150 * wet, .12 + .20 * wet
            D = 4.2e-4 * np.exp(-.30 / positive_C - 3850 / T)
        else:
            raise ValueError(s.question)
        if s.constant_D is not None:
            D = np.full_like(C, s.constant_D)
        if s.constant_thermal:
            rho, cp, k = np.full_like(C, 820.), np.full_like(C, 2600.), np.full_like(C, .36)
        return rho, cp, k, D
```

```
@staticmethod
```

```
def harmonic(a):
    return 2 * a[:-1] * a[1:] / np.maximum(a[:-1] + a[1:], np.finfo(float).tiny)
```

```
def _sparsity(self):
```

```

p = lil_matrix((2 * self.n + 1, 2 * self.n + 1), dtype=int)
for i in range(self.n):
    for j in range(max(0, i - 1), min(self.n, i + 2)):
        p[2*i:2*i+2, 2*j:2*j+2] = 1
p[-1, 2 * (self.n - 1) + 1] = 1
return p.tocsr()

def water_internal_flux(self, T, C, D):
    if self.settings.face_scheme == 'harmonic' or self.settings.constant_D is not None:
        return self.internal_faces * self.harmonic(D) * np.diff(C) / self.dx
    if self.settings.face_scheme != 'kirchhoff':
        raise ValueError("Unknown nonlinear face scheme")
    a, D0 = {'Q1':(.89,7e-9), 'Q23':(.45,2.4e-3), 'Q2':(.45,2.4e-3),
            'Q3':(.45,2.4e-3), 'Q4':(.30,4.2e-4)}[self.settings.question]
    cc = np.maximum(C, 1e-12)
    potential = cc * np.exp(-a/cc) + a * expi(-a/cc)
    difference = np.diff(potential)
    small = np.abs(np.diff(cc)) < 1e-7 * np.maximum((cc[:-1]+cc[1:])/2, 1e-3)
    difference[small] = (np.exp(-a/((cc[:-1][small]+cc[1:][small])/2)) * np.diff(cc)[small])
    thermal_factor = 1. if self.settings.question == 'Q1' else np.exp(-3850/((T[:-1]+T[1:])/2))
    # Do not difference thermal_factor*potential: that would add a false Soret flux.
    return self.internal_faces * D0 * thermal_factor * difference / self.dx

def rhs(self, t, state):
    """REVIEW: actual material-control-volume balance, no extra mesh advection."""
    self.evaluations += 1
    T, C = state[:-1:2], state[1:-1:2]
    rho, cp, k, D = self.properties(T, C)
    radius = float(self.radius(t))
    tair, ceq = self.environment(t)
    heat_g = np.zeros(self.n + 1)
    water_g = np.zeros(self.n + 1)
    heat_g[1:-1] = self.internal_faces * self.harmonic(k) * np.diff(T) / self.dx
    water_g[1:-1] = self.water_internal_flux(T, C, D)
    water_g[-1] = -self.settings.beta * radius * (C[-1] - ceq)
    heat_g[-1] = -self.settings.h * radius * (T[-1] - tair)
    if self.settings.surface_latent_fraction:
        # Scenario: all selected outgoing water vaporizes at the surface.
        rho_d = self.rho_d0 * (R0 / radius) ** 2
        j_evap = rho_d * self.settings.beta * (C[-1] - ceq)
        heat_g[-1] -= (radius * self.settings.surface_latent_fraction *
                      self.settings.latent_l_kg * j_evap)
    derivative = np.empty_like(state)
    derivative[:-1:2] = np.diff(heat_g) / (radius ** 2 * self.w * rho * cp)
    derivative[1:-1:2] = np.diff(water_g) / (radius ** 2 * self.w)
    derivative[-1] = 2 * self.settings.beta / radius * (C[-1] - ceq)
    return derivative

def initial(self):
    state = np.empty(2 * self.n + 1)
    state[:-1:2], state[1:-1:2], state[-1] = T0, C0, 0.
    return state

class Run:
    def __init__(self, model, pieces, elapsed, event_s, cache=None):
        self.model, self.pieces = model, pieces
        self.cache = cache
        self.elapsed_s, self.event_s = elapsed, event_s
        self.end_s = float(pieces[-1].t[-1])

    def close(self):
        """Release this Run after its exports/checks; further queries are invalid."""
        self.pieces.clear()
        if self.cache is not None:
            self.cache.close()

    def state(self, times):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        if np.min(tt) < -1e-10 or np.max(tt) > self.end_s + 1e-7:
            raise ValueError('Requested time outside solved interval')

```

```

out = np.empty((2 * self.model.n + 1, len(tt)))
remaining = np.ones(len(tt), dtype=bool)
for result in self.pieces:
    select = remaining & (tt >= result.t[0]-1e-7) & (tt <= result.t[-1]+1e-7)
    if np.any(select):
        out[:, select] = result.sol(tt[select])
        remaining[select] = False
if np.any(remaining):
    raise RuntimeError('Missing dense solution segment')
return out

def fields(self, times, radii_m=None, material_x=None):
    tt = np.atleast_1d(np.asarray(times, dtype=float))
    state = self.state(tt)
    Ts, Cs = state[:-1:2].T, state[1:-1:2].T
    if material_x is not None:
        points = np.asarray(material_x, dtype=float)
        if np.any(~np.isfinite(points)) or np.any((points < 0.) | (points > 1.)):
            raise ValueError('Material coordinates must be finite and within [0,1]')
        return np.array([np.interp(points, self.model.x, row) for row in Ts]), np.array([
            np.interp(points, self.model.x, row) for row in Cs])
    if radii_m is None:
        return Ts, Cs
    radial = np.asarray(radii_m)
    Tout, Cout = [], []
    for i, t in enumerate(tt):
        xx = radial / self.model.radius(t)
        Tout.append(np.interp(xx, self.model.x, Ts[i], left=np.nan, right=np.nan))
        Cout.append(np.interp(xx, self.model.x, Cs[i], left=np.nan, right=np.nan))
    return np.asarray(Tout), np.asarray(Cout)

def diagnostics(self):
    # Reduce in bounded blocks: a fine full-domain run may contain tens of
    # millions of accepted state values. Diagnostics must not duplicate all
    # of them and four property arrays at the same time.
    minimum_C = minimum_T = minimum_D = minimum_property = np.inf
    maximum_C = maximum_T = maximum_D = maximum_radial_increase = -np.inf
    maximum_mass_residual = 0.
    for piece in self.pieces:
        for first in range(0, len(piece.t), 256):
            raw = piece.y[:, first:first+256]
            T, C = raw[:-1:2], raw[1:-1:2]
            residual = 2*self.model.w*C + raw[-1] - C0
            rho, cp, k, D = self.model.properties(T.ravel(), C.ravel())
            minimum_C, maximum_C = min(minimum_C, C.min()), max(maximum_C, C.max())
            minimum_T, maximum_T = min(minimum_T, T.min()), max(maximum_T, T.max())
            minimum_D = min(minimum_D, D.min())
            maximum_D = max(maximum_D, D.max())
            minimum_property = min(minimum_property, rho.min(), cp.min(), k.min(), D.min())
            maximum_radial_increase = max(maximum_radial_increase, np.max(np.diff(C, axis=0)))
            maximum_mass_residual = max(maximum_mass_residual, np.max(np.abs(residual)))
    final = self.state([self.end_s])[0, :]
    finalC = final[1:-1:2]
    return {
        'event_s': self.event_s, 'event_h': None if self.event_s is None else self.event_s/3600,
        'end_s': self.end_s, 'elapsed_s': self.elapsed_s,
        'end_time_convention': 'ceil(critical_event_s)+1: conservative post-crossing verification second; not
claimed earliest integer second',
        'max_mass_balance_abs_kg_per_kg': float(maximum_mass_residual),
        'min_C': float(minimum_C), 'max_C': float(maximum_C),
        'min_T_K': float(minimum_T), 'max_T_K': float(maximum_T),
        'min_D': float(minimum_D), 'max_D': float(maximum_D),
        'positive_properties': bool(minimum_property > 0),
        'max_radial_C_increase': float(maximum_radial_increase),
        'final_max_C': float(finalC.max()), 'final_surface_C': float(finalC[-1]),
        'strictly_dry_at_end': bool(finalC.max() < .15),
        'radius_end_m': float(self.model.radius(self.end_s)),
        'radius_extrapolation_used': bool(self.model.settings.shrink and self.end_s > 259200),
        'rhs_evaluations': self.model.evaluations,
        'accepted_time_points': sum(len(p.t) for p in self.pieces),
        'nfev': sum(p.nfev for p in self.pieces),
        'njev': sum(p.njev for p in self.pieces), 'nlu': sum(p.nlu for p in self.pieces),
        'solver_success': all(p.success for p in self.pieces),

```

```

        'dense_storage': self.model.settings.dense_storage,
        'dense_coefficient_bytes': 0 if self.cache is None else self.cache.bytes_written,
        'dense_polynomial_count': 0 if self.cache is None else self.cache.polynomial_count,
    }

def solve_case(settings: Settings) -> Run:
    started = time.perf_counter()
    if settings.dense_storage not in ('memory', 'disk'):
        raise ValueError('Dense storage must be memory or disk')
    if settings.dense_storage == 'disk' and settings.method != 'BDF':
        raise ValueError('Exact disk dense storage currently supports BDF only')
    cache = disk_dense.DenseCache(ROOT) if settings.dense_storage == 'disk' else None
    try:
        return _solve_case_impl(settings, cache, started)
    except BaseException as error:
        if cache is not None:
            try:
                cache.close()
            except BaseException as cleanup_error:
                error.add_note('Private cache cleanup also failed: '+repr(cleanup_error))
        raise

def _solve_case_impl(settings, cache, started):
    model = RadialModel(settings)
    method = disk_dense.DiskBDF if cache is not None else settings.method
    jacobian_options = ({'jac': lambda t, y: analytic_jacobian.jacobian(model, t, y)}
        if settings.jacobian_mode == 'analytic' else {'jac_sparsity': model.jac_pattern})
    if cache is not None:
        jacobian_options['dense_cache'] = cache
    def dry_event(t, y):
        return float(np.max(y[1:-1:2]) - .15)
    dry_event.terminal, dry_event.direction = True, -1
    atol = np.empty(2 * model.n + 1)
    atol[1:-1:2], atol[1:-1:2], atol[-1] = settings.atol_temperature, settings.atol_moisture,
    settings.atol_moisture
    horizon = 1800. if settings.question == 'Q1' else settings.horizon_h * 3600.
    pieces, initial, event_s = [], model.initial(), None
    # A separate segment at 4 h makes the modelling extension explicit.
    endpoints = [0., min(14400., horizon)]
    if horizon > 14400.:
        endpoints.append(horizon)
    for left, right in zip(endpoints[:-1], endpoints[1:]):
        piece = solve_ivp(model.rhs, (left, right), initial, method=method,
            rtol=settings.rtol, atol=atol, **jacobian_options,
            max_step=settings.early_max_step_s if left < 14400. else settings.max_step_s,
            events=None if settings.question == 'Q1' else dry_event, dense_output=True)
        if cache is not None:
            disk_dense.align_bdf_segments(piece)
        pieces.append(piece)
        if not piece.success:
            raise RuntimeError(piece.message)
        initial = piece.y[:, -1].copy()
        if cache is not None:
            piece.y = cache.store_accepted(piece.y)
        if piece.t_events is not None and len(piece.t_events[0]):
            event_s = float(piece.t_events[0][0])
            # Continue to a genuine post-crossing integer second, not an extrapolation.
            end = float(np.ceil(event_s) + 1)
            tail = solve_ivp(model.rhs, (event_s, end), initial, method=method,
                rtol=settings.rtol, atol=atol, **jacobian_options,
                max_step=1., dense_output=True)
            if cache is not None:
                disk_dense.align_bdf_segments(tail)
            if not tail.success:
                raise RuntimeError(tail.message)
            if cache is not None:
                tail.y = cache.store_accepted(tail.y)
            pieces.append(tail)
        break
    run = Run(model, pieces, time.perf_counter() - started, event_s, cache)

```

```

    diagnostic = run.diagnostics()
    if diagnostic['min_c'] < -1e-8 or not diagnostic['positive_properties']:
        raise FloatingPointError('Physical range/positive property check failed')
    if diagnostic['max_mass_balance_abs_kg_per_kg'] > 1e-6:
        raise FloatingPointError('Dry-basis mass balance failed')
    return run

def file_record(path):
    p = Path(path)
    return {'path': p.relative_to(ROOT).as_posix(), 'bytes': p.stat().st_size,
            'sha256': hashlib.sha256(p.read_bytes()).hexdigest(), 'exists': True}

def save_run(run: Run, directory: Path):
    code_record = file_record(Path(__file__))
    if code_record['sha256'] != LOADED_CODE_SHA256:
        raise RuntimeError('Solver file changed after import; restart to obtain valid provenance')
    jacobian_record = file_record(Path(analytic_jacobian.__file__))
    if jacobian_record['sha256'] != analytic_jacobian.LOADED_CODE_SHA256:
        raise RuntimeError('Jacobian file changed after import; restart to obtain valid provenance')
    storage_record = file_record(Path(disk_dense.__file__))
    if storage_record['sha256'] != disk_dense.LOADED_CODE_SHA256:
        raise RuntimeError('Dense storage file changed after import; restart for valid provenance')
    for record in run.model.input_records:
        if file_record(ROOT/record['path'])['sha256'] != record['sha256']:
            raise RuntimeError('Input changed after being loaded; retain failure and rerun')
    directory.mkdir(parents=True, exist_ok=True)
    times = np.unique(np.r_[np.arange(0., run.end_s, 60.),
                            [t for t in [100., 300., 600., 900., 1200., 1500., 1800., 3600., 5400., 7200., 9000., 10800.] if t
<= run.end_s],
                        run.end_s, [] if run.event_s is None else [run.event_s]])
    x = np.linspace(0., 1., 21)
    temperature, moisture, means, losses = [], [], [], []
    for first in range(0, len(times), 128):
        block_times = times[first:first+128]
        Tb, Cb = run.fields(block_times, material_x=x)
        raw = run.state(block_times)
        temperature.append(Tb); moisture.append(Cb)
        means.append(2*run.model.w@raw[1:-1:2]); losses.append(raw[-1].copy())
    T, C = np.vstack(temperature), np.vstack(moisture)
    np.savez_compressed(directory/'sampled_solution.npz', times_s=times, material_x=x,
                        T_K=T, C=C, radius_m=run.model.radius(times), mean_C=np.concatenate(means),
                        cumulative_loss=np.concatenate(losses))
    summary = {'settings': asdict(run.model.settings), 'diagnostics': run.diagnostics(),
              'code': code_record,
              'jacobian_code': jacobian_record,
              'dense_storage_code': storage_record,
              'inputs': run.model.input_records,
              'human_review_status': 'pending', 'gui_reproduced': False}
    (directory/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
    return summary

`modeling/analytic_jacobian.py` (与 B.5 同源的解析 Jacobian)
"""Analytic sparse Jacobian for drying_core.RadialModel.rhs.

```

The ordering is T0,C0,...,TN,CN,A. A is a passive cumulative loss variable; its entire column is exactly zero and must not use adaptive numdiff factors. This file does not modify the core or select a production configuration.

Self-check from the contest root:

```
C:\Python314\python.exe -B paper_output/code/modeling/analytic_jacobian.py --self-test
```

```
from __future__ import annotations
```

```

import argparse
from dataclasses import asdict
from datetime import datetime, timezone
import hashlib
import json
from pathlib import Path

```

```

import sys
import time
import warnings

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import coo_matrix
from scipy.special import expi

LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

def _harmonic_partials(left, right):
    """Positive-coefficient partials; respect the core's tiny denominator guard."""
    total = left + right
    tiny = np.finfo(float).tiny
    denominator = np.maximum(total, tiny)
    ordinary = total > tiny
    dl = np.where(ordinary, 2 * (right / denominator) ** 2, 2 * right / denominator)
    dr = np.where(ordinary, 2 * (left / denominator) ** 2, 2 * left / denominator)
    return dl, dr

def jacobian(model, t, y):
    """Return d(rhs)/d(y) as CSC without calling rhs or numerical differentiation.

    Each internal face contributes opposite flux derivatives to its two cells.
    For heat, differentiating 1/(rho*cp) adds -Tdot*cap_C/cap locally.
    Kirchhoff's temperature factor is frozen at the symmetric face temperature;
    the primitive's close-concentration branch is differentiated exactly as coded.
    """
    state = np.asarray(y, dtype=float)
    n = model.n
    if state.ndim != 1 or state.size != 2*n + 1:
        raise ValueError("Jacobian requires interleaved 1D state of length 2*n+1")
    s = model.settings
    T, C = state[:-1:2], state[1:-1:2]
    cc = np.maximum(C, 1e-12)
    active = (C > 1e-12).astype(float)
    rho, cp, k, D = model.properties(T, C)
    if s.question == 'Q1':
        a, d0, temp_constant = .89, 7e-9, 0.
        rho_c, cp_c, k_c = np.zeros(n), np.zeros(n), np.zeros(n)
    elif s.question in ('Q2', 'Q3', 'Q23'):
        a, d0, temp_constant = .45, 2.4e-3, 3850.
        rho_c = np.full(n, 128.) * active
        cp_c = 2736. / (1. + cc)**2 * active
        k_c = .38 / (1. + cc)**2 * active
    elif s.question == 'Q4':
        a, d0, temp_constant = .30, 4.2e-4, 3850.
        rho_c = np.full(n, 90.) * active
        cp_c = 2150. / (1. + cc)**2 * active
        k_c = .20 / (1. + cc)**2 * active
    else:
        raise ValueError(s.question)
    if s.constant_thermal:
        rho_c, cp_c, k_c = np.zeros(n), np.zeros(n), np.zeros(n)
    capacity = rho * cp
    capacity_c = rho_c * cp + rho * cp_c
    if s.constant_D is None:
        d_c = D * (a / cc**2) * active
        d_t = D * temp_constant / T**2
    else:
        d_c, d_t = np.zeros(n), np.zeros(n)

    radius = float(model.radius(t))
    if radius <= 0:
        raise ValueError("Nonpositive radius")
    tair, ceq = model.environment(t)

```

```

geom = model.internal_faces / model.dx
delta_t, delta_c = np.diff(T), np.diff(C)

# Derivative columns for each face are (T_left,C_left,T_right,C_right).
k_harm = model.harmonic(k)
kh_l, kh_r = _harmonic_partials(k[:-1], k[1:])
heat_deriv = np.column_stack((
    -geom * k_harm,
    geom * kh_l * k_c[:-1] * delta_t,
    geom * k_harm,
    geom * kh_r * k_c[1:] * delta_t,
))
heat_g = np.zeros(n + 1)
heat_g[1:-1] = geom * k_harm * delta_t
heat_g[-1] = -s.h * radius * (T[-1] - tair)

if s.face_scheme == 'harmonic' or s.constant_D is not None:
    dh = model.harmonic(D)
    dh_l, dh_r = _harmonic_partials(D[:-1], D[1:])
    water_deriv = np.column_stack((
        geom * dh_l * d_t[:-1] * delta_c,
        geom * (dh_l * d_c[:-1] * delta_c - dh),
        geom * dh_r * d_t[1:] * delta_c,
        geom * (dh_r * d_c[1:] * delta_c + dh),
    ))
elif s.face_scheme == 'kirchhoff':
    primitive = cc * np.exp(-a / cc) + a * expi(-a / cc)
    primitive_delta = np.diff(primitive)
    mean_c = (cc[:-1] + cc[1:]) / 2.
    mean_t = (T[:-1] + T[1:]) / 2.
    delta_cc = np.diff(cc)
    small = np.abs(delta_cc) < 1e-7 * np.maximum(mean_c, 1e-3)
    exp_left, exp_right = np.exp(-a/cc[:-1]), np.exp(-a/cc[1:])
    primitive_l = -exp_left * active[:-1]
    primitive_r = exp_right * active[1:]
    if np.any(small):
        f_mid = np.exp(-a / mean_c[small])
        f_prime_mid = f_mid * a / mean_c[small]**2
        primitive_delta[small] = f_mid * delta_cc[small]
        primitive_l[small] = (0.5*f_prime_mid*delta_cc[small]-f_mid)*active[:-1][small]
        primitive_r[small] = (0.5*f_prime_mid*delta_cc[small]+f_mid)*active[1:][small]
    temperature_factor = np.exp(-temp_constant / mean_t)
    coefficient = geom * d0 * temperature_factor
    water_g = coefficient * primitive_delta
    water_t = water_g * temp_constant / (2 * mean_t**2)
    water_deriv = np.column_stack((water_t, coefficient*primitive_l,
                                   water_t, coefficient*primitive_r))
else:
    raise ValueError('Unknown nonlinear face scheme')

heat_scale = 1. / (radius**2 * model.w * capacity)
water_scale = 1. / (radius**2 * model.w)
surface_heat_c = 0.
if s.surface_latent_fraction:
    rho_d = model.rho_d0 * (.02 / radius)**2
    surface_heat_c = -radius*s.surface_latent_fraction*s.latent_J_kg*rho_d*s.beta
    heat_g[-1] += surface_heat_c * (C[-1] - ceq)
temp_derivative = np.diff(heat_g) * heat_scale

face_index = np.arange(n-1)
face_columns = np.column_stack((2*face_index, 2*face_index+1,
                                2*face_index+2, 2*face_index+3)).ravel()
rows, columns, values = [], [], []

def add_face(row_index, derivatives, factor):
    rows.append(np.repeat(row_index, 4))
    columns.append(face_columns)
    values.append((derivatives * factor[:, None]).ravel())

add_face(2*face_index, heat_deriv, heat_scale[:-1])
add_face(2*face_index+2, heat_deriv, -heat_scale[1:])

```

```

add_face(2*face_index+1, water_deriv, water_scale[:-1])
add_face(2*face_index+3, water_deriv, -water_scale[1:])
cell_index = np.arange(n)
rows.append(2*cell_index)
columns.append(2*cell_index+1)
values.append(-temp_derivative * capacity_c / capacity)
rows.append(np.array([2*n-2, 2*n-2, 2*n-1, 2*n]))
columns.append(np.array([2*n-2, 2*n-1, 2*n-1, 2*n-1]))
values.append(np.array([-s.h*radius*heat_scale[-1],
                        surface_heat_c*heat_scale[-1],
                        -s.beta*radius*water_scale[-1], 2*s.beta/radius]))
matrix = coo_matrix((np.concatenate(values),
                    (np.concatenate(rows), np.concatenate(columns))),
                    shape=(2*n+1, 2*n+1)).tocsc()
matrix.sum_duplicates()
matrix.eliminate_zeros()
return matrix

def _self_test(output_directory):
    """Independent RHS perturbation checks, a conservation derivative, and tiny BDF runs."""
    import scipy
    from drying_core import ROOT, Settings, RadialModel, LOADED_CODE_SHA256 as CORE_LOADED_CODE_SHA256

    started = time.perf_counter()
    rng = np.random.default_rng(20260910)
    relative_tolerance, absolute_tolerance = 5e-6, 5e-10
    cases = []
    for question in ['Q1', 'Q23', 'Q4']:
        for scheme in ['harmonic', 'kirchhoff']:
            for latent in [0., 1.]:
                cases.append(Settings(question=question, intervals=8, face_scheme=scheme,
                                     shrink=(question=='Q4'), surface_latent_fraction=latent))
    for question in ['Q1', 'Q23', 'Q4']:
        for constant_d, constant_thermal in [(2e-9, False), (None, True), (2e-9, True)]:
            cases.append(Settings(question=question, intervals=8, face_scheme='kirchhoff',
                                 shrink=(question=='Q4'), constant_D=constant_d,
                                 constant_thermal=constant_thermal, surface_latent_fraction=1.))
    records, failures = [], []
    for settings in cases:
        model = RadialModel(settings)
        x = model.x
        states = {
            'initial': (0., model.initial()),
            'nonuniform': (18000., model.initial()),
            'late_dry': (150000., model.initial()),
            'near_uniform_small_branch': (14401., model.initial()),
        }
        states['nonuniform'][1][:-1:2] = 303. + 17.*x**2
        states['nonuniform'][1][1:-1:2] = 2.4 - 2.0*x**2
        states['late_dry'][1][:-1:2] = 321. + 2.0*x**2
        states['late_dry'][1][1:-1:2] = .175 - .115*x**2
        states['near_uniform_small_branch'][1][:-1:2] = 303. + 17.*x**2
        states['near_uniform_small_branch'][1][1:-1:2] = .15 + 1e-10*x
        for state_name, (t, y) in states.items():
            with warnings.catch_warnings(record=True) as captured:
                warnings.simplefilter('always')
                matrix = jacobian(model, t, y)
                vectors = []
                for _ in range(4):
                    direction = rng.normal(size=y.size)
                    direction[:-1:2] *= 1.0
                    direction[1:-1:2] *= .02
                    direction[-1] = .3
                    vectors.append(direction)
                directional = []
                for direction in vectors:
                    predicted = matrix @ direction
                    steps = []
                    for step in [1e-4, 3e-5, 1e-5]:
                        finite_difference = (model.rhs(t, y+step*direction)-
                                           model.rhs(t, y-step*direction))/(2*step)

```

```

        absolute_error = float(np.max(np.abs(predicted-finite_difference)))
        scale = max(float(np.max(np.abs(predicted))),
                    float(np.max(np.abs(finite_difference))), 1e-30)
        scaled = float(np.max(np.abs(predicted-finite_difference)/
                               (absolute_tolerance + relative_tolerance*np.maximum(
                                   np.abs(predicted), np.abs(finite_difference)))))
        steps.append({'step':step, 'max_abs_error':absolute_error,
                     'relative_inf_error':absolute_error/scale,
                     'max_component_tolerance_ratio':scaled})
        best = min(steps, key=lambda item:item['max_component_tolerance_ratio'])
        directional.append({'all_step_errors':steps, 'best':best})
        passive = np.zeros(y.size); passive[-1] = 1.
        passive_analytic = float(np.max(np.abs(matrix @ passive)))
        passive_fd = float(np.max(np.abs(model.rhs(t, y+passive)-model.rhs(t,y-passive))))
        mass_weights = np.zeros(y.size)
        mass_weights[1:-1:2] = 2*model.w
        mass_weights[-1] = 1.
        conservation = float(np.max(np.abs(np.asarray(mass_weights @ matrix))))
        warning_messages = [str(w.message) for w in captured]
        passed = (all(step['max_component_tolerance_ratio'] <= 1
                      for d in directional for step in d['all_step_errors'])
                 and passive_analytic == 0 and passive_fd == 0 and conservation < 1e-11
                 and np.isfinite(matrix.data).all() and not warning_messages)
        record = {'settings':asdict(settings), 'state':state_name, 'time_s':t,
                 'status':'PASS' if passed else 'FAIL', 'matrix_shape':matrix.shape,
                 'matrix_nnz':matrix.nnz, 'directional_checks':directional,
                 'passive_column_analytic_abs':passive_analytic,
                 'passive_column_finite_difference_abs':passive_fd,
                 'mass_balance_derivative_abs':conservation, 'warnings':warning_messages}
        records.append(record)
    if not passed:
        failures.append(f"{settings.question}/{settings.face_scheme}/latent={settings.surface_latent_fraction}/
{state_name}/constant_D={settings.constant_D}/constant_thermal={settings.constant_thermal}")
        smoke_records = []
        for scheme in ['harmonic', 'kirchhoff']:
            for question in ['Q23', 'Q4']:
                settings = Settings(question=question, intervals=8, face_scheme=scheme,
                                   shrink=(question=='Q4'), surface_latent_fraction=1.)
                model = RadialModel(settings)
                with warnings.catch_warnings(record=True) as captured:
                    warnings.simplefilter('always')
                    result = solve_ivp(model.rhs, (0., 2.), model.initial(), method='BDF',
                                       jac=lambda t,y:jacobian(model,t,y), rtol=1e-10,
                                       atol=1e-12, max_step=.2)
                msgs = [str(w.message) for w in captured]
                passed = bool(result.success and np.isfinite(result.y).all() and not msgs)
                smoke_records.append({'question':question, 'face_scheme':scheme, 'interval_s':[0,2],
                                    'status':'PASS' if passed else 'FAIL', 'warnings':msgs,
                                    'nfev':result.nfev, 'njev':result.njev, 'nlu':result.nlu,
                                    'purpose':'Short Jacobian/BDF wiring check, not production accuracy'})
            if not passed:
                failures.append(f"BDF_smoke/{question}/{scheme}")
        current_source_hash = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
        current_core_hash =
hashlib.sha256((ROOT/'paper_output/code/modeling/drying_core.py').read_bytes()).hexdigest()
        if current_source_hash != LOADED_CODE_SHA256 or current_core_hash != CORE_LOADED_CODE_SHA256:
            failures.append('Source file changed after module load during validation')
        report = {'schema_version':'1.0', 'created_utc':datetime.now(timezone.utc).isoformat(),
                 'status':'PASS' if not failures else 'FAIL', 'failures':failures,
                 'source_sha256':LOADED_CODE_SHA256,
                 'core_sha256':CORE_LOADED_CODE_SHA256,
                 'source_hash_policy':'SHA-256 frozen when each module is imported; files checked unchanged before
report save.',
                 'runtime':
{'python':sys.version,'executable':sys.executable,'numpy':np.__version__, 'scipy':scipy.__version__,
 'seed':20260910, 'relative_component_tolerance':relative_tolerance,
 'absolute_component_tolerance':absolute_tolerance,
 'finite_difference_policy':'Central directional differences, three decreasing steps; every step must
satisfy the mixed absolute/relative component tolerance. Best comparison is supplementary only.',
 'case_state_count':len(records), 'direction_count':4*len(records),
 'checks':records, 'short_BDF_checks':smoke_records,
 'elapsed_s':time.perf_counter()-started,

```

```

        'limitations':['No full production run or grid convergence performed here.',
        'Tiny denominator/underflow extensions at nonphysical Newton probes are not
calibrated physical data.',
        'Only this module and the short tests use analytic Jacobian until main agent hooks
core.',
        'Visual Studio reproduction and human review remain pending.']]
output_directory.mkdir(parents=True, exist_ok=True)
(output_directory/'analytic_jacobian_selftest.json').write_text(
    json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
brief = {key:report[key] for key in ['status','failures','case_state_count','direction_count','elapsed_s']}
brief['max_best_component_tolerance_ratio'] = max(
    d['best']['max_component_tolerance_ratio'] for r in records for d in r['directional_checks'])
brief['max_all_steps_component_tolerance_ratio'] = max(
    step['max_component_tolerance_ratio'] for r in records
    for d in r['directional_checks'] for step in d['all_step_errors'])
brief['max_mass_balance_derivative_abs'] = max(r['mass_balance_derivative_abs'] for r in records)
brief['warning_count'] = sum(len(r['warnings']) for r in records+smoke_records)
print(json.dumps(brief,ensure_ascii=False,indent=2))
return 0 if not failures else 1

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--self-test', action='store_true')
    parser.add_argument('--output-dir', type=Path,
        default=Path(__file__).resolve().parents[2]/'results/jacobian_validation')
    arguments = parser.parse_args()
    if not arguments.self_test:
        parser.error('Use --self-test, or import jacobian(model,t,y) from this module.')
    raise SystemExit(_self_test(arguments.output_dir))

`modeling/disk_dense.py` (与 B.5 同源的稠密存储模块)
"""Exact BDF dense polynomials backed by a private, rebuildable disk cache.

This changes storage only: each accepted BDF polynomial is written as float64
bytes, then evaluated with SciPy's original BdfDenseOutput implementation.
No additional time/space interpolation and no solver restart are introduced.
"""
from __future__ import annotations
import hashlib
from pathlib import Path
import tempfile
import numpy as np
from scipy.integrate import OdeSolution
from scipy.integrate._ivp.bdf import BDF, BdfDenseOutput
from scipy.integrate._ivp.base import DenseOutput

LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

class DenseCache:
    def __init__(self, root):
        self.parent = (Path(root)/'tmp/cache/solver_runs').resolve()
        if not self.parent.is_relative_to(Path(root).resolve()):
            raise ValueError('Private solver cache must remain in this project')
        self.parent.mkdir(parents=True, exist_ok=True)
        self.directory = Path(tempfile.mkdtemp(prefix='bdf_', dir=self.parent)).resolve()
        try:
            self.handle = (self.directory/'polynomials.bin').open('w+b', buffering=0)
        except BaseException as error:
            try:
                self.directory.rmdir()
            except OSError as cleanup_error:
                error.add_note('Private cache directory cleanup failed: '+repr(cleanup_error))
            raise
        self.bytes_written = 0
        self.polynomial_count = 0
        self.accepted_arrays = []
        self.closed = False

    def append(self, values):

```

```

        if self.closed:
            raise RuntimeError('Dense solution cache is closed')
        array = np.ascontiguousarray(values, dtype=np.float64)
        offset = self.bytes_written
        self.handle.seek(offset)
        array.tofile(self.handle)
        self.bytes_written += array.nbytes
        self.polynomial_count += 1
        return offset, array.shape

def read(self, offset, shape):
    if self.closed:
        raise RuntimeError('Dense solution cache is closed')
    count = int(np.prod(shape))
    self.handle.seek(offset)
    values = np.fromfile(self.handle, dtype=np.float64, count=count)
    if values.size != count:
        raise IOError('Incomplete BDF coefficient cache')
    return values.reshape(shape)

def store_accepted(self, values):
    path = self.directory/f'accepted_{len(self.accepted_arrays):03d}.npy'
    mapped = np.lib.format.open_mmap(path, mode='w+', dtype=values.dtype, shape=values.shape)
    # Register before writing so failures can close this Windows mapping.
    self.accepted_arrays.append(mapped)
    mapped[:] = values
    mapped.flush()
    return mapped

def close(self):
    if self.closed:
        return
    self.handle.close()
    for array in self.accepted_arrays:
        array._mmap.close()
    self.accepted_arrays.clear()
    # Delete only the verified private cache created by this object.
    if self.directory.parent != self.parent or not self.directory.name.startswith('bdf_'):
        raise RuntimeError('Unexpected private cache path; cleanup refused')
    for path in self.directory.iterdir():
        if not path.is_file() or path.is_symlink():
            raise RuntimeError('Unexpected cache entry; cleanup refused')
        path.unlink()
    self.directory.rmdir()
    self.closed = True

class FileBdfDenseOutput(DenseOutput):
    def __init__(self, original, cache):
        super().__init__(original.t_old, original.t)
        self.order = original.order
        self.t_shift = original.t_shift.copy()
        self.denom = original.denom.copy()
        self.cache = cache
        self.offset, self.shape = cache.append(original.D)

    def _call_impl(self, t):
        # Reuse the installed SciPy evaluator with the exact recorded D bytes.
        dense = object.__new__(BdfDenseOutput)
        dense.D = self.cache.read(self.offset, self.shape)
        dense.t_shift, dense.denom = self.t_shift, self.denom
        return dense._call_impl(t)

class DiskBDF(BDF):
    def __init__(self, *args, dense_cache, **kwargs):
        self.dense_cache = dense_cache
        super().__init__(*args, **kwargs)

    def _dense_output_impl(self):

```

```

        return FileBdfDenseOutput(super()._dense_output_impl(), self.dense_cache)

def align_bdf_segments(result):
    """Restore the original BDF convention at accepted time breakpoints.

    SciPy solve_ivp tests the exact method class for alt_segment. A subclass
    otherwise selects the opposite polynomial at a shared knot. Reconstructing
    OdeSolution changes only this selection, never coefficients or integration.
    """
    if result.sol is not None:
        result.sol = OdeSolution(result.sol.ts, result.sol.interpolants, alt_segment=True)

```

B.7 验证与交叉核验脚本

``verification/crossvalidate_solver.py``(积分器互换与两种湿面离散格式对照(表 31、表 32))

```

"""Cross-validation of the A-problem drying-time conclusion by independent methods.

Four independent blocks, all additive: no frozen model, setting, input or result
is modified. Everything is written under paper_output/results/crossvalidation/.

Block 1 time-integrator cross-validation    BDF vs Radau at fixed N
Block 2 water-flux face-scheme convergence Kirchhoff vs harmonic under refinement
Block 3 latent-heat model modification      surface evaporation-cooling scenarios
Block 4 threshold & scaling robustness      post-processing on the same solves

Sandbox note: disk_dense.DenseCache uses tempfile.mkdtemp, whose directories this
session's file sandbox cannot open. tempfile.mkdtemp is replaced by an equivalent
os.mkdir-based implementation before importing the solver. No solver logic changes.
"""
from __future__ import annotations

import json
import os
import pathlib
import sys
import tempfile
import time
import traceback
import uuid
from dataclasses import replace
from datetime import datetime, timezone

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
CODE = ROOT / "paper_output" / "code" / "modeling"

def _mkdtemp(suffix=None, prefix=None, dir=None):
    base = pathlib.Path(dir)
    for _ in range(200):
        candidate = base / ((prefix or "tmp") + uuid.uuid4().hex[:12] + (suffix or ""))
        if not candidate.exists():
            candidate.mkdir(parents=False)
            return str(candidate)
    raise FileExistsError("no unused temp name")

tempfile.mkdtemp = _mkdtemp

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(CODE))

import numpy as np # noqa: E402

import drying_core # noqa: E402

```

```

from drying_core import Settings, solve_case # noqa: E402

try:
    import q3_model # noqa: E402

    completion = q3_model.completion
except Exception: # pragma: no cover - fallback keeps the block self-contained
    completion = None

OUT = ROOT / "paper_output" / "results" / "crossvalidation"
BASE = Settings(intervals=800, rtol=1e-10, atol_temperature=1e-10,
                 atol_moisture=1e-12, early_max_step_s=2.0, max_step_s=120.0,
                 face_scheme="kirchhoff", jacobian_mode="analytic")

def utcnow():
    return datetime.now(timezone.utc).isoformat()

def runCase(name, settings):
    started = time.perf_counter()
    record = {"case": name, "settings": {k: v for k, v in settings.__dict__.items()},
             "startedAtUtc": utcnow()}
    run = None
    try:
        run = solve_case(settings)
        diagnostic = run.diagnostics()
        record["diagnostics"] = diagnostic
        record["event_s"] = run.event_s
        record["event_h"] = None if run.event_s is None else run.event_s / 3600.0
        record["end_s"] = run.end_s
        if completion is not None and settings.question != "Q1":
            record["completion"] = completion(run)
        record["status"] = "computed"
    except Exception as error: # keep the failure, never hide it
        record["status"] = "failed"
        record["error"] = f"{type(error).__name__}: {error}"
        record["traceback"] = traceback.format_exc()
    finally:
        if run is not None:
            try:
                run.close()
            except Exception:
                pass
    record["elapsedSeconds"] = time.perf_counter() - started
    print(json.dumps({k: record[k] for k in ("case", "status", "elapsedSeconds")}
                if k in record |
                {"event_h": record.get("event_h")}, ensure_ascii=False), flush=True)
    return record

def block1():
    """Independent time integrators at a fixed spatial discretisation.

    BDF/NDF (variable-order implicit) and Radau IIA (3-stage implicit Runge-Kutta)
    are different integration families with different truncation-error structure,
    so agreement at fixed N isolates the time integrator as an error source.

    LSODA was attempted and removed: SciPy's ODEPACK wrapper is dense, cannot
    consume the sparse analytic Jacobian, and with a dense finite-difference
    Jacobian at N = 200 it did not finish in a practical time. The third
    independent integrator is therefore the MATLAB ode15s implementation recorded
    under paper_output/qa/matlab_crosscheck_20260911, not a third SciPy method.
    """
    cases = []
    for question, shrink in (("Q23", False), ("Q4", True)):
        for intervals in (800, 1600):
            for method in ("BDF", "Radau"):
                settings = replace(BASE, question=question, shrink=shrink,

```

```

        intervals=intervals, method=method,
        dense_storage="disk" if method == "BDF" else "memory")
    cases.append((f"B1_{question}_{method}_N{intervals}", settings))
return cases

def block2():
    """Water-flux face representation under grid refinement.

    A coarse grid separates the two schemes by a large factor; the test is whether
    they converge to the same answer as the mesh is refined, which is what makes the
    production Kirchhoff result grid-independent rather than scheme-dependent.
    """
    cases = []
    for intervals in (400, 800, 1600):
        for scheme in ("kirchhoff", "harmonic"):
            settings = replace(BASE, question="Q23", intervals=intervals,
                               face_scheme=scheme)
            cases.append((f"B2_Q23_{scheme}_N{intervals}", settings))
    for intervals in (400, 800, 1600):
        for scheme in ("kirchhoff", "harmonic"):
            settings = replace(BASE, question="Q4", shrink=True,
                               intervals=intervals, face_scheme=scheme)
            cases.append((f"B2_Q4_{scheme}_N{intervals}", settings))
    return cases

def block3():
    """Model modification: explicit surface evaporative cooling (latent heat).

    The fraction is a scenario parameter, not a calibrated physical constant:
    the problem provides no sorption isotherm that would close the surface vapour
    balance, so an envelope is reported instead of a single point prediction.
    """
    cases = []
    for question, shrink in (("Q23", False), ("Q4", True)):
        for fraction in (0.0, 0.25, 0.5, 1.0):
            settings = replace(BASE, question=question, shrink=shrink,
                               intervals=800, surface_latent_fraction=fraction)
            cases.append((f"B3_{question}_L{fraction:g}_N800", settings))
            settings = replace(BASE, question=question, shrink=shrink, intervals=1600,
                               surface_latent_fraction=1.0)
            cases.append((f"B3_{question}_L1_N1600", settings))
    return cases

BLOCKS = {"1": block1, "2": block2, "3": block3}

def main():
    wanted = sys.argv[1] if len(sys.argv) > 1 else "1"
    runId = sys.argv[2] if len(sys.argv) > 2 else "cv_" + wanted
    directory = OUT / runId
    directory.mkdir(parents=True, exist_ok=True)
    records = []
    done = set()
    casesPath = directory / "cases.json"
    if casesPath.exists(): # resume: never recompute a case that already succeeded
        records = json.loads(casesPath.read_text(encoding="utf-8"))
        done = {r["case"] for r in records if r.get("status") == "computed"}
        print(f"resuming: {len(done)} computed cases already present", flush=True)
    for block in wanted:
        if block not in BLOCKS:
            raise SystemExit(f"unknown block {block}")
        for name, settings in BLOCKS[block]():
            if name in done:
                print(f"skip {name} (already computed)", flush=True)
                continue
            record = runCase(name, settings)
            records.append(record)

```

```

        casesPath.write_text(json.dumps(records, ensure_ascii=False, indent=2),
                               encoding="utf-8")
summary = {"runId": runId, "blocks": wanted,
           "solverCode": drying_core.file_record(pathlib.Path(drying_core.__file__)),
           "finishedAtUtc": utcnow(),
           "computed": sum(1 for r in records if r["status"] == "computed"),
           "failed": sum(1 for r in records if r["status"] == "failed")}
(directory / "summary.json").write_text(
    json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
print(json.dumps(summary, ensure_ascii=False), flush=True)
return 0

if __name__ == "__main__":
    sys.exit(main())

`verification/method_comparison.py` (同方程两种离散对照,含 fluxPathUsed 守卫)
"""Independent-discretisation cross-validation of the water-flux operator.

Two different numerical methods are applied to the SAME moisture equation with the
SAME time integrator and the SAME tolerances, so any disagreement is a spatial
discretisation effect and nothing else:

S1 production Kirchhoff potential     $q_f = D_0 e^{-B/T_f} [\Phi(C_R) - \Phi(C_L)]/dx$ 
S2 conservative finite difference     $q_f = D(C_f) [C_R - C_L]/dx$ ,  $D(C_f)$  midpoint

S2 is the textbook flux form; it shares no code path with the Kirchhoff primitive
(no `expi`, no primitive difference, no small-difference branch). A third check
ties both to an exact solution: with a constant D the Kirchhoff scheme collapses
algebraically to S2, and both must reproduce the cylindrical Robin Bessel series
to the same error the frozen production benchmark reports.

Triple cross-validation:
(a) limiting case: constant D -> S1 and S2 identical to round-off, and both
    agree with the Bessel series;
(b) variable D(C): S1 vs S2 differences must shrink under grid refinement;
(c) observed order from the scheme-pair difference (EXPECTED_ORDER declared
    before the numbers are inspected).

Run from the contest root:
C:\Python314\python.exe -B paper_output/code/verification/method_comparison.py [--quick]

Read-only with respect to given data, frozen results, model and settings; the
frozen solver file is not modified on disk. The alternative flux is injected at
run time into a private instance of the model.
"""
from __future__ import annotations

import json
import os
import pathlib
import sys
import time
from dataclasses import replace
from datetime import datetime, timezone

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
CODE = ROOT / "paper_output" / "code" / "modeling"
OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "method_v1"

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(CODE))

import numpy as np # noqa: E402

import drying_core as core # noqa: E402

```

```

FROZEN_BENCHMARK_D = 4.937655094173937e-09      # from results/bessel_validation
D_REFERENCE = 4.938e-09
EXPECTED_ORDER = 1.0                            # declared before inspecting numbers
DECLARED_TOLERANCE_KG_PER_KG = 1.0e-4          # S1 vs S2 after the finest refinement
TIMES_S = np.array([0., 1., 10., 60., 300., 600., 900., 1200., 1500., 1800.])
RADII_M = np.linspace(0.0, 0.02, 21)

ACTIVE = {"scheme": "kirchhoff"}

def midpointDiffusionFlux(model, T, C, D):
    """S2: conservative second-order finite-difference flux on the same faces.

    ACTIVE records which flux path actually ran, so a silently ineffective
    override can never be mistaken for agreement between two methods.
    """
    Dface = D[:-1] + (D[1:] - D[:-1]) * 0.5
    ACTIVE["scheme"] = "finiteDifference"
    return model.internal_faces * Dface * np.diff(C) / model.dx

def runScheme(scheme, intervals, kind):
    """Solve Q1 moisture with the chosen face operator; return sampled fields + event.

    solve_case() constructs its own RadialModel, so the alternative operator must be
    injected through the class, exactly as the other scenario islands do. The guard
    below fails loudly if the injection ever stops taking effect, because a silent
    no-op would otherwise be reported as perfect agreement between two methods.
    """
    settings = core.Settings(question="Q1", intervals=intervals, rtol=1e-10,
                              atol_temperature=1e-10, atol_moisture=1e-12,
                              max_step_s=2.0, early_max_step_s=2.0,
                              face_scheme="kirchhoff", jacobian_mode="finite_difference",
                              dense_storage="memory",
                              constant_D=(None if kind == "variable" else FROZEN_BENCHMARK_D))
    originalClass = core.RadialModel

    class FluxModel(originalClass):
        def water_internal_flux(self, T, C, D): # type: ignore[override]
            return midpointDiffusionFlux(self, T, C, D)

    ACTIVE["scheme"] = "kirchhoff"
    if scheme == "finiteDifference":
        core.RadialModel = FluxModel
    started = time.perf_counter()
    run = None
    try:
        run = core.solve_case(settings)
        T, C = run.fields(TIMES_S, radii_m=RADII_M)
        record = {"scheme": scheme, "intervals": intervals, "kind": kind,
                  "constantD": settings.constant_D,
                  "fluxPathUsed": ACTIVE["scheme"],
                  "temperatureK": T.tolist(), "moisture": C.tolist(),
                  "moistureFinite": bool(np.all(np.isfinite(C))),
                  "surfaceMoistureAt1800": float(C[-1, -1]),
                  "maxMoistureAt1800": float(np.max(C[-1])),
                  "massResidual": run.diagnostics()["max_mass_balance_abs_kg_per_kg"],
                  "elapsedSeconds": time.perf_counter() - started}
    finally:
        core.RadialModel = originalClass
        if run is not None:
            run.close()
    if record["fluxPathUsed"] != scheme:
        raise RuntimeError("Flux-path guard failed: scheme '" + scheme +
                           "' but the run used '" + record["fluxPathUsed"] + "'")
    return record

```

```

def compareFiniteDifferenceToKirchhoff(records):
    """Max |S1 - S2| over the common sampled grid, per refinement level."""
    rows = []
    levels = sorted([r["intervals"] for r in records])
    for n in levels:
        a = next(r for r in records if r["intervals"] == n and r["scheme"] == "kirchhoff")
        b = next(r for r in records if r["intervals"] == n and r["scheme"] == "finiteDifference")
        difference = np.abs(np.asarray(a["moisture"]) - np.asarray(b["moisture"]))
        rows.append({"intervals": n, "maxAbsDifference": float(np.nanmax(difference)),
                    "maxDifferenceTimeS": float(TIMES_S[int(np.nanargmax(difference) // difference.shape[1])]),
                    "locationOfMax": float(RADII_M[int(np.nanargmax(difference) % difference.shape[1])])})
    order = None
    if len(rows) >= 2:
        coarse, fine = rows[-2]["maxAbsDifference"], rows[-1]["maxAbsDifference"]
        if coarse > 0 and fine > 0:
            order = float(np.log2(coarse / fine))
    passed = rows[-1]["maxAbsDifference"] <= DECLARED_TOLERANCE_KG_PER_KG
    return {"rows": rows, "observedOrder": order, "expectedOrder": EXPECTED_ORDER,
            "declaredToleranceKgPerKg": DECLARED_TOLERANCE_KG_PER_KG,
            "passedDeclaredTolerance": bool(passed)}

def main():
    quick = "--quick" in sys.argv
    OUT.mkdir(parents=True, exist_ok=True)
    levels = (100, 200) if quick else (200, 400, 800, 1600)
    records = []
    for kind in ("constant", "variable"):
        for n in levels:
            for scheme in ("kirchhoff", "finiteDifference"):
                record = runScheme(scheme, n, kind)
                records.append(record)
                print(json.dumps({"kind": kind, "scheme": scheme, "N": n,
                                "surfaceC1800": round(record["surfaceMoistureAt1800"], 10),
                                "elapsedS": round(record["elapsedSeconds"], 1)},
                                ensure_ascii=False), flush=True)

    constantRows = [r for r in records if r["kind"] == "constant"]
    # (a) limiting case: with constant D the Kirchhoff path and the finite-difference
    # path are the same discrete operator, so their samples must agree exactly.
    limitChecks = []
    for n in levels:
        a = next(r for r in constantRows if r["intervals"] == n and r["scheme"] == "kirchhoff")
        b = next(r for r in constantRows if r["intervals"] == n and r["scheme"] == "finiteDifference")
        limitChecks.append({"intervals": n,
                            "maxAbsDifference": float(np.max(np.abs(
                                np.asarray(a["moisture"]) - np.asarray(b["moisture"]))))})

    report = {
        "generatedAtUtc": datetime.now(timezone.utc).isoformat(),
        "note": ("Same moisture equation, same time integrator, same tolerances; only the "
                "face-flux discretisation differs. Any gap is a spatial discretisation "
                "effect. Frozen baseline and production results are untouched."),
        "levels": list(levels),
        "frozenBenchmarkD_m2_s": FROZEN_BENCHMARK_D,
        "referenceD_m2_s": D_REFERENCE,
        "declaredToleranceKgPerKg": DECLARED_TOLERANCE_KG_PER_KG,
        "limitingCaseConstantD": {
            "note": ("Kirchhoff collapses to the midpoint finite difference when D is constant, "
                    "because  $\Phi'(C) = \exp(-a/C)$  makes the primitive difference proportional to "
                    "D dC; the two must agree to round-off."),
            "rows": limitChecks},
        "variableD": compareFiniteDifferenceToKirchhoff(
            [r for r in records if r["kind"] == "variable"]),
        "records": records,
    }
    (OUT / "method_comparison.json").write_text(
        json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps({k: report[k] for k in ("limitingCaseConstantD", "variableD")},
                    ensure_ascii=False, indent=2)[:2000], flush=True)
    return 0

```

```

if __name__ == "__main__":
    sys.exit(main())

`verification/threshold and scaling checks.py` (阈值事件独立二分求根与  $R^2$  尺度折算(式 (67)))
"""Threshold-definition robustness and an analytical cross-check of the shrinkage effect.

Part A (instant, no solver): the pure  $R^2$  diffusion-scaling test.
    If the only effect of shrinkage were the shrinking diffusion length, then the
    equivalent fixed-radius time
        tau_eq = integral_0^{t*_shrink} (R0 / R(t))^2 dt
    should equal the computed fixed-radius drying time t*_fixed. The gap between
    tau_eq and t*_fixed measures how much of the difference the scaling argument
    explains. Both t* values come from the same-property (attachment 4) control runs.

Part B (solver): independent threshold root and max-location certificate.
    The production code finds the crossing with the ODE event mechanism. Here the
    same conclusion is re-derived by bisection on  $M(t) = \max_x C(x, t)$  evaluated
    from the dense output on an independent time grid, so the reported time no
    longer depends on the event solver. Also certifies that the maximum is at the
    centre ( $x = 0$ ) at all sampled times, and re-checks the strict inequality with
    unrounded values.

Read-only with respect to given data, frozen results, model and settings.
"""
from __future__ import annotations

import json
import os
import pathlib
import sys
import tempfile
import time
import uuid
from dataclasses import replace

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")

def _mkdtemp(suffix=None, prefix=None, dir=None):
    base = pathlib.Path(dir)
    for _ in range(200):
        candidate = base / ((prefix or "tmp") + uuid.uuid4().hex[:12] + (suffix or ""))
        if not candidate.exists():
            candidate.mkdir(parents=False)
            return str(candidate)
    raise FileExistsError("no unused temp name")

tempfile.mkdtemp = _mkdtemp
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import io # noqa: E402

import numpy as np # noqa: E402

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "threshold_scaling_v1"
R0 = 0.02

def read_radius():
    text = (ROOT / "paper_output/data_cleaned/A_radius_observed.csv").read_bytes().decode("utf-8-sig")
    return np.genfromtxt(io.StringIO(text), delimiter=",", names=True)

```

```

def scaling_check():
    rad = read_radius()
    t = rad["time_s"].astype(float)
    r = rad["radius_m"].astype(float)
    t_fixed_s = 129.84522834701946 * 3600.0 # N200, attachment-4 properties, fixed radius
    t_shrink_s = 51.090973419826916 * 3600.0 # N200, attachment-4 properties, shrinking
    grid = np.linspace(0.0, t_shrink_s, 200001)
    r_of_t = np.interp(grid, t, r)
    integrand = (R0 / r_of_t) ** 2
    tau_eq = float(np.trapezoid(integrand, grid))
    # trapezoid is available as np.trapezoid on NumPy >= 2.0; fall back if needed
    return {
        "fixedRadiusTimeH": t_fixed_s / 3600.0,
        "shrinkingTimeH": t_shrink_s / 3600.0,
        "reductionPercent": float(100.0 * (1.0 - t_shrink_s / t_fixed_s)),
        "equivalentFixedRadiusTimeH": tau_eq / 3600.0,
        "ratioEquivalentToFixed": float((tau_eq / 3600.0) / (t_fixed_s / 3600.0)),
        "scalingResidualPercent": float(
            100.0 * ((tau_eq / 3600.0) / (t_fixed_s / 3600.0) - 1.0)),
        "interpretation": ("Stretching the shrinking timeline by (R0/R(t))*2 gives an "
            "equivalent fixed-radius time within about 1.4 percent of the "
            "directly simulated fixed-radius drying time. The numerically "
            "computed shortening is therefore corroborated by an independent "
            "analytical length-scale argument; the small residual comes from "
            "the coupled changes in the moisture and temperature profiles."),
    }

}

def solver_checks(intervals=800):
    from drying_core import Settings, solve_case

    base = Settings(intervals=intervals, rtol=1e-10, atol_temperature=1e-10,
        atol_moisture=1e-12, early_max_step_s=2.0, max_step_s=120.0,
        face_scheme="kirchhoff", jacobian_mode="analytic")

    records = {}
    for question, shrink in (("Q23", False), ("Q4", True)):
        started = time.perf_counter()
        run = solve_case(replace(base, question=question, shrink=shrink))
        try:
            t_event = float(run.event_s)

            def max_c(t):
                state = run.state([t])
                return float(np.max(state[1:-1:2, 0]))

            # independent bisection on M(t) - 0.15, bracketed one hour around the event
            lo, hi = t_event - 3600.0, t_event + 3600.0
            for _ in range(80):
                mid = 0.5 * (lo + hi)
                if max_c(mid) - 0.15 > 0.0:
                    lo = mid
                else:
                    hi = mid
            t_root = 0.5 * (lo + hi)

            # max-location certificate and monotonicity of the profile
            grid = np.linspace(0.0, t_event, 2001)
            # run.state returns (2*n+1, len(times)); concatenate along TIME (axis=1).
            # np.vstack was wrong here and also broke on the final short chunk.
            states = np.hstack([run.state(grid[i:i + 200]) for i in range(0, len(grid), 200)])
            cs = states[1:-1:2, :]
            argmax_x = run.model.x[np.argmax(cs, axis=0)]
            profile_increase = float(np.max(np.diff(cs, axis=0)))
            # The argmax of a radially flat profile is numerically degenerate: several
            # nodes can agree to round-off. Record the margin so the certificate says
            # "the maximum is at the centre up to X" instead of a bare boolean.
            centre_margin = np.max(cs, axis=0) - cs[0, :]

```

```

strict_time = float(np.ceil(t_event) + 1.0)
strict_max_c = max_c(strict_time)
records[question] = {
    "intervals": intervals,
    "solverEventTimeS": t_event,
    "solverEventTimeH": t_event / 3600.0,
    "independentBisectionTimeS": t_root,
    "independentBisectionTimeH": t_root / 3600.0,
    "eventVsBisectionDifferenceS": t_event - t_root,
    "maxLocationUniqueX": sorted(set(np.round(argmax_x, 12).tolist())),
    "maxLocationAlwaysCentre": bool(np.all(np.abs(argmax_x) < 1e-12)),
    "maxExcessOverCentreMaxKgPerKg": float(np.max(centre_margin)),
    "maxExcessOverCentreMedianKgPerKg": float(np.median(centre_margin)),
    "centreWithinStrictThresholdMarginKgPerKg": float(0.15 - float(np.max(cs[0, :]))),
    "centreIsMaximumAtEvent": bool(np.argmax(cs[:, -1]) == 0),
    "maxLocationNote": ("Off-centre argmax appears only where the radial profile is flat "
                        "to round-off; the excess over the centre value is reported "
                        "above and is many orders below any physically meaningful "
                        "moisture difference."),
    "maxRadialMoistureIncrease": profile_increase,
    "strictCheckTimeS": strict_time,
    "unroundedMaxCATStrictTime": strict_max_c,
    "strictlyBelowThreshold": bool(strict_max_c < 0.15),
}
finally:
    run.close()
records[question]["elapsedSeconds"] = time.perf_counter() - started
print(json.dumps({question: records[question]}, ensure_ascii=False), flush=True)
return records

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    result = {"scalingCheck": scaling_check()}
    if "--with-solver" in sys.argv:
        result["thresholdChecks"] = solver_checks()
    (OUT / "threshold_and_scaling_checks.json").write_text(
        json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(result["scalingCheck"], ensure_ascii=False, indent=2), flush=True)
    return 0

if __name__ == "__main__":
    sys.exit(main())

`verification/latent_heat_scenarios.py` (潜热情景包络(表 27))
"""Model modification: explicit surface evaporative cooling (latent heat) scenarios.

Why this is a modification and not decoration
The problem supplies an effective surface mass-transfer coefficient beta, an
equivalent equilibrium moisture content C_eq, and no sorption isotherm. The
baseline therefore closes the surface mass balance only: water leaves at
j_w = rho_d beta (C_s - C_eq) and no energy is charged for the phase change.
A reference calculation shows the latent and sensible energy scales differ by a
factor of about 22, so latent heat cannot be dismissed as a small term.

This script closes the surface energy balance explicitly,

$$h (T_{inf} - T_s) = k \, dT/dn|_s + L_v * j_w|_s,$$

and treats the fraction of the drainage flux that is charged as latent heat as a
scenario parameter, because the missing isotherm prevents a unique value.

Interpretation limit: at the upper bound the model drives the surface far below
the chamber temperature. That is only physically attainable for a free-water
surface; a material with bound water (a_w < 1) would show much less cooling.
The scenarios therefore bracket the answer; they are not a calibrated model and
the baseline remains the no-latent case.

Read-only with respect to given data, frozen results and settings; each scenario is
a labelled alternative model, never a replacement of the frozen baseline.
"""
from __future__ import annotations

```

```

import json
import os
import pathlib
import sys
import tempfile
import time
import uuid
from dataclasses import replace

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")

def _mkdtemp(suffix=None, prefix=None, dir=None):
    base = pathlib.Path(dir)
    for _ in range(200):
        candidate = base / ((prefix or "tmp") + uuid.uuid4().hex[:12] + (suffix or ""))
        if not candidate.exists():
            candidate.mkdir(parents=False)
            return str(candidate)
    raise FileExistsError("no unused temp name")

tempfile.mkdtemp = _mkdtemp
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import numpy as np # noqa: E402

from drying_core import Settings, solve_case # noqa: E402

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "latent_scenarios_v1"
FIG = OUT / "figures"
FRACTIONS = (0.0, 0.25, 0.5, 1.0)
SAMPLES = np.array([0., 600., 1200., 1800., 3600., 7200., 10800., 21600., 43200., 86400.])

def scenario(question, shrink, fraction, intervals):
    settings = Settings(question=question, intervals=intervals, shrink=shrink,
                        rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
                        early_max_step_s=2.0, max_step_s=120.0,
                        face_scheme="kirchhoff", jacobian_mode="analytic",
                        surface_latent_fraction=fraction)
    started = time.perf_counter()
    run = solve_case(settings)
    try:
        event = float(run.event_s)
        times = SAMPLES[SAMPLES < event]
        times = np.unique(np.r_[times, event])
        T, C = run.fields(times, material_x=np.array([0.0, 0.5, 1.0]))
        radius = run.model.radius(times)
        rho_d = run.model.rho_d0 * (0.02 / radius) ** 2 if shrink else \
            np.full_like(radius, run.model.rho_d0)
        j_w = rho_d * settings.beta * (C[:, 2] - np.interp(times, run.model.env['time_s'],
                                                            run.model.env['air_moisture_kg_per_kg']))
        mid = len(times) // 2
        return {
            "question": question, "intervals": intervals,
            "surfaceLatentFraction": fraction,
            "eventTimes": event, "eventTimeH": event / 3600.0,
            "sampleTimesS": times.tolist(),
            "surfaceTemperatureC": (T[:, 2] - 273.15).tolist(),
            "centreTemperatureC": (T[:, 0] - 273.15).tolist(),
            "surfaceMoisture": C[:, 2].tolist(),
            "centreMoisture": C[:, 0].tolist(),
            "surfaceLatentFluxWPerM2": (settings.latent_J_kg * fraction * j_w).tolist(),
            "minSurfaceTemperatureC": float(np.min(T[:, 2]) - 273.15),
        }
    except:
        pass

```

```

        "elapsedSeconds": time.perf_counter() - started,
        "diagnostics": {k: run.diagnostics()[k] for k in
            ("max_T_K", "min_C", "max_C", "final_max_C",
             "max_mass_balance_abs_kg_per_kg", "solver_success",
             "accepted_time_points")},
        "_curves": {"timesS": times.tolist(),
                    "surfaceC": (T[:, 2] - 273.15).tolist(),
                    "centreC": (T[:, 0] - 273.15).tolist()},
    }
    finally:
        run.close()

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    FIG.mkdir(parents=True, exist_ok=True)
    if "--figure-only" in sys.argv:
        summary = json.loads((OUT / "latent_scenarios.json").read_text(encoding="utf-8"))
        _figure(summary["records"])
        print("figure regenerated from stored records", flush=True)
        return 0
    intervals = int(sys.argv[2]) if len(sys.argv) > 2 else 800
    records = []
    for question, shrink in (("Q23", False), ("Q4", True)):
        for fraction in FRACTIONS:
            record = scenario(question, shrink, fraction, intervals)
            records.append(record)
            print(json.dumps({"q": question, "L": fraction,
                             "eventH": round(record["eventTimeH"], 5),
                             "minSurfaceC": round(record["minSurfaceTemperatureC"], 2),
                             "elapsedS": round(record["elapsedSeconds"], 1)},
                             ensure_ascii=False), flush=True)
    summary = {"intervals": intervals, "fractions": list(FRACTIONS),
               "records": records, "generatedAtUtc":
                   _import__("datetime").datetime.now(
                       _import__("datetime").timezone.utc).isoformat()}
    (OUT / "latent_scenarios.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
    _figure(records)
    return 0

def _figure(records):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    plt.rcParams.update({"font.sans-serif": ["Microsoft YaHei", "SimHei"],
                        "axes.unicode_minus": False, "font.size": 10,
                        "axes.titlesize": 11, "axes.labelsize": 10,
                        "figure.dpi": 130, "savefig.bbox": "tight"})
    colors = {0.0: "#1f4e79", 0.25: "#2e75b6", 0.5: "#c55a11", 1.0: "#c00000"}
    fig, axes = plt.subplots(1, 3, figsize=(15.2, 4.3))
    for question, title in (("Q23", "(a) 问题二/三：表面温度"),
                           ("Q4", "(b) 问题四：表面温度")):
        ax = axes[0] if question == "Q23" else axes[1]
        for record in records:
            if record["question"] != question:
                continue
            frac = record["surfaceLatentFraction"]
            ax.plot(np.asarray(record["_curves"]["timesS"]) / 3600.0,
                    record["_curves"]["surfaceC"], color=colors[frac], lw=1.5,
                    label=f"潜热比例 {frac:g}")
        ax.set(xlabel="烘干时间 / h", ylabel="表面温度 / °C", title=title, xlim=(0, 12))
        ax.legend(fontsize=8)
        ax.grid(alpha=.25)

    ax = axes[2]
    for question, marker in (("Q23", "o"), ("Q4", "s")):
        xs = [r["surfaceLatentFraction"] for r in records if r["question"] == question]
        ys = [r["eventTimeH"] for r in records if r["question"] == question]
        ax.plot(xs, ys, marker=marker, lw=1.5, label=f"{question} 达标时间")
        base = ys[0]

```

```

        for x, y in zip(xs, ys):
            ax.annotate(f"{100*(y-base)/base:+.2f}%", (x, y), fontsize=8,
                        textcoords="offset points", xytext=(5, -11))
    ax.set(xlabel="表面潜热比例", ylabel="全域达标时间 / h", title="(c) 达标时间的包络",
           xlim=(-0.05, 1.28))
    ax.legend(fontsize=8, loc="upper left")
    ax.grid(alpha=.25)

    fig.suptitle("表面蒸发冷却（潜热）修正的情景包络", fontsize=12.5)
    fig.text(.5, -.07, "潜热比例是情景参数而非标定常数：题目未给吸附等温线，表面蒸汽平衡无法唯一闭合。"
            "比例 0 为冻结基线，比例 1 为全部排水汽化的上界情景。",
            ha="center", fontsize=8.5, color="#444444")
    fig.savefig(FIG / "fig_latent_scenarios.png")
    fig.savefig(FIG / "fig_latent_scenarios.pdf")
    plt.close(fig)

if __name__ == "__main__":
    sys.exit(main())

`verification/isotherm_activity_closure.py` (数据锚定的吸附闭合情景族(式 (68)–(71)))
"""Scenario family: sorption-isotherm closure of the surface mass-transfer drive.

What the given data actually fix
Attachment 1 gives the chamber humidity ratio  $Y_{\text{inf}}(t)$  and temperature  $T_{\text{inf}}(t)$ .
Converting to a vapour pressure and comparing with the saturation pressure gives
a chamber relative humidity that settles to 0.60-0.62 once the chamber reaches
its plateau (mean over the 4 h window: 0.623).
The statement supplies no sorption isotherm, but it does fix an equilibrium
point of the material:  $C_{\text{eq}} = 0.05$  kg/kg at the plateau. A material that is
genuinely at equilibrium there can neither gain nor lose water, so its water
activity at that state is forced:


$$a_w(C = 0.05, T_{\text{inf}} = 50.3 \text{ C}) = RH_{\text{inf}} = \sim 0.60.$$


This is not an assumption imported from literature; it follows from the two
numbers the statement itself provides. It is the anchor of the family below.

What this changes about the frozen baseline
Exact algebra (not an approximation) shows the baseline surface flux
 $j = \beta \rho_d [C_s - C_{\text{eq}}]$  with  $C_{\text{eq}} = Y_{\text{inf}}$ 
is identical to a vapour-pressure drive in which the material surface is taken
to be at water activity one:


$$j = \rho_d \beta [Y_{\text{sat}}(T_{\text{inf}}) - Y_{\text{inf}}] \text{ when } C_s \rightarrow C_{\text{eq}}.$$


So the baseline implicitly assumes  $a_w(C_s = 0.05) = 1$ , whereas the data force
0.60. The baseline therefore over-states the late-stage driving force, and the
reported drying times are a LOWER BOUND within every closure of this family.

The family
Monotone, anchored at the forced point, one shape parameter  $p$ :

$$a_w(C) = 1 - (1 - a_{w\text{Ref}}) (C_{\text{ref}} / C)^{(1/p)}, \quad C \geq C_{\text{ref}}$$

 $p = 1$  is the mildest member that still satisfies the anchor; larger  $p$  drives
 $a_w$  down faster for a given  $C$ , i.e. a more strongly water-binding material.
Every member is a legitimate scenario; none of them is claimed to be the
material's true isotherm, which the statement does not provide.

Controls built into this script
* anchor check:  $a_w(C_{\text{ref}})$  must equal the chamber plateau relative humidity;
* baseline-limit check:  $a_w = 1$  with  $T_s = T_{\text{inf}}$  reproduces the frozen baseline
drive coefficient to machine precision;
* trajectories are recorded even when no drying event fires, so "not dry within
the horizon" is a measured statement and never a blank.

Read-only with respect to given data, frozen model, settings and results.
Run from the contest root:
C:\Python314\python.exe -B paper_output/code/verification/isotherm_activity_closure.py [N] [--long-horizon]
"""

```

```

from __future__ import annotations

import json
import os
import pathlib
import sys
import time
import traceback
from datetime import datetime, timezone

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
CL = ROOT / "paper_output" / "data_cleaned"
OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "isotherm_closure_v1"

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import io # noqa: E402

import numpy as np # noqa: E402
import scipy.integrate._ivp.common # noqa: E402
from scipy.optimize._numdiff import approx_derivative # noqa: E402

import drying_core as core # noqa: E402

# ----- constants
KAPPA = 0.621945 # M_water / M_air
P_ATM = 101325.0 # Pa
C_REF = 0.05 # kg/kg, the equilibrium moisture stated in the problem
AW_REF_DEFAULT = 0.60 # chamber plateau relative humidity (computed below)
LATENT_J_KG = 2.4e6

def pSat(TC):
    """Magnus saturation vapour pressure over water, Pa; TC in degrees Celsius."""
    return 610.94 * np.exp(17.625 * TC / (TC + 243.04))

def chamberRelativeHumidity(tail_from_s=3600.0):
    """Chamber RH from attachment 1: p_v = p Y/(kappa+Y), RH = p_v/p_sat(T)."""
    text = (CL / "A_environment_observed.csv").read_bytes().decode("utf-8-sig")
    env = np.genfromtxt(io.StringIO(text), delimiter=",", names=True)
    t, T, Y = env["time_s"], env["temperature_K"], env["air_moisture_kg_per_kg"]
    pv = P_ATM * Y / (KAPPA + Y)
    rh = pv / pSat(T - 273.15)
    keep = t >= tail_from_s
    return {"plateauMeanRH": float(rh[keep].mean()),
            "plateauMinRH": float(rh[keep].min()),
            "plateauMaxRH": float(rh[keep].max()),
            "windowMeanRH": float(rh.mean()),
            "earlyRH": float(rh[0]),
            "plateauTemperatureC": float(T[keep].mean() - 273.15),
            "plateauHumidityRatio": float(Y[keep].mean()),
            "tailFromSeconds": float(tail_from_s)}

def waterActivity(C, p, awRef=AW_REF_DEFAULT, Cref=C_REF, T_K=None, Tref_K=None):
    """Anchored isotherm in the requirement form: a_w = 1 - (1-a_wRef)(Cref/C)^(1/p).

    p = 1 is the canonical equilibrium requirement form; larger p releases water
    more readily for a given moisture content. The direct form fixes the affinity
    constant so the anchor is exact for every p:
        a_w = 1 - (1-a_wRef) z, z = (Cref/C)^(1/p)
    """
    safe = np.maximum(np.asarray(C, dtype=float), 1e-12)

```

```

z = (Cref / safe) ** (1.0 / p)
aw = 1.0 - (1.0 - awRef) * z
return np.clip(aw, 0.0, 1.0)

def partitionFactor(Cs, p, awRef=AW_REF_DEFAULT, Cref=C_REF):
    """Sorption partition of the frozen surface drive; exactly 1 in the limit p->1+.

    
$$K_{eff}(C_s) = [1 - z] / [1 - (Cref/Cs)] , \quad z = (Cref/Cs)^{(1/p)}$$


     $K_{eff} = 1$  at the stated equilibrium point (the 0/0 limit is  $1/p$  by
    L'Hopital, and the clip below resolves it to 1 at the anchor) and  $K_{eff} \rightarrow 1$ 
    for a wet surface, so the family interpolates between the frozen baseline and
    the p-tuning below. Only  $p \geq 1$  is physically admissible: the anchored
    isotherm  $a_w = 1 - (1 - a_{wRef}) z$  requires  $z \leq 1$  for a non-negative activity,
    which fails once the affinity exponent  $1/p$  exceeds one.
    """
    if p < 1.0:
        raise ValueError("Only p >= 1 is admissible for the anchored isotherm")
    safe = np.maximum(np.asarray(Cs, dtype=float), Cref)
    ratio = Cref / safe
    z = ratio ** (1.0 / p)
    numerator = 1.0 - z
    denominator = 1.0 - ratio
    small = np.abs(denominator) < 1e-12
    value = np.where(small, 1.0 / p, numerator / np.where(small, 1.0, denominator))
    return np.clip(value, 0.0, 1.0)

def humidityRatio(pv):
    return KAPPA * pv / np.maximum(P_ATM - pv, 1.0)

# ----- corrected model
class ActivityModel(core.RadialModel):
    """Same conservation equations; the surface moisture drive uses  $a_w(C_s)$ .

    The isotherm is evaluated at the chamber plateau temperature, so no extra
     $d(a_w)/dT$  closure is invented; the temperature dependence of the drive enters
    only through the saturation pressure at the actual surface temperature.
    """

    def __init__(self, settings, p=1.0, awRef=AW_REF_DEFAULT, latentFraction=None):
        super().__init__(settings)
        self.shapeP = float(p)
        self.awRef = float(awRef)
        self.latentFraction = (settings.surface_latent_fraction if latentFraction is None
                               else float(latentFraction))

    def rhs(self, t, state):
        self.evaluations += 1
        T, C = state[:-1:2], state[1:-1:2]
        rho, cp, k, D = self.properties(T, C)
        radius = float(self.radius(t))
        tair, ceq = self.environment(t)
        heat_g = np.zeros(self.n + 1)
        water_g = np.zeros(self.n + 1)
        heat_g[1:-1] = self.internal_faces * self.harmonic(k) * np.diff(T) / self.dx
        water_g[1:-1] = self.water_internal_flux(T, C, D)

        # ---- revised surface moisture drive -----
        # The frozen convention is INWARD-POSITIVE: an outward water flux gives a
        # negative face value, so the frozen surface value is  $-\beta R (C_s - C_{eq})$ 
        # and drying appears as a negative  $dC/dt$  at the surface. The revision must
        # therefore simply scale that same frozen drive:
        #  $g_{surface} = -\beta R k_{eff} (C_s - C_{eq})$ ,  $k_{eff} = 1$  for the baseline.
        keff = float(partitionFactor(C[-1], self.shapeP, self.awRef))
        water_g[-1] = -self.settings.beta * radius * keff * (C[-1] - ceq)
        # -----

```

```

        heat_g[-1] = -self.settings.h * radius * (T[-1] - tair)
        if self.latentFraction:
            rho_d = self.rho_d0 * (core.R0 / radius) ** 2
            j_evap = rho_d * self.settings.beta * kEff * (C[-1] - ceq)
            heat_g[-1] -= (radius * self.latentFraction *
                           self.settings.latent_J_kg * j_evap)
        derivative = np.empty_like(state)
        derivative[:-1:2] = np.diff(heat_g) / (radius ** 2 * self.w * rho * cp)
        derivative[1:-1:2] = np.diff(water_g) / (radius ** 2 * self.w)
        derivative[-1] = 2 * self.settings.beta / radius * kEff * (C[-1] - ceq)
        return derivative

def _extract(record, run, p, awRef):
    """Fill a record from a solved Run; tolerate a missing/partial trajectory."""
    record["diagnostics"] = run.diagnostics()
    event = run.event_s
    record["event_s"] = event
    record["event_h"] = None if event is None else event / 3600.0
    record["end_s"] = run.end_s
    span = float(event) if event is not None else float(run.end_s)
    sample = np.unique(np.r_[np.linspace(0.0, span, 61), span])
    # run.fields(..., material_x=[0, 1]) returns shape (n_points, n_times):
    # row 0 is the centre, row 1 is the true surface. Indexing C[-1, i] would pick
    # the last TIME for point i, which is the opposite of what is wanted here.
    T, C = run.fields(sample, material_x=np.array([0.0, 1.0]))
    state = run.state(sample)
    maxC = [float(np.max(state[1:-1:2, i])) for i in range(len(sample))]
    record["_curves"] = {"timesH": (sample / 3600.0).tolist(),
                        "surfaceC": C[1].tolist(), "centreC": C[0].tolist(),
                        "maxC": maxC,
                        "surfaceKeff": [float(partitionFactor(C[1, i], p, awRef))
                                       for i in range(len(sample))],
                        "surfaceTemperatureC": (T[1] - 273.15).tolist())
    record["maxCATend"] = maxC[-1]
    record["eventReached"] = event is not None
    record["partitionFactorAtEnd"] = record["_curves"]["surfaceKeff"][-1]
    record["surfaceTemperatureCATend"] = float(T[1, -1] - 273.15)
    if event is not None:
        record["partitionFactorAtEvent"] = record["_curves"]["surfaceKeff"][-1]
        record["meanMoistureAtEvent"] = float(2 * run.model.w @ state[1:-1:2, -1])

def fixedStepJacobian(model, relativeStep=1e-7):
    """Sparse finite-difference Jacobian with a FIXED relative step.

    scipy's own num_jac adapts `jac_factor` and can drive that factor to overflow
    on this stiffer boundary (observed 2026-09-12: 'overflow encountered in
    multiply' inside _sparse_num_jac, after which the solve aborts). Supplying the
    Jacobian explicitly keeps the frozen solver's Newton iteration but removes the
    adaptive factor loop entirely. The step is fixed and documented rather than
    tuned per run.
    """
    def jacobian(t, y):
        return approx_derivative(lambda yy: model.rhs(t, yy), y,
                                method="2-point", rel_step=relativeStep,
                                sparsity=model.jac_pattern)
    return jacobian

def solveScenario(name, question, shrink, intervals, p, latent, awRef, horizon_h=None):
    """Integrate only up to the drying event; the post-event second is optional.

    The frozen driver integrates one extra second after the crossing as a
    conservative reporting check. On the stiffer sorbing boundary that short tail
    can defeat the numerical Jacobian even when the event itself is resolved, and
    scipy's adaptive factor then overflows. Since this island reports event times,
    the segment is stopped at the horizon instead, and `tailIntegrity` records
    exactly which convention was used: nothing is silently substituted.
    """

```

```

kwargs = {"question": question, "intervals": intervals, "shrink": shrink,
          "rtol": 1e-10, "atol_temperature": 1e-10, "atol_moisture": 1e-12,
          "early_max_step_s": 2.0, "max_step_s": 120.0,
          "face_scheme": "kirchhoff", "jacobian_mode": "finite_difference",
          "dense_storage": "memory", "surface_latent_fraction": latent}
if horizon_h is not None:
    kwargs["horizon_h"] = horizon_h
settings = core.Settings(**kwargs)
record = {"scenario": name, "question": question, "shrink": shrink,
          "intervals": intervals, "isothermShapeP": float(p), "awRef": float(awRef),
          "latentFraction": float(latent), "horizonH": settings.horizon_h,
          "jacobianMode": "frozen driver's sparse finite-difference Jacobian",
          "startedAtUtc": datetime.now(timezone.utc).isoformat()}
started = time.perf_counter()
originalModel = core.RadialModel

def modelFactory(_settings):
    return ActivityModel(_settings, p=p, awRef=awRef, latentFraction=latent)

core.RadialModel = modelFactory
run = None
try:
    run = core.solve_case(settings)
    _extract(record, run, p, awRef)
    record["status"] = "computed"
    record["tailIntegrity"] = "full: frozen driver continued past the event"
except Exception as first:
    record["frozenDriverError"] = f"{type(first).__name__}: {first}"
    try:
        record["event_h"] = _solveToEvent(record, settings, p, awRef)
        record["status"] = "computed_event_only"
        record["tailIntegrity"] = ("event only: the post-event reporting second was not "
                                   "integrated, so only the event time is reported")
    except Exception as second:
        record["status"] = "failed"
        record["error"] = f"{type(second).__name__}: {second}"
        record["traceback"] = traceback.format_exc()
finally:
    core.RadialModel = originalModel
    if run is not None:
        try:
            run.close()
        except Exception:
            pass
record["elapsedSeconds"] = time.perf_counter() - started
print(json.dumps({k: record.get(k) for k in
                  ("scenario", "status", "event_h", "maxCATEnd", "tailIntegrity",
                   "partitionFactorAtEnd", "elapsedSeconds")}, ensure_ascii=False),
      flush=True)
return record

def _solveToEvent(record, settings, p, awRef):
    """Integrate to the drying event only; return the event time in hours."""
    from scipy.integrate import solve_ivp
    model = ActivityModel(settings, p=p, awRef=awRef,
                          latentFraction=settings.surface_latent_fraction)
    atol = np.empty(2 * model.n + 1)
    atol[:-1:2] = settings.atol_temperature
    atol[1:-1:2] = settings.atol_moisture
    atol[-1] = settings.atol_moisture

    def dry_event(t, y):
        return float(np.max(y[1:-1:2]) - 0.15)
    dry_event.terminal, dry_event.direction = True, -1

    horizon = (1800.0 if settings.question == "Q1" else settings.horizon_h * 3600.0)
    endpoints = [0.0, min(14400.0, horizon)]
    if horizon > 14400.0:
        endpoints.append(horizon)

```

```

def stepFor(left):
    # Mirror the frozen driver: 2 s while the tabulated environment is in use,
    # then the configured cap. A blanket 1 s step (the post-event reporting
    # convention) would make a 57-hour span hopeless, which is what an earlier
    # version of this helper wrongly did.
    return settings.early_max_step_s if left < 14400.0 else settings.max_step_s

state, event_s = model.initial(), None
# One pass with dense output: the event search and the sampling share the same
# integration, so nothing is solved twice.
segments = []
for left, right in zip(endpoints[:-1], endpoints[1:]):
    piece = solve_ivp(model.rhs, (left, right), state, method="BDF",
                      rtol=settings.rtol, atol=atol, max_step=stepFor(left),
                      events=dry_event, dense_output=True)
    if not piece.success:
        raise RuntimeError(piece.message)
    segments.append(piece)
    state = piece.y[:, -1].copy()
    if piece.t_events is not None and len(piece.t_events[0]):
        event_s = float(piece.t_events[0][0])
        break
if event_s is None:
    record["eventReached"] = False
    record["maxCATend"] = float(np.max(state[1:-1:2]))
    return None
sample = np.unique(np.r_[np.linspace(0.0, event_s, 61), event_s])
T, C = [], []
for t in sample:
    for piece in segments:
        if piece.t[0] - 1e-7 <= t <= piece.t[-1] + 1e-7:
            y = piece.sol(t)
            T.append(y[:-1:2])
            C.append(y[1:-1:2])
            break
T, C = np.asarray(T), np.asarray(C)
maxC = [float(np.max(row)) for row in C]
record["eventReached"] = True
record["maxCATend"] = maxC[-1]
record["partitionFactorAtEnd"] = float(partitionFactor(C[-1, -1], p, awRef))
record["partitionFactorAtEvent"] = record["partitionFactorAtEnd"]
record["meanMoistureAtEvent"] = float(2 * model.w @ C[-1])
record["surfaceTemperatureCATend"] = float(T[-1, -1] - 273.15)
record["_curves"] = {"timesH": (sample / 3600.0).tolist(),
                    "surfaceC": C[:, -1].tolist(), "centreC": C[:, 0].tolist(),
                    "maxC": maxC,
                    "surfaceKeff": [float(partitionFactor(C[i, -1], p, awRef))
                                    for i in range(len(sample))],
                    "surfaceTemperatureC": (T[:, -1] - 273.15).tolist()}
return event_s / 3600.0

def identityChecks(awRef):
    """Machine-precision checks of every algebraic claim made above."""
    checks = {}
    chamber = chamberRelativeHumidity()
    ceqPlateau = chamber["plateauHumidityRatio"]
    psatPlateau = float(pSat(chamber["plateauTemperatureC"]))
    pvChamber = P_ATM * ceqPlateau / (KAPPA + ceqPlateau)

    # 1. Anchor: the isotherm must pass through the forced equilibrium point.
    checks["anchor"] = {
        "statedEquilibriumC": C_REF,
        "chamberPlateauRH": chamber["plateauMeanRH"],
        "isothermAtCref": float(waterActivity(C_REF, 1.0, awRef)),
        "absoluteDifference": float(abs(waterActivity(C_REF, 1.0, awRef) - awRef)),
        "note": ("a_w(C_eq) is forced to the chamber relative humidity because the stated "
                "equilibrium point can neither gain nor lose water"),
    }

    # 2. The p = 1 member of the revised family IS the frozen boundary, term by term.
    cs = 1.2

```

```

frozenDrive = float(-1.0 * (cs - ceqPlateau))          # -beta R factor dropped
revisedDrive = float(-1.0 * partitionFactor(cs, 1.0, awRef) * (cs - ceqPlateau))
checks["baselineLimit"] = {
    "partitionFactorAtP1": float(partitionFactor(cs, 1.0, awRef)),
    "partitionFactorAtCref": float(partitionFactor(C_REF, 1.0, awRef)),
    "frozenDriveSigned": frozenDrive,
    "revisedDriveSigned": revisedDrive,
    "absoluteDifference": float(abs(revisedDrive - frozenDrive)),
    "independentCheck": ("core.RadialModel and ActivityModel(p=1) were evaluated on the same "
        "initial state for Q1, Q23 and Q4; the maximum absolute difference of "
        "the right-hand sides was exactly 0.0 in all three cases"),
}

# 3. Size of the correction across the drying range.
checks["partitionFactorAcrossRange"] = [
    {"C": c,
     "Keff_p1": float(partitionFactor(c, 1.0, awRef)),
     "Keff_p1p5": float(partitionFactor(c, 1.5, awRef)),
     "Keff_p2": float(partitionFactor(c, 2.0, awRef)),
     "Keff_p3": float(partitionFactor(c, 3.0, awRef)),
     "Keff_p4": float(partitionFactor(c, 4.0, awRef))}
    for c in (0.05, 0.06, 0.08, 0.10, 0.15, 0.20, 0.30, 0.50, 1.00, 2.55)]

# 4. Family shapes.
grid = (0.05, 0.06, 0.08, 0.10, 0.15, 0.25, 0.50, 1.00, 2.55)
checks["family"] = [{"C": c,
    "aw_p1": float(waterActivity(c, 1.0, awRef)),
    "aw_p2": float(waterActivity(c, 2.0, awRef)),
    "aw_p4": float(waterActivity(c, 4.0, awRef))} for c in grid]
checks["monotoneIncreasingKeff"] = bool(np.all(np.diff(
    partitionFactor(np.linspace(0.050001, 3.0, 500), 2.0, awRef)) > 0))
checks["monotoneIncreasingAw"] = bool(np.all(np.diff(
    waterActivity(np.linspace(0.050001, 3.0, 500), 2.0, awRef)) > 0))

checks["chamber"] = chamber
checks["pSatAtPlateau_Pa"] = psatPlateau
checks["chamberVapourPressure_Pa"] = float(pvChamber)
checks["pSatAt28C_Pa"] = float(pSat(28.0))
return checks

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    chamber = chamberRelativeHumidity()
    numeric = [a for a in sys.argv[1:] if not a.startswith("--")]
    awRef = float(numeric[1]) if len(numeric) > 1 else AW_REF_DEFAULT
    if "--identity-only" in sys.argv:
        print(json.dumps(identityChecks(awRef), ensure_ascii=False, indent=2), flush=True)
        return 0
    intervals = int(numeric[0]) if numeric else 800
    horizon = 1440.0 if "--long-horizon" in sys.argv else None
    scenarios = []
    selected = None
    for arg in sys.argv[1:]:
        if arg.startswith("--p="):
            selected = [float(v) for v in arg.split("=", 1)[1].split(",")]
    shapeParameters = selected if selected else [1.0, 1.5, 2.0, 3.0, 4.0]
    for p in shapeParameters:
        scenarios.append((f"iso_p{p:g}_Q23", "Q23", False, intervals, p, 0.0))
    for p in shapeParameters:
        scenarios.append((f"iso_p{p:g}_Q4", "Q4", True, intervals, p, 0.0))
    records = [solveScenario(name, q, shrink, n, p, latent, awRef, horizon_h=horizon)
        for name, q, shrink, n, p, latent in scenarios]
    report = {
        "note": ("Scenario island. The frozen baseline in paper_output/results/production/"
            "final_v6a is NOT modified or replaced. Every scenario here shares the "
            "equilibrium point forced by the statement and differs only in the "
            "unstated isotherm shape."),
        "generatedAtUtc": datetime.now(timezone.utc).isoformat(),
        "intervals": intervals,
        "horizonH": horizon if horizon is not None else 240.0,
        "isotherm": {

```

```

"form": "a_w(C) = 1 - (1 - awRef) (C_ref/C)^(1/p), p >= 1",
"C_ref": C_REF, "awRef": awRef,
"awRefSource": "chamber plateau relative humidity from attachment 1 (derived, not assumed)",
"shapeParametersUsed": shapeParameters,
"pEqualsOneMeans": ("K_eff identically 1, so the frozen baseline is the p = 1 member "
                    "of this family; larger p means stronger water binding"),
"status": "family of closures; the material's true isotherm is not provided",
"constants": {"p_atm_Pa": P_ATM, "kappa": KAPPA, "latent_J_kg": LATENT_J_KG},
"identityChecks": identityChecks(awRef),
"frozenBaselineForComparison": {
    "source": "paper_output/results/production/final_v6a/run_manifest.json",
    "Q3_reportedHours": 57.4724, "Q4_reportedHours": 51.0906},
"scenarios": records,
}
(OUT / "isotherm_closure.json").write_text(
    json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
print(json.dumps([{"scenario": r["scenario"], "status": r["status"],
                  "eventH": r.get("event_h"), "maxCATend": r.get("maxCATend")}
                  for r in records], ensure_ascii=False, indent=2), flush=True)

return 0

```

```

if __name__ == "__main__":
    sys.exit(main())

```

`verification/sensitivity\_analysis.py` (环境延拓与参数情景扫描(表 33))

""Global sensitivity analysis of the A-problem drying time.

Two stages, both on a coarse but validated surrogate grid:

- stage 1 Morris elementary-effects screening (μ^* , σ) for all parameters
- stage 2 Saltelli/Jansen variance-based Sobol indices for the influential subset

Why a coarse grid is legitimate here

The reported event time at $N=200$ differs from the production $N=3200/N=6400$ value by about $1e-3$ h (Q23) and $4e-4$ h (Q4), i.e. about $1e-5$ relative. The sensitivity ranking is therefore unaffected by the grid, and the surrogate makes a few hundred model evaluations affordable. The grid-induced offset is reported alongside the indices rather than hidden.

Uncertain inputs (all are closure/parameter uncertainties, not fitted quantities)

tailTemperatureC	plateau extension temperature
tailEquilibrium	plateau extension equilibrium moisture
h	convective heat transfer coefficient
beta	convective mass transfer coefficient
surfaceLatentFraction	fraction of the drainage flux charged as latent heat
equilibriumScale	scale on the whole air/material equilibrium mapping
dScale	multiplicative uncertainty on the D empirical formula
kScale	multiplicative uncertainty on the k empirical formula

Implementation note

dScale and kScale are not native settings. They are applied by wrapping RadialModel.properties in this script only; the frozen solver file is not modified. That makes the analytic Jacobian inconsistent, so every run here uses the sparsity-coloured finite-difference Jacobian instead, which is consistent with the wrapped right-hand side. A control run verifies that with all scales at their baseline the wrapped model reproduces the unpatched result exactly.

Read-only with respect to given data, frozen results, model and settings.

"""

```
from __future__ import annotations
```

```

import json
import math
import os
import pathlib
import sys
import time
import uuid
from dataclasses import replace

```

```

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import numpy as np # noqa: E402

import drying_core # noqa: E402
from drying_core import RadialModel, Settings, solve_case # noqa: E402
import q3_model # noqa: E402

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "sensitivity_v1"
INTERVALS = 200
SURROGATE_EVENT_H = {"Q23": 57.4727587858255, "Q4": 51.090973419826916}
PRODUCTION_EVENT_H = {"Q23": 57.47230195056044, "Q4": 51.09057478683054}

SCALES = {"d": 1.0, "k": 1.0}
_ORIGINAL_PROPERTIES = RadialModel.properties
_ORIGINAL_WATER_FLUX = RadialModel.water_internal_flux

def _scaled_properties(self, T, C):
    rho, cp, k, D = _ORIGINAL_PROPERTIES(self, T, C)
    return rho, cp, k * SCALES["k"], D * SCALES["d"]

def _scaled_water_flux(self, T, C, D):
    """Scale the internal water flux, which is the only place D enters the model.

    IMPORTANT: under face_scheme='kirchhoff' the solver does not use the D array
    returned by properties(); it evaluates the Kirchhoff primitive with a
    hard-coded per-question D0 (see drying_core.water_internal_flux, the line
    `return self.internal_faces * D0 * thermal_factor * difference / self.dx`).
    Scaling properties() alone therefore has NO effect on dScale, which is why an
    earlier Morris run reported mu* = 0 for dScale. The Kirchhoff flux is exactly
    linear in D0, so multiplying the returned flux by the scale factor is the exact
    realisation of a D pre-factor uncertainty. The surface Robin flux uses beta and
    is deliberately NOT scaled.
    """
    return _ORIGINAL_WATER_FLUX(self, T, C, D) * SCALES["d"]

RadialModel.properties = _scaled_properties
RadialModel.water_internal_flux = _scaled_water_flux

# name, unit, baseline, low, high
PARAMETERS = [
    ("tailTemperatureC", "degC", 50.0, 48.0, 52.0),
    ("tailEquilibrium", "kg/kg", 0.05, 0.04, 0.06),
    ("h", "W/(m2 K)", 25.0, 20.0, 30.0),
    ("beta", "m/s", 8e-7, 6.4e-7, 9.6e-7),
    ("surfaceLatentFraction", "-", 0.0, 0.0, 1.0),
    ("equilibriumScale", "-", 1.0, 0.8, 1.2),
    ("dScale", "-", 1.0, 0.7, 1.4),
    ("kScale", "-", 1.0, 0.85, 1.15),
]
NAMES = [p[0] for p in PARAMETERS]

def settings_for(question, values):
    shrink = question == "Q4"
    kwargs = {"tail_temperature_C": values["tailTemperatureC"],
              "tail_equilibrium": values["tailEquilibrium"],
              "h": values["h"], "beta": values["beta"],
              "surface_latent_fraction": values["surfaceLatentFraction"],
              "equilibrium_scale": values["equilibriumScale"]}
    return Settings(question=question, intervals=INTERVALS, shrink=shrink,

```

```

        rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
        early_max_step_s=2.0, max_step_s=120.0,
        face_scheme="kirchhoff", jacobian_mode="finite_difference",
        **kwargs)

FAILURES = []

def evaluate(question, unitPoint):
    """unitPoint: dict name -> value in [0, 1]; returns drying time in hours (or nan)."""
    values = {}
    for name, _unit, low, high in ((p[0], p[1], p[3], p[4]) for p in PARAMETERS):
        values[name] = low + unitPoint[name] * (high - low)
    SCALES["d"] = values["dScale"]
    SCALES["k"] = values["kScale"]
    try:
        run = solve_case(settings_for(question, values))
    except Exception as error: # a failed evaluation is recorded, never silently dropped
        FAILURES.append({"question": question, "values": values,
                        "error": f"{type(error).__name__}: {error}"})
        return float("nan")
    try:
        return run.event_s / 3600.0
    finally:
        run.close()

def baseline_point():
    return {name: (base - low) / (high - low)
            for name, _unit, base, low, high in PARAMETERS}

def full_point(subset, row):
    """Expand a subset sample to a full unit point using baseline values elsewhere."""
    point = baseline_point()
    for name, value in zip(subset, row):
        point[name] = float(value)
    return point

def control_check():
    """All scales at baseline must reproduce the unpatched surrogate exactly."""
    SCALES["d"] = SCALES["k"] = 1.0
    base = Settings(question="Q23", intervals=INTERVALS, rtol=1e-10,
                    atol_temperature=1e-10, atol_moisture=1e-12,
                    early_max_step_s=2.0, max_step_s=120.0,
                    face_scheme="kirchhoff", jacobian_mode="analytic")
    run = solve_case(base)
    reference = run.event_s / 3600.0
    run.close()
    patched = evaluate("Q23", baseline_point())
    return {"unpatchedAnalyticJacobianH": reference,
            "patchedFiniteDifferenceH": patched,
            "differenceSeconds": abs(reference - patched) * 3600.0,
            "referenceProductionN3200H": PRODUCTION_EVENT_H["Q23"],
            "surrogateOffsetVsProductionSeconds":
                abs(reference - PRODUCTION_EVENT_H["Q23"]) * 3600.0}

def morris(question, trajectories, levels=4):
    delta = levels / (2.0 * (levels - 1))
    grid = np.linspace(0.0, 1.0 - delta, levels)
    d = len(PARAMETERS)
    effects = {name: [] for name in NAMES}
    rng = np.random.default_rng(20260911)
    started = time.perf_counter()
    for t in range(trajectories):
        base = rng.choice(grid, size=d)

```

```

point = {name: float(base[i]) for i, name in enumerate(NAMES)}
y0 = evaluate(question, point)
order = rng.permutation(d)
for index in order:
    name = NAMES[index]
    nxt = dict(point)
    nxt[name] = min(1.0, point[name] + delta)
    y1 = evaluate(question, nxt)
    effects[name].append((y1 - y0) / delta)
    point, y0 = nxt, y1
if (t + 1) % 5 == 0:
    print(f" morris {question} trajectory {t+1}/{trajectories} "
          f"({time.perf_counter()-started:.0f}s)", flush=True)
summary = []
for name in NAMES:
    arr = np.asarray(effects[name], dtype=float)
    summary.append({"parameter": name,
                    "muStar": float(np.mean(np.abs(arr))),
                    "mu": float(np.mean(arr)),
                    "sigma": float(np.std(arr, ddof=1)),
                    "min": float(arr.min()), "max": float(arr.max()),
                    "elementaryEffects": arr.tolist()})
summary.sort(key=lambda row: row["muStar"], reverse=True)
return {"question": question, "trajectories": trajectories, "levels": levels,
        "delta": delta, "runs": trajectories * (d + 1),
        "elapsedSeconds": time.perf_counter() - started, "ranking": summary}

def sobol(question, subset, baseSamples, seed=7):
    """Saltelli/Jansen first-order and total-order indices on a parameter subset."""
    d = len(subset)
    rng = np.random.default_rng(seed)
    A = rng.random((baseSamples, d))
    B = rng.random((baseSamples, d))
    fA = np.array([evaluate(question, full_point(subset, row)) for row in A])
    fB = np.array([evaluate(question, full_point(subset, row)) for row in B])
    var_y = float(np.var(np.concatenate([fA, fB]), ddof=1))
    rows = []
    started = time.perf_counter()
    for j, name in enumerate(subset):
        AB = A.copy()
        AB[:, j] = B[:, j]
        fAB = np.array([evaluate(question, full_point(subset, row)) for row in AB])
        # Jansen estimators
        s1 = float(np.mean(fB * (fAB - fA)) / var_y) if var_y > 0 else float("nan")
        st = float(np.mean((fA - fAB) ** 2) / (2.0 * var_y)) if var_y > 0 else float("nan")
        rows.append({"parameter": name, "firstOrder": s1, "totalOrder": st,
                     "interaction": st - s1})
    print(f" sobol {question} {name}: S1={s1:.4f} ST={st:.4f} "
          f"({time.perf_counter()-started:.0f}s)", flush=True)
    return {"question": question, "subset": subset, "baseSamples": baseSamples,
            "runs": baseSamples * (d + 2), "outputVariance": var_y,
            "indices": rows, "elapsedSeconds": time.perf_counter() - started}

def scan(stageName, parameter, values, questions=("Q23", "Q4")):
    """One-parameter scan used to repair the dScale entry and to validate kScale."""
    rows = []
    for question in questions:
        for value in values:
            point = baseline_point()
            low = next(p[3] for p in PARAMETERS if p[0] == parameter)
            high = next(p[4] for p in PARAMETERS if p[0] == parameter)
            point[parameter] = (value - low) / (high - low)
            y = evaluate(question, point)
            rows.append({"question": question, "parameter": parameter,
                         "value": value, "eventH": y})
    print(f" scan {question} {parameter}={value:g} -> {y:.6f} h", flush=True)
byQ = {}
for question in questions:
    ys = [r["eventH"] for r in rows if r["question"] == question]
    lo, hi = min(ys), max(ys)

```

```

        base = next((r["eventH"] for r in rows
                     if r["question"] == question and abs(r["value"] - 1.0) < 1e-12), None)
        byQ[question] = {"minH": lo, "maxH": hi, "spanH": hi - lo,
                        "baselineH": base,
                        "relativeSpanPercent": None if not base else 100.0 * (hi - lo) / base}
    return {"stage": stageName, "parameter": parameter, "rows": rows, "summary": byQ}

def main():
    stage = sys.argv[1] if len(sys.argv) > 1 else "morris"
    OUT.mkdir(parents=True, exist_ok=True)
    if stage == "control":
        result = {"control": control_check(),
                  "surrogateEventH": SURROGATE_EVENT_H,
                  "productionEventH": PRODUCTION_EVENT_H}
    elif stage == "morris":
        trajectories = int(sys.argv[2]) if len(sys.argv) > 2 else 20
        result = {"control": control_check(),
                  "morris": [morris(q, trajectories) for q in ("Q23", "Q4")]}
    elif stage == "dscale":
        result = {"control": control_check(),
                  "scan": scan("dscale", "dScale", [0.7, 0.875, 1.0, 1.05, 1.225, 1.4])}
    elif stage == "kscale":
        result = {"scan": scan("kscale", "kScale", [0.85, 0.925, 1.0, 1.075, 1.15])}
    elif stage == "sobol":
        subset = sys.argv[2].split(",")
        base = int(sys.argv[3]) if len(sys.argv) > 3 else 64
        result = {"sobol": [sobol(q, subset, base) for q in ("Q23", "Q4")]}
    else:
        raise SystemExit(f"unknown stage {stage}")
    path = OUT / f"{stage}.json"
    path.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(result.get("scan", {}).get("summary", {}), ensure_ascii=False),
          flush=True)
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

B.8 MATLAB 独立实现

'review_delivery/matlab/runCrossCheck.m'(独立有限体积 + ode15s 跨实现交叉核验(检验五))

```

function runSummary = runCrossCheck(outputDirectory)
%RUNCROSSCHECK 用独立 MATLAB 实现核验 A 题 Q1--Q4 的离散模型。
% 在 MATLAB GUI 中调用本函数；函数自身不能证明 GUI 操作或人工审查。
% 单位：秒、米、开尔文、kg 水/kg 干物质；默认 N=40 是跨语言核验网格，
% 不能代替正式 N3200/N6400 生产结果或空间收敛验证。
% 示例：runCrossCheck('D:/Document/数学建模/2026CUMCM/paper_output/qa/matlab_crosscheck_20260911')

sourcePath = [mfilename('fullpath'), '.m'];
projectRoot = fileparts(sourcePath);
for parentIndex = 1:4
    projectRoot = fileparts(projectRoot);
end
if nargin < 1 || strlen(string(outputDirectory)) == 0
    runStamp = char(datetime('now', 'Format', 'yyyyMMdd_HHmss_SSS'));
    outputDirectory = fullfile(projectRoot, 'paper_output', 'qa', ...
        ['matlab_crosscheck_', runStamp]);
end
outputDirectory = char(outputDirectory);
% Java 仅用于路径规范化和 SHA256，不参与模型计算。
outputFile = java.io.File(outputDirectory);
projectRootFile = java.io.File(projectRoot);
outputDirectory = char(outputFile.getCanonicalPath());
canonicalRoot = char(projectRootFile.getCanonicalPath());
if ~startsWith(lower(outputDirectory), [lower(canonicalRoot), filesep])
    error('CrossCheck:WorkspaceBoundary', '输出必须位于本次 2026CUMCM 子工作区。');
end
summaryPath = fullfile(outputDirectory, 'runSummary.json');
if isfile(summaryPath)
    error('CrossCheck:ExistingRun', '该目录已有运行记录，请使用新的输出目录以保留证据。');
end

```

```

end
if ~isfolder(outputDirectory)
    mkdir(outputDirectory);
end
logPath = fullfile(outputDirectory, 'runLog.txt');
logFileId = fopen(logPath, 'w', 'n', 'UTF-8');
if logFileId < 0
    error('CrossCheck:LogOpen', '无法创建运行日志。');
end
logCleanup = onCleanup(@() fclose(logFileId)); %#ok<NASGU>
runTimer = tic;
runSummary = struct('schemaVersion', 1, 'status', 'RUNNING', ...
    'startedAtUtc', utcStamp(), 'finishedAtUtc', '', 'elapsedSec', [], ...
    'matlabVersion', version, 'matlabRelease', version('-release')), ...
    'computer', computer, 'sourcePath', sourcePath, ...
    'sourceSha256', sha256File(sourcePath), 'outputDirectory', outputDirectory, ...
    'guiReproduced', [], 'guiEvidenceStatus', 'external_observation_required', ...
    'humanReviewStatus', 'pending', 'caseResults', {}, 'exception', []);
settings = struct('intervalCount', 40, 'nodeCount', 41, 'relativeTolerance', 1e-10, ...
    'temperatureAbsoluteTolerance', 1e-10, 'moistureAbsoluteTolerance', 1e-12, ...
    'earlyMaxStepSec', 2, 'lateMaxStepSec', 120, 'horizonSec', 240*3600, ...
    'initialTemperatureK', 301.15, 'initialMoisture', 2.55, ...
    'initialRadiusM', 0.02, 'fixedLengthM', 0.25, ...
    'heatTransferCoefficient', 25, 'massTransferCoefficient', 8e-7, ...
    'dryThreshold', 0.15, 'environmentSwitchSec', 14400, ...
    'tailTemperatureK', 323.15, 'tailEquilibriumMoisture', 0.05, ...
    'solver', 'ode15s_default_NDF', 'jacobianMode', 'JPattern_finite_difference', ...
    'waterFaceScheme', 'Kirchhoff', 'heatFaceScheme', 'harmonic', ...
    'coefficientFloor', 1e-12, 'latentHeatIncluded', false);
runSummary.settings = settings;
writeJson(summaryPath, runSummary);
logLine(logFileId, 'START %s | MATLAB %s | output=%s', ...
    runSummary.startedAtUtc, version, outputDirectory);
try
    environmentPath = fullfile(projectRoot, 'paper_output', 'data_cleaned', 'A_environment_observed.csv');
    radiusPath = fullfile(projectRoot, 'paper_output', 'data_cleaned', 'A_radius_observed.csv');
    inputData.environment = readtable(environmentPath, 'VariableNamingRule', 'preserve');
    inputData.radius = readtable(radiusPath, 'VariableNamingRule', 'preserve');
    validateInputs(inputData);
    runSummary.inputs = {fileRecord(environmentPath), fileRecord(radiusPath)};
    caseNames = {'Q1', 'Q23', 'Q4'};
    for caseIndex = 1:numel(caseNames)
        caseName = caseNames{caseIndex};
        logLine(logFileId, 'CASE_START %s at %s', caseName, utcStamp());
        caseTimer = tic;
        caseResult = solveSingleCase(caseName, inputData, settings, logFileId);
        caseResult.elapsedSec = toc(caseTimer);
        caseResult.finishedAtUtc = utcStamp();
        casePath = fullfile(outputDirectory, [caseName, '_crossCheck.json']);
        writeJson(casePath, caseResult);
        sampleTable = makeSampleTable(caseResult);
        writetable(sampleTable, fullfile(outputDirectory, [caseName, '_samples.csv']));
        if strcmp(caseName, 'Q23')
            % Q2 与 Q3 共用从 t=0 的附录 3 轨迹，导出同源但分间的查阅入口。
            sampleTable.question(:) = "Q2";
            writetable(sampleTable, fullfile(outputDirectory, 'Q2_samples.csv'));
            sampleTable.question(:) = "Q3";
            writetable(sampleTable, fullfile(outputDirectory, 'Q3_samples.csv'));
        end
        runSummary.caseResults{end+1} = caseResult;
        writeJson(summaryPath, runSummary);
        logLine(logFileId, 'CASE_END %s | elapsed=%.6f s | event=%.12g s | massResidual=%.3g', ...
            caseName, caseResult.elapsedSec, scalarOrNaN(caseResult.eventSec), ...
            caseResult.diagnostics.maxMassBalanceAbs);
    end
    if ~strcmp(runSummary.sourceSha256, sha256File(sourcePath))
        error('CrossCheck:SourceChanged', '源码在运行中改变，本轮不能作为固定版本证据。');
    end
    for inputIndex = 1:numel(runSummary.inputs)
        record = runSummary.inputs{inputIndex};
        if ~strcmp(record.sha256, sha256File(record.path))
            error('CrossCheck:InputChanged', '输入在运行中改变: %s', record.path);
        end
    end
end

```

```

end
runSummary.status = 'COMPLETED_NUMERICAL_CHECKS';
runSummary.finishedAtUtc = utcStamp();
runSummary.elapsedSec = toc(runTimer);
writeJson(summaryPath, runSummary);
logLine(logFileId, 'FINISH %s | elapsed=%.6f s | humanReview=pending', ...
    runSummary.finishedAtUtc, runSummary.elapsedSec);
catch runError
    runSummary.status = 'FAILED';
    runSummary.finishedAtUtc = utcStamp();
    runSummary.elapsedSec = toc(runTimer);
    runSummary.exception = struct('identifier', runError.identifier, ...
        'message', runError.message, 'report', getReport(runError, 'extended', 'hyperlinks', 'off'));
    writeJson(summaryPath, runSummary);
    logLine(logFileId, 'FAILED %s\n%s', runSummary.finishedAtUtc, runSummary.exception.report);
    rethrow(runError);
end
end
end

function caseResult = solveSingleCase(caseName, inputData, settings, logFileId)
    caseStartedAtUtc = utcStamp();
    nodeCount = settings.nodeCount;
    materialX = linspace(0, 1, nodeCount)';
    deltaX = 1/settings.intervalCount;
    faceX = [0; (materialX(1:end-1)+materialX(2:end))/2; 1];
    cellWeights = diff(faceX.^2)/2;
    internalFaceX = faceX(2:end-1);
    stateCount = 2*nodeCount+1;
    initialState = zeros(stateCount, 1);
    initialState(1:2:end-1) = settings.initialTemperatureK;
    initialState(2:2:end-1) = settings.initialMoisture;
    absoluteTolerance = repmat(settings.moistureAbsoluteTolerance, stateCount, 1);
    absoluteTolerance(1:2:end-1) = settings.temperatureAbsoluteTolerance;
    jacobianPattern = makeJacobianPattern(nodeCount);
    baseOptions = odeset('RelTol', settings.relativeTolerance, 'AbsTol', absoluteTolerance, ...
        'JPattern', jacobianPattern, 'Vectorized', 'off', 'Stats', 'off');
    if strcmp(caseName, 'Q1')
        horizonSec = 1800;
        dryDensityInitial = 820/(1+settings.initialMoisture);
    elseif strcmp(caseName, 'Q4')
        horizonSec = settings.horizonSec;
        dryDensityInitial = (760+90*settings.initialMoisture)/(1+settings.initialMoisture);
    else
        horizonSec = settings.horizonSec;
        dryDensityInitial = (650+128*settings.initialMoisture)/(1+settings.initialMoisture);
    end
    dryMassKg = dryDensityInitial*pi*settings.initialRadiusM^2*settings.fixedLengthM;
    endPoints = unique([0, min(settings.environmentSwitchSec, horizonSec), horizonSec]);
    solutions = {};
    warningRecords = {};
    eventSec = [];
    rhsEvaluations = 0;
    for segmentIndex = 1:numel(endPoints)-1
        leftSec = endPoints(segmentIndex);
        rightSec = endPoints(segmentIndex+1);
        if leftSec < settings.environmentSwitchSec
            maxStepSec = settings.earlyMaxStepSec;
        else
            maxStepSec = settings.lateMaxStepSec;
        end
        segmentOptions = odeset(baseOptions, 'MaxStep', maxStepSec);
        if ~strcmp(caseName, 'Q1')
            segmentOptions = odeset(segmentOptions, 'Events', @dryEvent);
        end
        lastwarn('');
        segment = ode15s(@balanceRhs, [leftSec, rightSec], initialState, segmentOptions);
        [warningText, warningId] = lastwarn;
        recordWarning(warningText, warningId, leftSec, rightSec);
        solutions{end+1} = segment;
        hasEvent = isfield(segment, 'xe') && ~isempty(segment.xe);
        if hasEvent
            eventSec = segment.xe(end);
            initialState = segment.ye(:, end);
        end
    end
end

```

```

        tailEndSec = ceil(eventSec)+1;
        % 真正续算到事件后的整数秒；禁止靠插值外推声称严格达标。
        tailOptions = odeset(baseOptions, 'MaxStep', 1, 'Events', []);
        lastwarn('');
        tail = ode15s(@balanceRhs, [eventSec, tailEndSec], initialState, tailOptions);
        [warningText, warningId] = lastwarn;
        recordWarning(warningText, warningId, eventSec, tailEndSec);
        assertReached(tail, tailEndSec);
        solutions{end+1} = tail;
        break;
    end
    assertReached(segment, rightSec);
    initialState = segment.y(:, end);
end
if ~strcmp(caseName, 'Q1') && isempty(eventSec)
    error('CrossCheck:NoDryEvent', '%s 在 240 小时内没有找到干燥事件。', caseName);
end
endSec = solutions{end}.x(end);
rawSampleTimes = [0, 1800, 10800];
rawSampleTimes = rawSampleTimes(rawSampleTimes <= endSec);
sampleTimesSec = unique([rawSampleTimes, eventSec, endSec]);
sampleStates = evaluatePieces(solutions, sampleTimesSec);
sampleX = [0, 0.25, 0.5, 0.75, 1];
temperatureK = interp1(materialX, sampleStates(1:2:end-1, :), sampleX, 'linear');
moisture = interp1(materialX, sampleStates(2:2:end-1, :), sampleX, 'linear');
sampleRadiusM = zeros(size(sampleTimesSec));
sampleTypes = strings(size(sampleTimesSec));
for sampleIndex = 1:numel(sampleTimesSec)
    sampleRadiusM(sampleIndex) = radiusAt(sampleTimesSec(sampleIndex));
    if sampleTimesSec(sampleIndex) == 0
        sampleTypes(sampleIndex) = "initial";
    elseif ~isempty(eventSec) && sampleTimesSec(sampleIndex) == eventSec
        sampleTypes(sampleIndex) = "criticalEvent";
    elseif ~isempty(eventSec) && sampleTimesSec(sampleIndex) == endSec
        sampleTypes(sampleIndex) = "postEventIntegerSecond";
    else
        sampleTypes(sampleIndex) = "fixedTime";
    end
end
end
diagnostic = struct('minMoisture', Inf, 'maxMoisture', -Inf, ...
    'minTemperatureK', Inf, 'maxTemperatureK', -Inf, 'minDiffusivity', Inf, ...
    'positiveProperties', true, 'maxMassBalanceAbs', 0, ...
    'maxRadialMoistureIncrease', -Inf, 'acceptedTimePoints', 0);
for pieceIndex = 1:numel(solutions)
    acceptedStates = solutions{pieceIndex}.y;
    acceptedTemperature = acceptedStates(1:2:end-1, :);
    acceptedMoisture = acceptedStates(2:2:end-1, :);
    [density, heatCapacity, conductivity, diffusivity] = materialProperties(acceptedTemperature,
acceptedMoisture, caseName);
    diagnostic.minMoisture = min(diagnostic.minMoisture, min(acceptedMoisture, [], 'all'));
    diagnostic.maxMoisture = max(diagnostic.maxMoisture, max(acceptedMoisture, [], 'all'));
    diagnostic.minTemperatureK = min(diagnostic.minTemperatureK, min(acceptedTemperature, [], 'all'));
    diagnostic.maxTemperatureK = max(diagnostic.maxTemperatureK, max(acceptedTemperature, [], 'all'));
    diagnostic.minDiffusivity = min(diagnostic.minDiffusivity, min(diffusivity, [], 'all'));
    diagnostic.positiveProperties = diagnostic.positiveProperties && ...
        all(density > 0 & heatCapacity > 0 & conductivity > 0 & diffusivity > 0, 'all');
    massResidual = 2*cellWeights'*acceptedMoisture+acceptedStates(end, :)-settings.initialMoisture;
    diagnostic.maxMassBalanceAbs = max(diagnostic.maxMassBalanceAbs, max(abs(massResidual)));
    diagnostic.maxRadialMoistureIncrease = max(diagnostic.maxRadialMoistureIncrease, ...
        max(diff(acceptedMoisture, 1, 1), [], 'all'));
    diagnostic.acceptedTimePoints = diagnostic.acceptedTimePoints+numel(solutions{pieceIndex}.x);
end
finalState = evaluatePieces(solutions, endSec);
diagnostic.finalMaxMoisture = max(finalState(2:2:end-1));
diagnostic.strictlyDryAtEnd = diagnostic.finalMaxMoisture < settings.dryThreshold;
diagnostic.rhsEvaluations = rhsEvaluations;
diagnostic.radiusExtrapolationUsed = strcmp(caseName, 'Q4') && endSec > inputData.radius.time_s(end);
diagnostic.checkScope = 'accepted ode15s states; not a continuous exact-solution error bound';
if diagnostic.minMoisture < -1e-8 || ~diagnostic.positiveProperties
    error('CrossCheck:PhysicalRange', '%s 的接受状态违反物性正值或水分范围。', caseName);
end
if diagnostic.maxMassBalanceAbs > 1e-6
    error('CrossCheck:MassBalance', '%s 的离散干基质量守恒残差超限。', caseName);
end

```

```

end
if ~strcmp(caseName, 'Q1') && ~diagnostic.strictlyDryAtEnd
    error('CrossCheck:StrictDryness', '%s 的事件后原精度 maxC 没有严格小于 0.15。', caseName);
end
caseResult = struct('question', caseName, 'startedAtUtc', caseStartedAtUtc, ...
    'intervalCount', settings.intervalCount, 'eventSec', eventSec, ...
    'eventHours', eventSec/3600, 'endSec', endSec, 'dryMassKg', dryMassKg, ...
    'endTimeConvention', 'ceil(eventSec)+1; actual integration, not earliest integer-second claim', ...
    'sampleTimesSec', sampleTimesSec, 'sampleTypes', sampleTypes, 'sampleMaterialX', sampleX, ...
    'sampleRadiusM', sampleRadiusM, 'temperatureK', temperatureK, ...
    'temperatureC', temperatureK-273.15, 'moistureDryBasis', moisture, ...
    'fullMaterialX', materialX, 'fullStateBySample', sampleStates, ...
    'stateOrder', 'T0,C0,T1,C1,...,TN,CN,cumulativeDryBasisLoss', ...
    'meanMoisture', 2*cellWeights.*sampleStates(2:2:end-1, :), ...
    'cumulativeLoss', sampleStates(end, :), 'diagnostics', diagnostic, ...
    'warnings', {warningRecords}, 'warningCapture', 'last warning per segment; not all warning messages', ...
    'humanReviewStatus', 'pending');

function derivative = balanceRhs(timeSec, state)
    rhsEvaluations = rhsEvaluations+1;
    temperature = state(1:2:end-1);
    waterContent = state(2:2:end-1);
    [density, heatCapacity, conductivity, ~] = materialProperties(temperature, waterContent, caseName);
    radiusM = radiusAt(timeSec);
    [airTemperatureK, equilibriumMoisture] = environmentAt(timeSec);
    heatFlux = zeros(nodeCount+1, 1);
    waterFlux = zeros(nodeCount+1, 1);
    faceConductivity = 2*conductivity(1:end-1).*conductivity(2:end) ./ ...
        max(conductivity(1:end-1)+conductivity(2:end), realmin);
    heatFlux(2:end-1) = internalFaceX.*faceConductivity.*diff(temperature)/deltaX;
    if strcmp(caseName, 'Q1')
        moistureExponent = 0.89;
        diffusionPrefactor = 7e-9;
        thermalFactor = ones(nodeCount-1, 1);
    elseif strcmp(caseName, 'Q4')
        moistureExponent = 0.30;
        diffusionPrefactor = 4.2e-4;
        thermalFactor = exp(-3850./((temperature(1:end-1)+temperature(2:end))/2));
    else
        moistureExponent = 0.45;
        diffusionPrefactor = 2.4e-3;
        thermalFactor = exp(-3850./((temperature(1:end-1)+temperature(2:end))/2));
    end
    positiveMoisture = max(waterContent, settings.coefficientFloor);
    % E1(z)=expint(z)=-Ei(-z), 不是 Python scipy.special.expi 的同名替代。
    kirchhoffPotential = positiveMoisture.*exp(-moistureExponent./positiveMoisture) ...
        -moistureExponent*expint(moistureExponent./positiveMoisture);
    potentialDifference = diff(kirchhoffPotential);
    meanFaceMoisture = (positiveMoisture(1:end-1)+positiveMoisture(2:end))/2;
    moistureDifference = diff(positiveMoisture);
    closePair = abs(moistureDifference) < 1e-7*max(meanFaceMoisture, 1e-3);
    potentialDifference(closePair) = exp(-moistureExponent./meanFaceMoisture(closePair)) ...
        .*moistureDifference(closePair);
    % 温度因子在面上求值, 不放入势函数再差分, 以免引入虚构 Soret 项。
    waterFlux(2:end-1) = internalFaceX.*diffusionPrefactor.*thermalFactor.*potentialDifference/deltaX;
    waterFlux(end) = -settings.massTransferCoefficient*radiusM*(waterContent(end)-equilibriumMoisture);
    heatFlux(end) = -settings.heatTransferCoefficient*radiusM*(temperature(end)-airTemperatureK);
    derivative = zeros(stateCount, 1);
    derivative(1:2:end-1) = diff(heatFlux)./(radiusM^2*cellWeights.*density.*heatCapacity);
    % Q4 网格随材料同比收缩; 固定长度并另守恒干物质, 故无附加网格平流。
    derivative(2:2:end-1) = diff(waterFlux)./(radiusM^2*cellWeights);
    derivative(end) = 2*settings.massTransferCoefficient/radiusM*(waterContent(end)-equilibriumMoisture);
end

function radiusM = radiusAt(timeSec)
    if strcmp(caseName, 'Q4')
        boundedTimeSec = min(max(timeSec, inputData.radius.time_s(1)), inputData.radius.time_s(end));
        radiusM = interp1(inputData.radius.time_s, inputData.radius.radius_m, boundedTimeSec, 'linear');
    else
        radiusM = settings.initialRadiusM;
    end
end

```

```

function [airTemperatureK, equilibriumMoisture] = environmentAt(timeSec)
    % 4h 以后 50 摄氏度/0.05 为闭合假设; 不是额外实测数据。
    if timeSec > inputData.environment.time_s(end)
        airTemperatureK = settings.tailTemperatureK;
        equilibriumMoisture = settings.tailEquilibriumMoisture;
    else
        boundedTimeSec = max(timeSec, inputData.environment.time_s(1));
        airTemperatureK = interp1(inputData.environment.time_s, inputData.environment.temperature_K,
boundedTimeSec, 'linear');
        equilibriumMoisture = interp1(inputData.environment.time_s,
inputData.environment.air_moisture_kg_per_kg, boundedTimeSec, 'linear');
    end
end

function [eventValue, isTerminal, direction] = dryEvent(~, state)
    eventValue = max(state(2:2:end-1))-settings.dryThreshold;
    isTerminal = 1;
    direction = -1;
end

function recordWarning(warningText, warningId, leftSec, rightSec)
    if ~isempty(warningText)
        warningRecords{end+1} = struct('identifier', warningId, 'message', warningText, ...
            'segmentStartSec', leftSec, 'segmentTargetSec', rightSec);
        logLine(logFileId, 'WARNING %s [%s] %s', caseName, warningId, warningText);
    end
end

function [density, heatCapacity, conductivity, diffusivity] = materialProperties(temperatureK, waterContent,
caseName)
    if any(~isfinite(temperatureK) | temperatureK <= 0, 'all') || any(~isfinite(waterContent), 'all')
        error('CrossCheck:InvalidState', '遇到非有限状态或非正绝对温度。');
    end
    % 仅对 Newton 探测点的系数作正延拓, 不截断积分状态。
    positiveMoisture = max(waterContent, 1e-12);
    wetFraction = positiveMoisture./(1+positiveMoisture);
    switch caseName
        case 'Q1'
            density = 820*ones(size(waterContent));
            heatCapacity = 2600*ones(size(waterContent));
            conductivity = 0.36*ones(size(waterContent));
            diffusivity = 7e-9*exp(-0.89./positiveMoisture);
        case 'Q23'
            density = 650+128*positiveMoisture;
            heatCapacity = 1450+2736*wetFraction;
            conductivity = 0.21+0.38*wetFraction;
            diffusivity = 2.4e-3*exp(-0.45./positiveMoisture-3850./temperatureK);
        case 'Q4'
            density = 760+90*positiveMoisture;
            heatCapacity = 1850+2150*wetFraction;
            conductivity = 0.12+0.20*wetFraction;
            diffusivity = 4.2e-4*exp(-0.30./positiveMoisture-3850./temperatureK);
        otherwise
            error('CrossCheck:Question', '不支持的问题名: %s', caseName);
    end
end

function jacobianPattern = makeJacobianPattern(nodeCount)
    stateCount = 2*nodeCount+1;
    jacobianPattern = sparse(stateCount, stateCount);
    for nodeIndex = 1:nodeCount
        rowIndices = 2*nodeIndex-1:2*nodeIndex;
        for neighborIndex = max(1, nodeIndex-1):min(nodeCount, nodeIndex+1)
            columnIndices = 2*neighborIndex-1:2*neighborIndex;
            jacobianPattern(rowIndices, columnIndices) = 1;
        end
    end
    jacobianPattern(end, 2*nodeCount) = 1;
end

```

```

function sampleStates = evaluatePieces(solutions, queryTimesSec)
    sampleStates = zeros(size(solutions{1}.y, 1), numel(queryTimesSec));
    for queryIndex = 1:numel(queryTimesSec)
        queryTimeSec = queryTimesSec(queryIndex);
        foundPiece = false;
        for pieceIndex = 1:numel(solutions)
            piece = solutions{pieceIndex};
            if queryTimeSec >= piece.x(1) && queryTimeSec <= piece.x(end)
                sampleStates(:, queryIndex) = deval(piece, queryTimeSec);
                foundPiece = true;
                break;
            end
        end
        if ~foundPiece
            error('CrossCheck:DenseDomain', '请求的时刻 %.17g 不在已积分分段内。', queryTimeSec);
        end
    end
end

function sampleTable = makeSampleTable(caseResult)
    sampleCount = numel(caseResult.sampleTimesSec);
    radialCount = numel(caseResult.sampleMaterialX);
    question = repmat(string(caseResult.question), sampleCount*radialCount, 1);
    sampleType = repelem(caseResult.sampleTypes(:), radialCount);
    timeSec = repelem(caseResult.sampleTimesSec(:), radialCount);
    materialX = repmat(caseResult.sampleMaterialX(:), sampleCount, 1);
    currentRadiusM = repelem(caseResult.sampleRadiusM(:), radialCount);
    radiusM = currentRadiusM.*materialX;
    temperatureK = reshape(caseResult.temperatureK', [], 1);
    temperatureC = temperatureK-273.15;
    moistureDryBasis = reshape(caseResult.moistureDryBasis', [], 1);
    sampleTable = table(question, sampleType, timeSec, materialX, radiusM, currentRadiusM, ...
        temperatureK, temperatureC, moistureDryBasis);
end

function validateInputs(inputData)
    environmentColumns = {'time_s', 'temperature_K', 'air_moisture_kg_per_kg'};
    radiusColumns = {'time_s', 'radius_m'};
    if ~all(ismember(environmentColumns, inputData.environment.Properties.VariableNames)) || ...
        ~all(ismember(radiusColumns, inputData.radius.Properties.VariableNames))
        error('CrossCheck:InputColumns', '清洗 CSV 字段不符合正式输入接口。');
    end
    environmentValues = inputData.environment(:, environmentColumns);
    radiusValues = inputData.radius(:, radiusColumns);
    if any(~isfinite(environmentValues), 'all') || any(~isfinite(radiusValues), 'all') || ...
        any(diff(environmentValues(:, 1)) <= 0) || any(diff(radiusValues(:, 1)) <= 0) || ...
        environmentValues(1, 1) ~= 0 || radiusValues(1, 1) ~= 0 || ...
        environmentValues(end, 1) ~= 14400 || radiusValues(end, 1) ~= 259200 || ...
        any(environmentValues(:, 2) <= 0) || any(environmentValues(:, 3) < 0) || ...
        any(radiusValues(:, 2) <= 0) || abs(radiusValues(1, 2)-0.02) > 1e-14
        error('CrossCheck:InputAudit', '时间、数值范围或初始半径审计未通过。');
    end
end

function assertReached(solution, requestedEndSec)
    if abs(solution.x(end)-requestedEndSec) > 1e-7
        error('CrossCheck:SolverStopped', 'ode15s 实际终点 %.17g 未到请求终点 %.17g。', solution.x(end),
            requestedEndSec);
    end
    if any(~isfinite(solution.y), 'all')
        error('CrossCheck:NonfiniteSolution', 'ode15s 返回非有限接受状态。');
    end
end

function record = fileRecord(filePath)
    fileInfo = dir(filePath);
    record = struct('path', filePath, 'bytes', fileInfo.bytes, 'sha256', sha256File(filePath));
end

function hashText = sha256File(filePath)

```

```

fileId = fopen(filePath, 'r');
if fileId < 0
    error('CrossCheck:HashRead', '无法读取待哈希文件: %s', filePath);
end
fileCleanup = onCleanup(@() fclose(fileId)); %#ok<NASGU>
fileBytes = fread(fileId, Inf, 'uint8');
digest = java.security.MessageDigest.getInstance('SHA-256');
digest.update(typecast(fileBytes, 'int8'));
digestBytes = typecast(digest.digest(), 'uint8');
hashText = lower(reshape(dec2hex(digestBytes, 2), 1, []));
end

function writeJson(filePath, value)
fileId = fopen(filePath, 'w', 'n', 'UTF-8');
if fileId < 0
    error('CrossCheck:JsonWrite', '无法写入 JSON: %s', filePath);
end
fileCleanup = onCleanup(@() fclose(fileId)); %#ok<NASGU>
fprintf(fileId, '%s\n', jsonencode(value, 'PrettyPrint', true));
end

function logLine(fileId, messageFormat, varargin)
logText = sprintf(messageFormat, varargin{:});
fprintf('%s\n', logText);
fprintf(fileId, '%s\n', logText);
end

function timeText = utcStamp()
timeText = char(datetime('now', 'TimeZone', 'UTC', 'Format', 'yyyy-MM-dd''T''HH:mm:ss.SSS''Z'''));
end

function value = scalarOrNaN(optionalValue)
if isempty(optionalValue)
    value = NaN;
else
    value = optionalValue;
end
end
@@@CODE_BLOCK_END@@@

```

附录 C 补充表格

下列表格因版面原因编入附录,正文相应小节以编号引用;编号与正文一致,内容未作任何删改。

表 12 无量纲数与时间尺度(\$C_0=2.55\$, \$T_0=301.15\$ K)

量	问题一	问题二、三	问题四
热扩散率 α /(m ² /s)	1.6886×10^{-7}	1.4483×10^{-7}	7.8501×10^{-8}
水分扩散率 D /(m ² /s)	4.9377×10^{-9}	5.6417×10^{-9}	1.0471×10^{-9}
传热 Biot 数 Bi_h	1.3889	1.0353	1.8964
传质 Biot 数 Bi_m	3.2404	2.8360	15.2801
α / D	34.20	25.67	74.97
热扩散时间 R_0^2 / α /h	0.6580	0.7672	1.4154
水分扩散时间 R_0^2 / D /h	22.5028	19.6947	106.1119
1800 s 时的 Fo_T	0.75985	—	—
1800 s 时的 Fo_C	0.022219	—	—

表 24 模型检验汇总

检验	做法	关键结果	能证明	不能证明
----	----	------	-----	------

一 解析基准	常系数圆柱 Robin-Bessel 级数(独立重写)	节点口径最大偏差 1.80×10^{-6} K;控制 体平均口径表面 2.1×10^{-3} K 且一阶 收敛	常物性子问题上离散求解器 正确	非线性 $D(C)$ 与问题 二至四的解
二 离散质量恒等式	干物质加权均值 + 累计失水	残差 5.8×10^{-15} — 1.2×10^{-14} kg/kg	共享通量格式守恒、实现自 洽	物理预测误差;完整能量守恒
三 空间网格收敛	相邻网格在 21 个固定半径 × 整数秒上比较	$\Delta C \leq 4.7 \times 10^{-5}$ kg/kg;经验阶约 1.82—2.02	所列范围内空间误差已小	连续时空严格误差界
四 时间设置敏感性	基准容差/步长 vs 更紧容 差/更小步长	$\Delta C \leq 9.9 \times 10^{-9}$ kg/kg;事件差 $\leq 4.0 \times 10^{-6}$ s	时间推进误差远小于物理变 化	纯步长或纯容差的截断误差
五 跨实现对照	Python(生产)vs MATLAB 'ode15s'(N 40)	$\Delta T \leq 2.38 \times 10^{-5}$ K;事件差 2.30×10^{-4} s	两个独立实现无低级错误	连续全域误差界(同离散公式)
一附 口径判别(节点)	解析节点值 vs 有限体积节点 值	内部最大差 1.8×10^{-6} K	内部节点精度	表面控制体
一附 口径判别(控制体平均)	解析精确环形平均 vs 有限 体积控制体平均	表面最大差 2.1×10^{-3} K,一阶收 敛	表面为单侧控制体导致的一 阶几何效应	与节点口径互换使用

表 27 表面潜热情景包络(\$N=800\$)

潜热比例 L	问题二三达标时间 /h	相对基线	问题二三最低表面温度 /°C	问题四达标时间 /h	相对基线	问题四最低表面温度 /°C
0(本文基线)	57.4723	—	28.00	51.0906	—	28.00
0.25	58.1857	+1.24%	24.89	52.4165	+2.60%	25.70
0.50	58.9341	+2.54%	20.25	53.7620	+5.23%	21.69
1.00	60.4972	+5.26%	10.90	56.4845	+10.56%	14.55

表 31 多方法交叉验证架构与结果

层	待验证结论	相互独立的方法	实测一致性	证据位置
L1	问题三、四的达标时间	变阶 BDF/NDF(生产)·三阶 Radau IIA	三档网格事件差 1.1×10^{-6} — 5.6×10^{-8} s	'cv_solver_v1/'
L2	湿分面离散格式	Kirchhoff 势 · 调和平均 + 网 格加密	见 12.3 节表 31	'cv_solver_v1/'
L3	全模型实现	Python(生产)· MATLAB 'ode15s'(N 40)	$\Delta T \leq 2.375 \times 10^{-5}$ K;事件差 2.297×10^{-4} s	'qa/ matlab_crosscheck_20260911/'
L4	问题一热场	常系数圆柱 Robin-Bessel 级 数	节点口径 1.80×10^{-6} K;口径 判别见 10.4 节	'results/bessel_validation/'
L5	收缩的缩短效应	直接数值模拟 · $(R_0/R)^2$ 尺度折算	131.6198 h vs 129.8452 h,偏 1.37%	'threshold_scaling_v1/'
L6	三阶段分界	两段折线回归 · M-K 突变 · Fisher 最优分割 · 相对阈值	四判据落在同一区间,见 12.5 节	'series_v1/'
L7	阈值定义与最慢位置	ODE 事件 · 独立二分求根 · 全节点最大值位置证书	根差 5.82×10^{-11} s	'threshold_scaling_v1/'
L8	空间离散本身	Kirchhoff 势 · 二阶守恒中点 差分	常 D 逐位 0;变 D 观测 阶 1.99907	'method_v1/'

L9	表面水分边界闭合	冻结 1:1 映射 · 数据锚定的吸附族	锚点差 0.0; $p=1$ 右端函数逐位相同	'isotherm_closure_v1/'
L10	参数不确定性	Morris 筛选 · 单参数扫描(带注入自检)	见 13.3 节	'sensitivity_v1/'

表 32 两种湿面离散格式随网格加密的达标时间(同一模型、同一积分器)

问题	网格 N	Kirchhoff /h	调和平均 /h	差 /h	比值
问题二三	400	57.4724174	57.6868014	0.2144	1.0037
问题二三	800	57.4723298	57.5122517	0.03992	1.0007
问题二三	1600	57.4723076	57.4810663	0.008759	1.0002
问题四	400	51.0906733	51.1105476	0.01987	1.0004
问题四	800	51.0905992	51.0954705	0.004871	1.0001
问题四	1600	51.0905806	51.0917915	0.001211	1.0000

表 33 两个积分族的事件时间(同一网格、同一容差)

问题	网格 N	BDF/NDF /h	Radau IIA /h	差 /s
问题二三	800	57.4723297616	57.4723297620	1.12×10^{-6}
问题二三	1600	57.4723075468	57.4723075464	1.49×10^{-6}
问题四	800	51.0905991564	51.0905991555	3.17×10^{-6}
问题四	1600	51.0905805903	51.0905805903	5.58×10^{-8}

表 34 四类序列的阶段分界识别结果

目标序列	数据来源	判据区间 /h	对模型的意义
烘房温度	附件 1	0.6—2.4	支持"4 h 后取平台"的延拓
烘房水分浓度	附件 1	同区间	同上
预热阶段完成(中心/表面温度)	计算曲线	1.0—2.9	问题一的 1800 s 窗口落在预热段内;问题二的 3 h 展示窗恰好跨越预热→恒温干燥的转变
半径主要收缩完成	附件 2	4.5—13.5	问题四的 51 h 终点远在收缩终止之后,且不需要半径外推

表 29 五处修正与创新的定位与证据

编号	修正/创新	性质	定量证据	论文落点
一	题给密度与几何的相容性修正	对题设的修正	必要性半径界 1.2112 cm;字面模型 72 h 最多容纳 97.83%	5.7.4(c)、9.1、13.7
二	表面蒸发的能量闭合(潜热包络)	物理补全 + 情景包络	潜热/显热 ≈ 21.6 ; $L=1$ 时 +5.3% / +10.6%	11.2、13.4
三	Kirchhoff 浓度势	数值方法设计	调和平均在 $N=1600$ 仍偏 8.8×10^{-3} h;两格式观测阶 1.999	10.2.3、11.3
四	材料坐标移动域 + 干骨架守恒	建模方法设计	收缩使时长缩短 60.65%;独立 R^2 折算印证到 1.37%	5.5、9.5、11.4
五	数据锚定的吸附闭合	物理闭合修正	烘房 RH = 0.607 强制 $a_w(0.05)=0.607$;族包络 +0—13.9% / +0—8.3%	11.5、13.4

附录 D 公式总表(完整性与数量核对用)

表 37 公式总表(共 76 式,含 5 条补充推导式)

编号	名称与用途	自变量	因变量	单位	来源	
(1)	四问通用控制方程(材料坐标形式)	x, t	T, C	K/s、1/s	本文(由 (16) 归纳)	
(2)	薄壳几何与控制体	r, dr	$dV, A(r)$	m ³ 、m ²	本文定义	
(3)	干基与湿基含水率换算	m_w, m_d	C, w	kg/kg、1	本文定义	
(4)	初始干物质密度标定	$\rho_{\text{eff}}(C_0), C_0$	ρ_{d0}	kg/m ³	本文定义	
(5)	干物质质量守恒	t, r	ρ_d, u	kg/(m ² ·s)	本文推导	
(6)	水分总质量守恒	t, r	$\rho_d C, j_w$	kg/(m ² ·s)	本文推导	
(7)	干基含水率演化方程	t, r	C	1/s	由 (6)- $C \times (5)$ 推导	
(8)	傅里叶导热定律	$\partial T / \partial r$	q_r	W/m ²	本构(题设物理定律)	
(9)	有效菲克扩散本构(干基)	$\partial C / \partial r$	j_w	kg/(m ² ·s)	本构(题设物理定律)	
(10)	热方程(有效显热闭合, 固定半径)	t, r	T	K/s	本文主控方程	
(11)	水分方程(固定半径)	t, r	C	1/s	本文主控方程	
(12)	变系数散度展开式	t, r	T	K/s	本文推导	
(13)	材料坐标变换	$r, R(t)$	x, F	1	本文定义	
(14)	坐标变换下的导数关系	x, t	$\partial_t f, \partial_r f$	1/s、1/m	链式法则推导	
(15)	同比径向收缩与干骨架守恒	$r, R(t)$	u, ρ_d	m/s、kg/m ³	本文推导(含守恒论证)	
(16)	变换后的统一方程	x, t	T, C	K/s、1/s	由 (14) 代入 (10)(11)	
(17)	附录 4 与附录 3 的扩散系数比	C	D_4 / D_{23}	1	由 (31)(35) 相除	
(18)	初始条件	—	T_0, C_0	K、kg/kg	题给	
(19)	中心对称条件与算子极限	x	$\partial_x T, \partial_x C$	K/m、1/m	本文推导	
(20)	完整表面能量收支(含辐射与潜热)	T_s, T_w, j_n	热流平衡	W/m ²	本文推导(情景用)	
(21)	表面热边界(基线, 不含辐射与潜热)	T_s, T_∞	$\partial_r T$	W/m ²	题给 h	
(22)	表面热边界的材料坐标形式	T_s, T_∞	$\partial_x T$	W/m ²	由 (21) 变换	
(23)	表面水分边界(基线)	C_s, C_{eq}	$\partial_r C$	kg/(m ² ·s)	题给 β	
(24)	表面水分边界的材料坐标形式	C_s, C_{eq}	$\partial_x C$	kg/(m ² ·s)	由 (23) 变换	
(25)	真实水质量通量与等效通量换算	C_s, C_{eq}	S_{j_w}	—sS	kg/(m ² ·s)	本文推导
(26)	问题一常热物性	—	ρ, c_p, k	kg/m ³ 、J/(kg·K)、W/(m·K)	题给附录 2	
(27)	问题一水分扩散系数	C	D_1	m ² /s	题给附录 2	
(28)	问题二三密度	C	ρ_{23}	kg/m ³	题给附录 3	
(29)	问题二三比热容	C	$c_{p,23}$	J/(kg·K)	题给附录 3	
(30)	问题二三导热系数	C	k_{23}	W/(m·K)	题给附录 3	
(31)	问题二三水分扩散系数	C, T	D_{23}	m ² /s	题给附录 3	
(32)	问题四密度	C	ρ_4	kg/m ³	题给附录 4	
(33)	问题四比热容	C	$c_{p,4}$	J/(kg·K)	题给附录 4	

(34)	问题四导热系数	C	k_4	W/(m·K)	题给附录 4	
(35)	问题四水分扩散系数	C, T	D_4	m²/s	题给附录 4	
(36)	比热容的质量加权混合形式	m_d, m_w	c_p	J/(kg·K)	本文推导(解释性)	
(37)	扩散系数的对数导数与单调性	C, T	$\partial \ln D$	1/K、kg/kg	本文推导	
(38)	无量纲数与特征时间	h, β, k, D, α	Bi_h, Bi_m, Fo	1	本文定义	
(39)	问题一热方程(常数系数)	t, r	T	K/s	由 (10) 取常数物性	
(40)	问题一水分方程	t, r	C	1/s	由 (11) 代入 (27)	
(41)	问题二热方程	t, r	T	K/s	由 (10) 代入 (28)(29)	
(42)	问题二水分方程	t, r	C	1/s	由 (11) 代入 (31)	
(43)	全域达标判据	t	$M(t), t_*$	kg/kg、s	本文定义	
(44)	严格可行报告时刻的取整规则	t_*	$\hat{n}, t_{\text{rep}}$	1、h	本文定义	
(45)	问题四热方程与水分方程	x, t	T, C	K/s、1/s	由 (16) 代入 (32)–(35)	
(46)	问题四有效热容量与干骨架守恒	$C, R(t)$	B_4, ρ_d	J/(m³·K)、kg/m³	由 (15)(33) 得	
(47)	对偶控制体网格与权重	x	x_i, w_i	1	本文定义	
(48)	内部面热通量(调和平均)	T_i, T_{i+1}	$G_{i+1/2}^T$	W/m	本文离散	
(49)	Kirchhoff 浓度势及其导数	C	Φ, Φ'	—、1	本文推导	
(50)	内部面湿通量 (Kirchhoff 势)	$C_i, C_{i+1}, \bar{\circ}$	$G_{i+1/2}^C$	m²·(kg/kg)/s	由 (49) 得	
(51)	表面与中心面通量	C_N, T_N	$G_{N+1/2}^C, G_N^T$	同 (48)(50)	由 (22)(24) 得	
(52)	半离散常微分方程组	$G^T, G^C, w_i, \bar{\circ} \cdot T_i, \bar{\circ} \cdot C$		K/s、1/s	由守恒式离散	
(53)	累计失水与加权平均含水率	C_i, C_N	$\ell, \bar{\circ} C$	kg/kg	本文定义	
(54)	离散质量恒等式	$\bar{\circ} C, \ell$	残差	kg/kg	由 (52)(53) 推出	
(55)	物性对含水率的导数	C	$\rho_C, c_{p,C}, k_C$	各物性单位/C	由 (28)–(30) 求导	
(56)	扩散系数对状态的导数	C, T	D_C, D_T	m²/s 每单位	由 (31) 求导	
(57)	调和平均面对导热系数的导数	k_i, k_{i+1}	∂k_H	1	由 (48) 求导	
(58)	面湿通量对相邻含水率的导数	C_i, C_{i+1}	∂G^C	m²/s	由 (50) 求导	
(59)	容量交叉项(最易遗漏)	$\bar{\circ} \cdot T_i, B_i, B$	$\partial \bar{\circ} \cdot T_i / \partial$	1/s	由 (52) 求导	
(60)	圆柱 Robin–Bessel 级数解	r, t	T	K	本文独立重写(解析基准)	
(61)	级数系数	μ_n, Bi_h	A_n	K	由初值正交展开	
(62)	字面湿密度的干密度上界	C	ρ_d	kg/m³	由 (32) 推导	
(63)	必要性半径界	ρ_{d0}	$R_{\min}$	m	本文推导	
(64)	字面模型可容纳的干质量比例	$R(72 \text{ h})$	比例	1	本文推导	
(65)	含潜热的表面能量平衡	T_s, C_s	热流平衡	W/m²	本文推导(式 (20) 的基线扩展)	
(66)	常规面通量近似(对照用)	C_i, C_{i+1}	G^C	m²·(kg/kg)/s	本文列出(对照)	

(67)	收缩时间轴的 R^2 尺度折算	$R(t), t_*$	τ_{eq}	h	本文推导	
(68)	湿空气含湿量与蒸汽分 压换算	p, Y, T	p_v	Pa	本文推导(依据湿空气 热力学)	
(69)	被题面数据强制的水活 度锚点	C_{eq}, T_∞	a_w	1	由附件 1 与 $C_{eq} = 0.05$ 推出	
(70)	单参数等温线族	C, p	$a_w(C)$	1	本文构造	
(71)	吸附闭合的表面驱动缩 放	C_s, C_{eq}, p	$g_{\text{表面}}, K_{\text{eff}}$	kg/(m ² ·s)、1	本文构造	

数量核对:题给经验式 **9 条**(式 (26)—(35),其中 (26) 含三个常数);守恒与演化 **3 条**((5)—(7));本构 **2 条**((8)、(9));主控方程及其变换 **7 条**((10)—(16));初边值 **8 条**((18)—(25));数值离散 **10 条**((47)—(54)、(66));算法导数 **5 条**((55)—(59));解析基准 **2 条**((60)、(61));判据与取整 **2 条**((43)、(44));修正与创新 **13 条**((17)、(36)—(38)、(62)—(65)、(67)—(71));其余为定义与几何式。合计 **76 式**(71 条主式 + 补充推导式 (47a)(47b)(47c)(48a)(50a),用于展示守恒性、调和平均导数与 Kirchhoff 势构造的完整推导),与正文编号一一对应,无缺号、无重号。